

ggNetView manual documentation

Yue Liu

2026-05-21

Contents

1	ggNetView manual documentation	5
1.1	Introduction	5
1.2	Installation	6
1.3	Citation	6
1.4	Source Code	7
1.5	Contact	7
2	Build graph object	9
2.1	Build graph from matrix	9
2.2	Build graph from data frame	12
2.3	Build graph from data frame with module	15
2.4	Build graph from adjacency matrix	17
2.5	Build graph from adjacency matrix with module information . .	19
2.6	Build graph from double matrix	20
2.7	Build graph from igraph object	22
2.8	Build graph from WGCNA	24
2.9	Build graph from multi-graph	25
3	Random Matrix Theory (RMT)–based random network	29
3.1	Random Matrix Theory (RMT)	29
3.2	Example data	30
3.3	Compute RMT threshold	30
3.4	Build graph from matrix based on RMT-threshold	42

4	Get network information	45
4.1	Full-network information	47
4.2	Sub-network (modularity) information	48
4.3	Sub-network (sample) information	51
5	Extract subgraph	55
5.1	Extract subgraph by module	57
5.2	Extract subgraph by sample	61
6	Network layout	67
6.1	Full graph layout	69
6.2	Sub graph layout	122
7	Network topology information	153
7.1	Get network topology information	155
7.2	Get network topology information with matrix	157
7.3	Get network topology information from sample	159
7.4	Get network topology information by parallel	204
7.5	Get network topology information with matrix by parallel	206
7.6	Integrated Value of Influence (IVI)	208
8	Multi-Network and comparison	211
8.1	Comparison of multi-sample networks	213
8.2	multi network (4 groups)	229
8.3	multi network (8 groups)	235
9	Network & Environment	243
9.1	Network Environment	243
10	Multi-omics network analysis	245
10.1	Multi-omics	245
11	Gallery of Reproducible Examples	247
11.1	Water	247
11.2	WGCNA	247

Chapter 1

ggNetView manual documentation

1.1 Introduction

Network analysis has been widely applied in the **life sciences, microbiology, ecology, and agronomy** to quantify interactions among **genes, proteins, metabolites, microorganisms, and environmental factors**. These interactions collectively shape the functioning of biological and ecological systems. Despite its broad adoption, existing tools often face limitations in flexibility, customization, reproducibility, and the generation of publication-ready figures.

To address these challenges, **ggNetView** was developed as an **R package** that integrates **ggplot2, ggraph, and tidygraph** within the **Grammar of Graphics framework**, enabling fully reproducible and highly customizable network visualizations. The package provides deterministic layout algorithms, comprehensive topological analyses, modular coloring schemes, hierarchical annotations, and consistent theming, ensuring standardized and interpretable graphical output.

Although initially designed for applications in **soil science and microbial ecology**, **ggNetView** is broadly applicable to network analyses in **molecular biology**, including **WGCNA and protein–protein interaction (PPI) networks**. By lowering technical barriers in network construction and visualization, **ggNetView** enables researchers across disciplines to efficiently produce reproducible, publication-quality network figures.

1.2 Installation

Step1: install CRAN dependencies

```
cran_pkgs <- c(
  "boot", "dplyr", "FNN", "future", "future.apply",
  "ggforce", "ggnewscale", "ggplot2", "gggraph", "ggrepel",
  "Hmisc", "huge", "igraph", "magrittr", "MASS",
  "Matrix", "patchwork", "progressr", "psych", "purrr",
  "qgraph", "Rcpp", "RcppArmadillo", "readr", "rlang",
  "scales", "scatterpie", "stringr", "tibble", "tidygraph",
  "tidyr", "vegan", "VGAM", "WGCNA"
)

new_pkgs <- cran_pkgs[!cran_pkgs %in% installed.packages()[, "Package"]]
if (length(new_pkgs)) install.packages(new_pkgs)
```

Step2: (optional) install suggested packages

These packages are not required for the core functionality, but enable additional features (e.g. dynamic tree cut, node influence, vignettes, tests):

```
install.packages(c("dynamicTreeCut", "influential",
  "knitr", "rmarkdown",
  "RobustRankAggreg", "testthat"))
```

Step3: install ggNetView from GitHub

```
# install.packages("devtools")
devtools::install_github("Jiawang1209/ggNetView")
```

1.3 Citation

If you use ggNetView in your research, please cite:

Yue Liu, Chao Wang (2025). ggNetView: An R package for complex biological and ecological
<https://github.com/Jiawang1209/ggNetView>

1.4 Source Code

The source code for ggNetView is available in the ggNetView repository.

<https://github.com/Jiawang1209/ggNetView>

1.5 Contact

- Email: Jiawang1209@163.com
- Github: <https://github.com/Jiawang1209/ggNetView>
- ggNetView-manual: <https://jiawang1209.github.io/ggNetView-manual/>

Chapter 2

Build graph object

Basic workflow of **ggNetView**

1. Build a **graph object**
 2. Understand and manipulate the **graph object**
 3. Visualize the network using **layout algorithms**
 4. Retrieve **topology** of network
-

Load R Package

```
library(tidyverse)    # data manipulation / piping utilities
library(ggNetView)    # graph builders + info-extraction helpers
library(igraph)       # igraph object
```

2.1 Build graph from matrix

Example data

```
# ---- Built-in example datasets shipped with ggNetView ----

# Raw ASV/OTU abundance table (rows = ASVs/OTUs, cols = samples; integer counts).
data("otu_tab")
dim(otu_tab)
```

```
## [1] 2859 18
```

```
otu_tab[1:5, 1:5]
```

```
##           K01  K02  K03  K04  K05
## ASV_1 1113 1968  816 1372 1062
## ASV_2 1922 1227 2355 2218 2885
## ASV_3  568  460  899  902 1226
## ASV_4 1433  400  535  759 1287
## ASV_6  882  673  819  888 1475
```

```
# Same table after rarefaction (sequencing depth normalised across samples).
data("otu_rare")
dim(otu_rare)
```

```
## [1] 2859 18
```

```
otu_rare[1:5, 1:5]
```

```
##           K01  K02  K03  K04  K05
## ASV_1  992 1636  604 1084  806
## ASV_2 1725 1018 1814 1743 2196
## ASV_3  520  389  687  701  932
## ASV_4 1280  328  425  580 1004
## ASV_6  794  557  633  706 1142
```

```
# Rarefied table converted to relative abundance - most common input for correlation n
data("otu_rare_relative")
dim(otu_rare_relative)
```

```
## [1] 2859 18
```

```
otu_rare_relative[1:5, 1:5]
```

```
##           K01           K02           K03           K04           K05
## ASV_1 0.03306667 0.05453333 0.02013333 0.03613333 0.02686667
## ASV_2 0.05750000 0.03393333 0.06046667 0.05810000 0.07320000
## ASV_3 0.01733333 0.01296667 0.02290000 0.02336667 0.03106667
## ASV_4 0.04266667 0.01093333 0.01416667 0.01933333 0.03346667
## ASV_6 0.02646667 0.01856667 0.02110000 0.02353333 0.03806667
```

```
# Taxonomic annotation for the ASVs/OTUs above.
# First column must be the ASV/OTU ID (matches rownames of the abundance tables).
data("tax_tab")
dim(tax_tab)
```

```
## [1] 2859      8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
##   OTUID Kingdom Phylum      Class      Order
##   <chr>  <chr>    <chr>      <chr>      <chr>
## 1 ASV_2  Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia  Spartobacteria  Unassigned
## 3 ASV_31 Bacteria Actinobacteria  Actinobacteria  Actinomycetales
## 4 ASV_27 Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned      Unassigned  Unassigned
```

Build graph object

```
# ---- Build a co-occurrence network directly from an abundance matrix ----
# Internal pipeline: transform -> correlation -> p-value adjustment ->
# edge filtering -> community detection -> attach taxonomy.
graph_obj <- build_graph_from_mat(
  mat           = otu_rare_relative, # variables x samples numeric matrix
  transform.method = "none",         # input is already relative abundance, no extra transform
  r.threshold    = 0.7,              # |r| cutoff (subjective; consider ggNetView_RMT)
  p.threshold    = 0.05,             # adjusted p-value cutoff
  method         = "WGCNA",          # correlation backend: WGCNA::corAndPvalue
  cor.method     = "pearson",         # Pearson correlation
  proc           = "bonferroni",      # multiple-testing correction
  module.method  = "Fast_greedy",     # community detection algorithm
  node_annotation = tax_tab,          # taxonomy joined onto each node by name
  top_modules    = 15,               # keep top-15 modules; rest -> "Others"
  seed           = 1115              # fix RNG for reproducibility
)

# Inspect node/edge counts and the columns now attached to nodes:
# modularity, modularity2, modularity3, Modularity, Degree, Strength + taxonomy.
graph_obj
```

```
## # A tbl_graph: 213 nodes and 844 edges
## #
```

```
## # An undirected simple graph with 29 components
## #
## # Node Data: 213 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_649 5          5          5          5          27      26.5 Bacter~
## 2 ASV_705 5          5          5          5          27      26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 4 ASV_13~ 5          5          5          5          27      26.5 Bacter~
## 5 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 6 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 7 ASV_24~ 5          5          5          5          27      26.5 Bacter~
## 8 ASV_25~ 5          5          5          5          27      26.4 Bacter~
## 9 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## 10 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## # i 203 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 844 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1 194    195 0.959      0.959 Positive
## 2 185    208 0.954      0.954 Positive
## 3 185    213 0.957      0.957 Positive
## # i 841 more rows
```

2.2 Build graph from data frame

Example data

```
# Edge-list style example data (protein-protein interactions).
data("ppi_example")

# `df`: edge list - typically 3 columns: node1 / node2 / weight (weight optional).
df = ppi_example$ppi
head(df)

##   from to    weight
## 1  A1 D40 9.306533
## 2  A2 D39 11.783920
```

```
## 3   A3 D38 23.005025
## 4   A4 D37  7.412060
## 5   A5 D36 18.778894
## 6   A6 D35 16.592965
```

```
# `node_annotation`: per-node metadata; first column = node ID.
node_annotation = ppi_example$annotation
head(node_annotation)
```

```
##   node group
## 1   A1      A
## 2   A2      A
## 3   A3      A
## 4   A4      A
## 5   A5      A
## 6   A6      A
```

Build graph object

```
# ---- Build a tbl_graph directly from an edge-list data.frame ----
# No correlation step - edges are taken as-is from `df`.
# Community detection still runs on the resulting graph.
graph_obj_from_df <- build_graph_from_df(
  df           = df,                # edge list (node1 | node2 | weight)
  node_annotation = node_annotation, # per-node metadata, joined by node name
  directed       = F,               # undirected network
  module.method  = "Fast_greedy",   # community detection algorithm
  top_modules    = 15,              # keep top-15 modules; rest -> "Others"
  seed           = 1115             # fix RNG for reproducibility
)

graph_obj_from_df
```

```
## # A tbl_graph: 100 nodes and 50 edges
## #
## # An unrooted forest with 50 trees
## #
## # Node Data: 100 x 9 (active)
##   name  group modularity modularity2 modularity3 Modularity Degree Segree
##   <chr> <chr> <fct>      <fct>      <chr>      <fct>      <dbl>  <dbl>
## 1 C13   C     1          1          1          1          1      1
## 2 C28   C     1          1          1          1          1      1
## 3 C2     C    10         10         10         10         1      1
## 4 D9     D    10         10         10         10         1      1
```

```
## 5 A3 A 11 11 11 11 1 1
## 6 D38 D 11 11 11 11 1 1
## 7 B12 B 12 12 12 12 1 1
## 8 D19 D 12 12 12 12 1 1
## 9 A1 A 13 13 13 13 1 1
## 10 D40 D 13 13 13 13 1 1
## # i 90 more rows
## # i 1 more variable: Strength <dbl>
## #
## # Edge Data: 50 x 4
## from to weight correlation
## <int> <int> <dbl> <dbl>
## 1 9 10 45.2 45.2
## 2 15 16 50.6 50.6
## 3 5 6 37.8 37.8
## # i 47 more rows

# ---- Same builder, but WITHOUT a node-annotation table ----
# Useful when only an edge list is available and no extra metadata to attach.
graph_obj_from_df2 <- build_graph_from_df(
  df = df, # edge list
  node_annotation = NULL, # skip the annotation join entirely
  directed = F, # undirected network
  module.method = "Fast_greedy", # community detection algorithm
  top_modules = 15, # keep top-15 modules; rest -> "Others"
  seed = 1115 # fix RNG for reproducibility
)

graph_obj_from_df2

## # A tbl_graph: 100 nodes and 50 edges
## #
## # An unrooted forest with 50 trees
## #
## # Node Data: 100 x 8 (active)
## name modularity modularity2 modularity3 Modularity Degree Segree Strength
## <chr> <fct> <fct> <chr> <fct> <dbl> <dbl> <dbl>
## 1 C13 1 1 1 1 1 1 26.7
## 2 C28 1 1 1 1 1 1 26.7
## 3 C2 10 10 10 10 1 1 37.4
## 4 D9 10 10 10 10 1 1 37.4
## 5 A3 11 11 11 11 1 1 37.8
## 6 D38 11 11 11 11 1 1 37.8
## 7 B12 12 12 12 12 1 1 41.3
## 8 D19 12 12 12 12 1 1 41.3
```

```
## 9 A1      13      13      13      13      1      1      45.2
## 10 D40    13      13      13      13      1      1      45.2
## # i 90 more rows
## #
## # Edge Data: 50 x 4
##   from    to weight correlation
##   <int> <int> <dbl>         <dbl>
## 1     9    10  45.2          45.2
## 2    15    16  50.6          50.6
## 3     5     6  37.8          37.8
## # i 47 more rows
```

2.3 Build graph from data frame with module

Example data

```
# Same shape as ppi_example, but `annotation` already contains a `Modularity`
# column (module assignments computed elsewhere - we will NOT recompute them).
data("ppi_module")

df = ppi_module$ppi
head(df)
```

```
##   from to    weight
## 1  A1 D40  9.306533
## 2  A2 D39 11.783920
## 3  A3 D38 23.005025
## 4  A4 D37  7.412060
## 5  A5 D36 18.778894
## 6  A6 D35 16.592965
```

```
node_annotation = ppi_module$annotation
head(node_annotation) # NOTE: must contain a `Modularity` column
```

```
##   node Modularity
## 1   A1          A
## 2   A2          A
## 3   A3          A
## 4   A4          A
## 5   A5          A
## 6   A6          A
```

Build graph object

```
# ---- `_module` variant: keep pre-computed module assignments ----
# Use this when you already have modules (e.g. from a previous WGCNA run,
# expert curation, biological prior). Internal community detection is skipped.
graph_obj_from_module <- build_graph_from_module(
  df           = df,                # edge list
  node_annotation = node_annotation, # MUST contain a `Modularity` column
  directed      = F,                # undirected network
  top_modules    = 15,              # keep top-15 modules; rest -> "Others"
  seed          = 1115              # fix RNG for reproducibility
)

graph_obj_from_module
```

```
## # A tbl_graph: 100 nodes and 50 edges
## #
## # An unrooted forest with 50 trees
## #
## # Node Data: 100 x 7 (active)
##   name Modularity modularity2 modularity3 Degree Segree Strength
##   <chr> <ord>      <ord>      <chr>      <dbl>  <dbl>  <dbl>
## 1 D1    D          D          D          1      1      37.1
## 2 D2    D          D          D          1      1      63.9
## 3 D3    D          D          D          1      1      54.6
## 4 D4    D          D          D          1      1      61.7
## 5 D5    D          D          D          1      1      36.1
## 6 D6    D          D          D          1      1      71.0
## 7 D7    D          D          D          1      1      27.9
## 8 D8    D          D          D          1      1      34.6
## 9 D9    D          D          D          1      1      37.4
## 10 D10  D          D          D          1      1      63.2
## # i 90 more rows
## #
## # Edge Data: 50 x 4
##   from to weight correlation
##   <int> <int> <dbl>      <dbl>
## 1    40   91  45.2        45.2
## 2    39   92  50.6        50.6
## 3    38   93  37.8        37.8
## # i 47 more rows
```

2.4 Build graph from adjacency matrix

Example data

```
# Square adjacency matrix (already-filtered weights for every node pair).
# Useful when the network was built externally (NetCoMi, FlashWeave, ...).
data("adjacency_matrix_example")
dim(adjacency_matrix_example)
```

```
## [1] 2859 2859
```

```
adjacency_matrix_example[1:5, 1:5]
```

```
##      ASV_1      ASV_2 ASV_3 ASV_4      ASV_6
## ASV_1      0 0.0000000      0      0 0.0000000
## ASV_2      0 0.0000000      0      0 0.8947427
## ASV_3      0 0.0000000      0      0 0.0000000
## ASV_4      0 0.0000000      0      0 0.0000000
## ASV_6      0 0.8947427      0      0 0.0000000
```

```
# Reuse the same taxonomy table as node annotation.
data("tax_tab")
dim(tax_tab)
```

```
## [1] 2859      8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
##   OTUID Kingdom Phylum      Class      Order
##   <chr>  <chr>    <chr>      <chr>      <chr>
## 1 ASV_2  Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia  Spartobacteria  Unassigned
## 3 ASV_31 Bacteria Actinobacteria  Actinobacteria  Actinomycetales
## 4 ASV_27 Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned      Unassigned  Unassigned
```

Build graph object

```
# ---- Build a tbl_graph from a pre-computed adjacency matrix ----
# Community detection runs internally because we have weights but no modules yet.
graph_obj_adj <- build_graph_from_adj_mat(
  adjacency_matrix = adjacency_matrix_example, # square weighted adjacency matrix
  module.method    = "Fast_greedy",          # community detection algorithm
  node_annotation  = tax_tab,                 # per-node metadata, joined by node name
  top_modules      = 15,                     # keep top-15 modules; rest -> "Others"
  seed             = 1115                    # fix RNG for reproducibility
)

graph_obj_adj
```

```
## # A tbl_graph: 2049 nodes and 9602 edges
## #
## # An undirected simple graph with 100 components
## #
## # Node Data: 2,049 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>   <dbl> <chr>
## 1 ASV_916 1          1          1          1          58      50.5 Bacter~
## 2 ASV_777 1          1          1          1          58      48.7 Bacter~
## 3 ASV_606 1          1          1          1          55      45.8 Bacter~
## 4 ASV_740 1          1          1          1          54      47.2 Bacter~
## 5 ASV_14~ 1          1          1          1          54      44.5 Bacter~
## 6 ASV_23~ 1          1          1          1          54      47.4 Bacter~
## 7 ASV_15~ 1          1          1          1          52      45.3 Bacter~
## 8 ASV_24~ 1          1          1          1          52      43.0 Bacter~
## 9 ASV_19~ 1          1          1          1          52      43.0 Bacter~
## 10 ASV_568 1          1          1          1          51      45.1 Bacter~
## # i 2,039 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 9,602 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1 1771 1825 0.793      0.793 Positive
## 2 594 597 0.895      0.895 Positive
## 3 588 597 0.864      0.864 Positive
## # i 9,599 more rows
```

2.5 Build graph from adjacency matrix with module information

```
# Add a pre-existing module assignment to the annotation table.
# (Demo only) treat the `Kingdom` column as the module label;
# in real use, supply your actual module assignments.
```

```
tax_tab_module <- tax_tab %>%
  dplyr::mutate(Modularity = Kingdom)
```

```
# ---- `_module` variant for adjacency input ----
# REQUIRES `node_annotation` to contain a `Modularity` column.
# Internal community detection is skipped; supplied modules are used directly.
graph_obj_adj_module <- build_graph_from_adj_mat_module(
  adjacency_matrix = adjacency_matrix_example, # square weighted adjacency matrix
  node_annotation = tax_tab_module,             # MUST contain a `Modularity` column
  top_modules     = 15,                         # keep top-15 modules; rest -> "Others"
  seed           = 1115                         # fix RNG for reproducibility
)
```

```
## The max module in network is 3 we use the 3 modules for next analysis
```

```
graph_obj_adj_module
```

```
## # A tbl_graph: 2049 nodes and 9602 edges
```

```
## #
```

```
## # An undirected simple graph with 100 components
```

```
## #
```

```
## # Node Data: 2,049 x 14 (active)
```

	name	Kingdom	Phylum	Class	Order	Family	Genus	Species	Modularity	modularity2
	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<ord>	<ord>
## 1	ASV_9~	Bacter~	Prote~	Gamm~	Unas~	Unass~	Unas~	Unassi~	Bacteria	Bacteria
## 2	ASV_7~	Bacter~	Acido~	Acid~	Unas~	Unass~	Gp6	Unassi~	Bacteria	Bacteria
## 3	ASV_6~	Bacter~	Actin~	Acti~	Acti~	Nocar~	Marm~	Unassi~	Bacteria	Bacteria
## 4	ASV_3~	Bacter~	Prote~	Beta~	Unas~	Unass~	Unas~	Unassi~	Bacteria	Bacteria
## 5	ASV_7~	Bacter~	Acido~	Acid~	Unas~	Unass~	Gp16	Unassi~	Bacteria	Bacteria
## 6	ASV_1~	Bacter~	Actin~	Acti~	Unas~	Unass~	Unas~	Unassi~	Bacteria	Bacteria
## 7	ASV_2~	Bacter~	Prote~	Alph~	Rhiz~	Hyphe~	Rhod~	Unassi~	Bacteria	Bacteria
## 8	ASV_25	Bacter~	Verru~	Spar~	Unas~	Unass~	Spar~	Unassi~	Bacteria	Bacteria
## 9	ASV_7~	Bacter~	Actin~	Acti~	Acti~	Cellu~	Cell~	Cellul~	Bacteria	Bacteria
## 10	ASV_1~	Bacter~	Acido~	Acid~	Unas~	Unass~	Gp3	Unassi~	Bacteria	Bacteria

```
## # i 2,039 more rows
```

```
## # i 4 more variables: modularity3 <chr>, Degree <dbl>, Segree <dbl>,
```

```
## # Strength <dbl>
## #
## # Edge Data: 9,602 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>         <dbl> <chr>
## 1  1211  1492  0.793           0.793 Positive
## 2   323  2013  0.895           0.895 Positive
## 3   267  2013  0.864           0.864 Positive
## # i 9,599 more rows
```

2.6 Build graph from double matrix

Example data

```
# Two abundance matrices on the SAME samples but DIFFERENT feature sets,
# typically used for cross-kingdom networks (e.g. bacteria <-> fungi).
```

```
# Bacterial ASV table.
```

```
data("BASV_tab")
dim(BASV_tab)
```

```
## [1] 50 10
```

```
BASV_tab[1:5, 1:5]
```

```
##           Sample1 Sample2 Sample3 Sample4 Sample5
## BASV1  7.769249 14.676091  6.822246 11.806957 12.086254
## BASV2 10.771846  7.749385 14.966987  7.486507 12.645461
## BASV3 11.746937 10.023547 10.025906 11.967562  8.748440
## BASV4  8.210859  4.408296 10.070862  8.954202 14.639298
## BASV5  9.833271  9.776280 10.465463  8.380012  8.905738
```

```
# Fungal ASV table.
```

```
data("FASV_tab")
dim(FASV_tab)
```

```
## [1] 50 10
```

```
FASV_tab[1:5, 1:5]
```

```
##           Sample1  Sample2  Sample3  Sample4  Sample5
## FASV1  8.195641  7.715472 13.247401 10.419618 11.961410
## FASV2 10.032701 12.427388  7.693471 10.711077 12.157527
## FASV3 10.109549 11.891917  9.607849  8.256637  9.989264
## FASV4  9.817056  8.038730  5.519155  7.601228 12.688531
## FASV5 13.370263  9.795554 13.639041 11.071635  8.176218
```

```
# Combined node-annotation table covering BOTH feature sets above.
data("double_mat_node_df")
dim(double_mat_node_df)
```

```
## [1] 100  2
```

```
head(double_mat_node_df)
```

```
##   name      type
## 1 BASV1 Bacterial
## 2 BASV2 Bacterial
## 3 BASV3 Bacterial
## 4 BASV4 Bacterial
## 5 BASV5 Bacterial
## 6 BASV6 Bacterial
```

Build graph object

```
# ---- Cross-domain (bipartite-style) network ----
# Correlations are computed BETWEEN features in mat1 and features in mat2
# (no within-mat1 / within-mat2 edges), then thresholded, modularised,
# and joined with the node-annotation table.
graph_obj_double_mat <- build_graph_from_double_mat(
  mat1      = BASV_tab,          # set A (e.g. bacteria), variables x samples
  mat2      = FASV_tab,          # set B (e.g. fungi),   variables x samples
  module.method = "Fast_greedy", # community detection algorithm
  node_annotation = double_mat_node_df, # node metadata covering BOTH mat1 and mat2
  top_modules   = 15,            # keep top-15 modules; rest -> "Others"
  seed          = 1115           # fix RNG for reproducibility
)
```

```
## The max module in network is 4 we use the 4 modules for next analysis
```

```
graph_obj_double_mat
```

```
## # A tbl_graph: 100 nodes and 2500 edges
## #
## # A bipartite simple graph with 1 component
## #
## # Node Data: 100 x 9 (active)
##   name      type      modularity modularity2 modularity3 Modularity Degree Segree
##   <chr>    <chr>    <fct>      <fct>      <chr>      <fct>      <dbl>  <dbl>
## 1 BASV3   Bacterial 1          1          1          1          50    50
## 2 BASV6   Bacterial 1          1          1          1          50    50
## 3 BASV8   Bacterial 1          1          1          1          50    50
## 4 BASV13  Bacterial 1          1          1          1          50    50
## 5 BASV17  Bacterial 1          1          1          1          50    50
## 6 BASV19  Bacterial 1          1          1          1          50    50
## 7 BASV25  Bacterial 1          1          1          1          50    50
## 8 BASV26  Bacterial 1          1          1          1          50    50
## 9 BASV27  Bacterial 1          1          1          1          50    50
## 10 BASV31 Bacterial 1          1          1          1          50    50
## # i 90 more rows
## # i 1 more variable: Strength <dbl>
## #
## # Edge Data: 2,500 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1    15    57  0.338      -0.338 Negative
## 2    57    90  0.648       0.648 Positive
## 3    16    57  0.162       0.162 Positive
## # i 2,497 more rows
```

2.7 Build graph from igraph object

```
# Load the PPI example dataset (edge list + node annotation).
data("ppi_example")

# Build a native igraph object directly from the edge list.
# This is the format users coming from an igraph workflow typically already have.
igraph_object <- igraph::graph_from_data_frame(
  d      = ppi_example$ppi,          # edge list
  vertices = ppi_example$annotation, # vertex metadata
  directed = FALSE                  # undirected network
)
```

```
igraph_object
```

```
## IGRAPH 9acdb55 UNW- 100 200 --
## + attr: name (v/c), group (v/c), weight (e/n)
## + edges from 9acdb55 (vertex names):
## [1] A1 --D40 A2 --D39 A3 --D38 A4 --D37 A5 --D36 A6 --D35 A7 --D34 A8 --D33
## [9] A9 --D32 A10--D31 B1 --D30 B2 --D29 B3 --D28 B4 --D27 B5 --D26 B6 --D25
## [17] B7 --D24 B8 --D23 B9 --D22 B10--D21 B11--D20 B12--D19 B13--D18 B14--D17
## [25] B15--D16 B16--D15 B17--D14 B18--D13 B19--D12 B20--D11 C1 --D10 C2 --D9
## [33] C3 --D8 C4 --D7 C5 --D6 C6 --D5 C7 --D4 C8 --D3 C9 --D2 C10--D1
## [41] C11--C30 C12--C29 C13--C28 C14--C27 C15--C26 C16--C25 C17--C24 C18--C23
## [49] C19--C22 C20--C21 C20--C21 C19--C22 C18--C23 C17--C24 C16--C25 C15--C26
## [57] C14--C27 C13--C28 C12--C29 C11--C30 C10--D1 C9 --D2 C8 --D3 C7 --D4
## + ... omitted several edges
```

```
# ---- Convert an existing igraph object into a ggNetView-compatible tbl_graph ----
# Adds Modularity / Degree / Strength columns and harmonises the schema so it
# plugs straight into ggNetView().
```

```
graph_obj_from_igraph <- build_graph_from_igraph(
  igraph      = igraph_object,    # an existing igraph object
  module.method = "Fast_greedy",  # community detection algorithm
  seed        = 1115              # fix RNG for reproducibility
)
```

```
graph_obj_from_igraph
```

```
## # A tbl_graph: 100 nodes and 50 edges
```

```
## #
```

```
## # An unrooted forest with 50 trees
```

```
## #
```

```
## # Node Data: 100 x 8 (active)
```

	name	group	modularity	modularity2	modularity3	Modularity	Degree	Strength
	<chr>	<chr>	<fct>	<ord>	<chr>	<ord>	<dbl>	<dbl>
## 1	C13	C	1	1	1	1	1	26.7
## 2	C28	C	1	1	1	1	1	26.7
## 3	C2	C	10	10	10	10	1	37.4
## 4	D9	D	10	10	10	10	1	37.4
## 5	A3	A	11	11	11	11	1	37.8
## 6	D38	D	11	11	11	11	1	37.8
## 7	B12	B	12	12	12	12	1	41.3
## 8	D19	D	12	12	12	12	1	41.3
## 9	A1	A	13	13	13	13	1	45.2
## 10	D40	D	13	13	13	13	1	45.2

```
## # i 90 more rows
```

```
## #
## # Edge Data: 50 x 3
##   from    to weight
##   <int> <int> <dbl>
## 1     9    10  45.2
## 2    15    16  50.6
## 3     5     6  37.8
## # i 47 more rows
```

2.8 Build graph from WGCNA

```
# ---- Importing an existing WGCNA pipeline output ----
# (Not evaluated here - requires external files saved by WGCNA::blockwiseModules.)

# 1. Load the topological overlap matrix (TOM) saved by WGCNA.
load(file = "WGCNA.tom-block.1.RData")
TOM_mat <- as.matrix(TOM)

# 2. Load the abundance matrix used to build that TOM
#   (kept for downstream re-thresholding / sanity checks).
plot.mat = readRDS(file = "plot.mat.Rds")

# 3. Sparsify the TOM: keep only the top_k strongest TOM connections per node.
trans_TOM <- trans_TOM_in_WGCNA(
  TOM_mat,      # square TOM matrix from WGCNA
  plot.mat,     # original abundance matrix the TOM came from
  top_k = 1     # keep top_k strongest neighbours per node
)

# 4. Load the WGCNA net object (contains module colour assignments).
net <- readRDS(file = "net.Rds")

# 5. Build the module table - one row per feature; columns: ID + Module.
module <- data.frame(
  ID      = names(net$colors),      # feature IDs
  Module  = as.character(net$colors) # WGCNA module colour as module label
)

# 6. Optional ordering of node IDs (used as node_annotation here).
id <- data.frame(
  ID = names(sort(net$colors))      # node IDs in module order
)

# 7. Assemble the tbl_graph.
```

```
# ---- Consensus-network workflow ----
# (1) Build several per-method networks from the SAME matrix.
# (2) Convert each tbl_graph back to an adjacency matrix.
# (3) Fuse the per-method adjacencies into ONE consensus tbl_graph.

# Restrict to the 40 most abundant OTUs to keep this chunk responsive.
data(otu_rare_relative)
mat <- as.matrix(otu_rare_relative)
mat <- mat[order(rowSums(mat), decreasing = TRUE)[seq_len(40)], ]

# ---- (1) Three per-method networks on the same data ----

# Spearman correlation through psych::corr.test - baseline method.
g_cor <- build_graph_from_mat(
  mat, # input matrix (variables x samples)
  method = "cor", # correlation backend: psych::corr.test
  cor.method = "spearman", # Spearman correlation
  proc = "BH", # multiple-testing correction (Benjamini-Hochberg)
  r.threshold = 0.4, # |r| cutoff
  p.threshold = 0.05, # adjusted p-value cutoff
  module.method = "Fast_greedy", # community detection algorithm
  seed = 1 # fix RNG for reproducibility
)

## The max module in network is 5 we use the 5 modules for next analysis

# WGCNA-style correlation (WGCNA::corAndPvalue + the package's thresholding).
g_wgcna <- build_graph_from_mat(
  mat, # input matrix (variables x samples)
```

```

method      = "WGCNA",          # correlation backend: WGCNA::corAndPvalue
cor.method   = "spearman",      # Spearman correlation
proc         = "BH",           # multiple-testing correction (Benjamini-Hochberg)
r.threshold  = 0.4,            # |r| cutoff
p.threshold  = 0.05,           # adjusted p-value cutoff
module.method = "Fast_greedy",  # community detection algorithm
seed         = 1                # fix RNG for reproducibility
)

```

The max module in network is 6 we use the 6 modules for next analysis

```

# Hmisc::rcorr - another correlation engine; slightly different NA/tie behaviour.
g_hmisc <- build_graph_from_mat(
  mat,                                # input matrix (variables x samples)
  method      = "Hmisc",              # correlation backend: Hmisc::rcorr
  cor.method   = "spearman",          # Spearman correlation
  r.threshold  = 0.4,                 # |r| cutoff
  p.threshold  = 0.05,                # raw p-value cutoff (no `proc` here)
  module.method = "Fast_greedy",      # community detection algorithm
  seed         = 1                    # fix RNG for reproducibility
)

```

The max module in network is 4 we use the 4 modules for next analysis

```

# ---- (2) tbl_graph -> adjacency matrix ----
# `get_graph_adjacency()` recovers the weighted adjacency matrix from a tbl_graph,
# which is the format expected by build_graph_from_consensus().
adj_cor   <- get_graph_adjacency(g_cor)
adj_wgcna <- get_graph_adjacency(g_wgcna)
adj_hmisc <- get_graph_adjacency(g_hmisc)

# Bundle per-method matrices into a NAMED list.
# Names ("cor", "wgcna", "hmisc") are kept as method labels downstream.
adj_list <- list(cor = adj_cor, wgcna = adj_wgcna, hmisc = adj_hmisc)

# ---- (3) Consensus fusion ----
# Borda rank fusion: ranks every pair by |w| in each method, averages the ranks,
# then min-max rescales to [-1, 1]. Robust to per-method weight-scale differences.
g_borda <- build_graph_from_consensus(
  adj_list      = adj_list,          # named list of per-method adjacency matrices
  method        = "rank_fusion",    # fusion strategy
  rank_fusion_algorithm = "borda",   # specific rank-fusion variant
  threshold     = 0.5,              # final |w| cutoff applied to the fused matrix
  top_modules   = 5,                # keep top-5 modules; rest -> "Others"
)

```

```
seed = 1 # fix RNG for reproducibility
)
```

```
## The max module in network is 3 we use the 3 modules for next analysis
```

```
g_borda
```

```
## # A tbl_graph: 8 nodes and 8 edges
```

```
## #
```

```
## # An undirected simple graph with 3 components
```

```
## #
```

```
## # Node Data: 8 x 7 (active)
```

##	name	modularity	modularity2	modularity3	Modularity	Degree	Strength
##	<chr>	<fct>	<ord>	<chr>	<ord>	<dbl>	<dbl>
## 1	ASV_6	1	1	1	1	3	2.33
## 2	ASV_10	1	1	1	1	3	2.33
## 3	ASV_2	1	1	1	1	3	2.33
## 4	ASV_17	1	1	1	1	3	2.33
## 5	ASV_36	2	2	2	2	1	1
## 6	ASV_38	2	2	2	2	1	1
## 7	ASV_33	3	3	3	3	1	1
## 8	ASV_37	3	3	3	3	1	1

```
## #
```

```
## # Edge Data: 8 x 5
```

##	from	to	weight	correlation	corr_direction
##	<int>	<int>	<dbl>	<dbl>	<chr>
## 1	1	2	0.667	0.667	Positive
## 2	1	3	0.667	0.667	Positive
## 3	1	4	1	1	Positive

```
## # i 5 more rows
```


Chapter 3

Random Matrix Theory (RMT)–based random network

When a numerical matrix is used as input, it is necessary to filter and screen the resulting correlation matrix. Empirical thresholds are commonly adopted, usually between 0.6 to 0.85; however, these thresholds are largely subjective.

Compared with empirical thresholding, random matrix theory (RMT) provides a data-driven and objective approach for determining correlation thresholds, effectively separating true biological signals from random noise and improving the robustness and comparability of ecological networks.

3.1 Random Matrix Theory (RMT)

Load R Package

```
library(tidyverse) # data manipulation / piping utilities
library(ggNetView) # graph builders + info-extraction helpers
```

3.2 Example data

Example data

```
# Access built-in example datasets in ggNetView
```

```
# Relative abundance table of rarefied ASVs or OTUs
```

```
data("otu_rare_relative")
```

```
dim(otu_rare_relative)
```

```
## [1] 2859 18
```

```
otu_rare_relative[1:5, 1:5]
```

```
##           K01      K02      K03      K04      K05
## ASV_1 0.03306667 0.05453333 0.02013333 0.03613333 0.02686667
## ASV_2 0.05750000 0.03393333 0.06046667 0.05810000 0.07320000
## ASV_3 0.01733333 0.01296667 0.02290000 0.02336667 0.03106667
## ASV_4 0.04266667 0.01093333 0.01416667 0.01933333 0.03346667
## ASV_6 0.02646667 0.01856667 0.02110000 0.02353333 0.03806667
```

```
# Taxonomic annotation table of ASVs or OTUs
```

```
data("tax_tab")
```

```
dim(tax_tab)
```

```
## [1] 2859 8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
```

```
##   OTUID Kingdom Phylum      Class      Order
##   <chr>  <chr>    <chr>      <chr>      <chr>
## 1 ASV_2  Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia Spartobacteria Unassigned
## 3 ASV_31 Bacteria Actinobacteria Actinobacteria Actinomycetales
## 4 ASV_27 Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned    Unassigned  Unassigned
```

3.3 Compute RMT threshold

```

# ---- RMT threshold scan #1: WGCNA correlation backend ----
# `ggNetView_RMT()` scans a series of candidate |r| thresholds, builds the
# thresholded correlation matrix at each, computes its eigenvalue spacing
# distribution (NNSD), and picks the threshold at which the NNSD transitions
# from Gaussian Orthogonal Ensemble (signal + noise) to Poisson (noise-free,
# modular network) - this is the RMT-recommended cutoff.
out <- ggNetView_RMT(
  mat           = otu_rare_relative, # input matrix (variables x samples)
  transfrom.method = "none",          # already relative-abundance, no extra transform
  method        = "WGCNA",           # use WGCNA::corAndPvalue for correlation
  cor.method     = "pearson",         # Pearson correlation
  unfold.method  = "gaussian",        # Gaussian KDE unfolding of eigenvalues
  bandwidth     = "nrd0",            # KDE bandwidth rule (stats::density default)
  nr.fit.points  = 20,                # support points (only used if unfold.method = "spline")
  discard.outliers = TRUE,            # IQR-trim extreme eigenvalues before unfolding
  discard.zeros  = TRUE,             # drop all-zero rows/cols after thresholding
  min.mat.dim    = 40,               # early-stop if effective dim drops below 40
  max.ev.spacing = 3,               # truncate NNSD tail at spacing = 3
  save_plots     = FALSE,            # don't write diagnostic PNGs to disk
  out_dir        = "RMT_plots",      # (only used when save_plots = TRUE)
  verbose        = TRUE,             # print scan progress to console
  seed          = 1115               # fix RNG for reproducibility
)

```

```
## [Info] Matrix dimension: 2859 x 2859
```

```
## [Info] #non-zeros: 8173751 | Sparseness: 0.0000
```

```
## [Scan] 1/51 | threshold = 0
```

```
## Effective dimension: 2859
```

```
## [Scan] 2/51 | threshold = 0.02
```

```
## Effective dimension: 2859
```

```
## [Scan] 3/51 | threshold = 0.04
```

```
## Effective dimension: 2859
```

```
## [Scan] 4/51 | threshold = 0.06
```

```

##           Effective dimension: 2859

## [Scan]    5/51 | threshold = 0.08

##           Effective dimension: 2859

## [Scan]    6/51 | threshold = 0.1

##           Effective dimension: 2859

## [Scan]    7/51 | threshold = 0.12

##           Effective dimension: 2859

## [Scan]    8/51 | threshold = 0.14

##           Effective dimension: 2859

## [Scan]    9/51 | threshold = 0.16

##           Effective dimension: 2859

## [Scan]   10/51 | threshold = 0.18

##           Effective dimension: 2859

## [Scan]   11/51 | threshold = 0.2

##           Effective dimension: 2859

## [Scan]   12/51 | threshold = 0.22

##           Effective dimension: 2859

## [Scan]   13/51 | threshold = 0.24

##           Effective dimension: 2859

## [Scan]   14/51 | threshold = 0.26

##           Effective dimension: 2859

```

```
## [Scan] 15/51 | threshold = 0.28

## Effective dimension: 2859

## [Scan] 16/51 | threshold = 0.3

## Effective dimension: 2859

## [Scan] 17/51 | threshold = 0.32

## Effective dimension: 2859

## [Scan] 18/51 | threshold = 0.34

## Effective dimension: 2859

## [Scan] 19/51 | threshold = 0.36

## Effective dimension: 2859

## [Scan] 20/51 | threshold = 0.38

## Effective dimension: 2859

## [Scan] 21/51 | threshold = 0.4

## Effective dimension: 2859

## [Scan] 22/51 | threshold = 0.42

## Effective dimension: 2859

## [Scan] 23/51 | threshold = 0.44

## Effective dimension: 2859

## [Scan] 24/51 | threshold = 0.46

## Effective dimension: 2859

## [Scan] 25/51 | threshold = 0.48
```

```

##           Effective dimension: 2859

## [Scan]  26/51 | threshold = 0.5

##           Effective dimension: 2859

## [Scan]  27/51 | threshold = 0.52

##           Effective dimension: 2859

## [Scan]  28/51 | threshold = 0.54

##           Effective dimension: 2859

## [Scan]  29/51 | threshold = 0.56

##           Effective dimension: 2859

## [Scan]  30/51 | threshold = 0.58

##           Effective dimension: 2859

## [Scan]  31/51 | threshold = 0.6

##           Effective dimension: 2859

## [Scan]  32/51 | threshold = 0.62

##           Effective dimension: 2859

## [Scan]  33/51 | threshold = 0.64

##           Effective dimension: 2857

## [Scan]  34/51 | threshold = 0.66

##           Effective dimension: 2855

## [Scan]  35/51 | threshold = 0.68

##           Effective dimension: 2842

```

```
## [Scan] 36/51 | threshold = 0.7
##           Effective dimension: 2800
## [Scan] 37/51 | threshold = 0.72
##           Effective dimension: 2705
## [Scan] 38/51 | threshold = 0.74
##           Effective dimension: 2546
## [Scan] 39/51 | threshold = 0.76
##           Effective dimension: 2333
## [Scan] 40/51 | threshold = 0.78
##           Effective dimension: 2049
## [Scan] 41/51 | threshold = 0.8
##           Effective dimension: 1735
## [Scan] 42/51 | threshold = 0.82
##           Effective dimension: 1396
## [Scan] 43/51 | threshold = 0.84
##           Effective dimension: 1082
## [Scan] 44/51 | threshold = 0.86
##           Effective dimension: 825
## [Scan] 45/51 | threshold = 0.88
##           Effective dimension: 600
## [Scan] 46/51 | threshold = 0.9
```

```
##           Effective dimension: 454

## [Scan]  47/51 | threshold = 0.92

##           Effective dimension: 324

## [Scan]  48/51 | threshold = 0.94

##           Effective dimension: 215

## [Scan]  49/51 | threshold = 0.96

##           Effective dimension: 157

## [Scan]  50/51 | threshold = 0.98

##           Effective dimension: 79

## [Scan]  51/51 | threshold = 1

##           Effective dimension: 2

##           Too small. Stop scanning.
```

```
# The chosen threshold to feed downstream into build_graph_from_mat(r.threshold = ...)
out$chosen_threshold
```

```
## [1] 0.82
```

```
# ---- RMT threshold scan #2: plain `cor()` backend for comparison ----
# Same parameters as `out` above, but with `method = "cor"` (base stats::cor /
# psych::corr.test) instead of WGCNA. Useful as a sanity check - the chosen
# threshold should be very close across the two backends
out2 <- ggNetView_RMT(
  mat           = otu_rare_relative,
  transfrom.method = "none",
  method        = "cor",           # <-- only difference vs. `out`
  cor.method    = "pearson",
  unfold.method = "gaussian",
  bandwidth     = "nrd0",
  nr.fit.points = 20,
```

```
discard.outliers = TRUE,
discard.zeros    = TRUE,
min.mat.dim      = 40,
max.ev.spacing   = 3,
save_plots       = FALSE,
out_dir          = "RMT_plots",
verbose          = TRUE,
seed             = 1115
)

## [Info] Matrix dimension: 2859 x 2859

## [Info] #non-zeros: 8173761 | Sparseness: 0.0000

## [Scan] 1/51 | threshold = 0

## Effective dimension: 2859

## [Scan] 2/51 | threshold = 0.02

## Effective dimension: 2859

## [Scan] 3/51 | threshold = 0.04

## Effective dimension: 2859

## [Scan] 4/51 | threshold = 0.06

## Effective dimension: 2859

## [Scan] 5/51 | threshold = 0.08

## Effective dimension: 2859

## [Scan] 6/51 | threshold = 0.1

## Effective dimension: 2859

## [Scan] 7/51 | threshold = 0.12

## Effective dimension: 2859
```

```

## [Scan] 8/51 | threshold = 0.14

## Effective dimension: 2859

## [Scan] 9/51 | threshold = 0.16

## Effective dimension: 2859

## [Scan] 10/51 | threshold = 0.18

## Effective dimension: 2859

## [Scan] 11/51 | threshold = 0.2

## Effective dimension: 2859

## [Scan] 12/51 | threshold = 0.22

## Effective dimension: 2859

## [Scan] 13/51 | threshold = 0.24

## Effective dimension: 2859

## [Scan] 14/51 | threshold = 0.26

## Effective dimension: 2859

## [Scan] 15/51 | threshold = 0.28

## Effective dimension: 2859

## [Scan] 16/51 | threshold = 0.3

## Effective dimension: 2859

## [Scan] 17/51 | threshold = 0.32

## Effective dimension: 2859

## [Scan] 18/51 | threshold = 0.34

```

```
##           Effective dimension: 2859

## [Scan]  19/51 | threshold = 0.36

##           Effective dimension: 2859

## [Scan]  20/51 | threshold = 0.38

##           Effective dimension: 2859

## [Scan]  21/51 | threshold = 0.4

##           Effective dimension: 2859

## [Scan]  22/51 | threshold = 0.42

##           Effective dimension: 2859

## [Scan]  23/51 | threshold = 0.44

##           Effective dimension: 2859

## [Scan]  24/51 | threshold = 0.46

##           Effective dimension: 2859

## [Scan]  25/51 | threshold = 0.48

##           Effective dimension: 2859

## [Scan]  26/51 | threshold = 0.5

##           Effective dimension: 2859

## [Scan]  27/51 | threshold = 0.52

##           Effective dimension: 2859

## [Scan]  28/51 | threshold = 0.54

##           Effective dimension: 2859
```

```

## [Scan] 29/51 | threshold = 0.56

## Effective dimension: 2859

## [Scan] 30/51 | threshold = 0.58

## Effective dimension: 2859

## [Scan] 31/51 | threshold = 0.6

## Effective dimension: 2859

## [Scan] 32/51 | threshold = 0.62

## Effective dimension: 2859

## [Scan] 33/51 | threshold = 0.64

## Effective dimension: 2857

## [Scan] 34/51 | threshold = 0.66

## Effective dimension: 2855

## [Scan] 35/51 | threshold = 0.68

## Effective dimension: 2842

## [Scan] 36/51 | threshold = 0.7

## Effective dimension: 2800

## [Scan] 37/51 | threshold = 0.72

## Effective dimension: 2705

## [Scan] 38/51 | threshold = 0.74

## Effective dimension: 2546

## [Scan] 39/51 | threshold = 0.76

```

```
##           Effective dimension: 2333

## [Scan]  40/51 | threshold = 0.78

##           Effective dimension: 2049

## [Scan]  41/51 | threshold = 0.8

##           Effective dimension: 1735

## [Scan]  42/51 | threshold = 0.82

##           Effective dimension: 1396

## [Scan]  43/51 | threshold = 0.84

##           Effective dimension: 1082

## [Scan]  44/51 | threshold = 0.86

##           Effective dimension: 825

## [Scan]  45/51 | threshold = 0.88

##           Effective dimension: 600

## [Scan]  46/51 | threshold = 0.9

##           Effective dimension: 454

## [Scan]  47/51 | threshold = 0.92

##           Effective dimension: 324

## [Scan]  48/51 | threshold = 0.94

##           Effective dimension: 215

## [Scan]  49/51 | threshold = 0.96

##           Effective dimension: 157
```

```
## [Scan] 50/51 | threshold = 0.98

##           Effective dimension: 79

## [Scan] 51/51 | threshold = 1

##           Effective dimension: 10

##           Too small. Stop scanning.

# Compare with out$chosen_threshold - they should be in the same neighborhood.
out2$chosen_threshold

## [1] 0.82
```

3.4 Build graph from matrix based on RMT-threshold

Build graph object

```
# ---- Construct the tidygraph network using the RMT-selected r threshold ----
# Replaces the usual subjective r-cutoff (e.g. 0.6-0.85) with the data-driven
# value `out$chosen_threshold` returned above.
graph_obj <- build_graph_from_mat(
  mat                = otu_rare_relative,      # variable x sample matrix
  transform.method   = "none",                 # match the transform used for RMT scan
  r.threshold        = out$chosen_threshold,    # RMT-derived |r| cutoff (objective)
  p.threshold        = 0.05,                   # significance cutoff on adjusted p-value
  method             = "WGCNA",                # correlation backend (must match RMT scan)
  cor.method         = "pearson",               # correlation type
  proc               = "bonferroni",            # multiple-testing correction for p-value
  module.method      = "Fast_greedy",           # community detection algorithm
  node_annotation    = tax_tab,                 # taxonomy joined onto each node (by name)
  top_modules        = 15,                      # keep top-15 modules; smaller ones -> "0"
  seed               = 1115                     # reproducibility
)

# Print to inspect node/edge counts and the columns now available on nodes
# (modularity, modularity2, modularity3, Modularity, Degree, Strength + taxonomy)
graph_obj
```

3.4. BUILD GRAPH FROM MATRIX BASED ON RMT-THRESHOLD 43

```
## # A tbl_graph: 213 nodes and 844 edges
## #
## # An undirected simple graph with 29 components
## #
## # Node Data: 213 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl> <chr>
## 1 ASV_649 5          5          5          5          27        26.5 Bacter~
## 2 ASV_705 5          5          5          5          27        26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27        26.5 Bacter~
## 4 ASV_13~ 5          5          5          5          27        26.5 Bacter~
## 5 ASV_14~ 5          5          5          5          27        26.5 Bacter~
## 6 ASV_14~ 5          5          5          5          27        26.5 Bacter~
## 7 ASV_24~ 5          5          5          5          27        26.5 Bacter~
## 8 ASV_25~ 5          5          5          5          27        26.4 Bacter~
## 9 ASV_28~ 5          5          5          5          27        26.5 Bacter~
## 10 ASV_28~ 5          5          5          5          27        26.5 Bacter~
## # i 203 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 844 x 5
##   from      to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1 194 195 0.959      0.959 Positive
## 2 185 208 0.954      0.954 Positive
## 3 185 213 0.957      0.957 Positive
## # i 841 more rows
```


Chapter 4

Get network information

Once the graph object is constructed, basic network information can be retrieved, and subnetworks can be extracted.

Load R Package

```
library(tidyverse)    # data manipulation / piping utilities
library(ggNetView)    # graph builders + info-extraction helpers
```

Example data

```
# ---- Built-in example datasets ----

# Relative abundance table of rarefied ASVs/OTUs (rows = ASVs, cols = samples).
data("otu_rare_relative")
dim(otu_rare_relative)
```

```
## [1] 2859  18
```

```
otu_rare_relative[1:5, 1:5]
```

```
##           K01           K02           K03           K04           K05
## ASV_1 0.03306667 0.05453333 0.02013333 0.03613333 0.02686667
## ASV_2 0.05750000 0.03393333 0.06046667 0.05810000 0.07320000
## ASV_3 0.01733333 0.01296667 0.02290000 0.02336667 0.03106667
## ASV_4 0.04266667 0.01093333 0.01416667 0.01933333 0.03346667
## ASV_6 0.02646667 0.01856667 0.02110000 0.02353333 0.03806667
```

```
# Taxonomic annotation; first column must be the ASV/OTU ID.
data("tax_tab")
dim(tax_tab)
```

```
## [1] 2859      8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
##   OTUID Kingdom Phylum      Class      Order
##   <chr>  <chr>    <chr>      <chr>      <chr>
## 1 ASV_2  Archaea  Thaumarchaeota Unassigned Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia Spartobacteria Unassigned
## 3 ASV_31 Bacteria Actinobacteria Actinobacteria Actinomycetales
## 4 ASV_27 Archaea  Thaumarchaeota Unassigned Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned  Unassigned  Unassigned
```

Build graph object

```
# ---- Build a co-occurrence network as the input for downstream queries ----
graph_obj <- build_graph_from_mat(
  mat              = otu_rare_relative, # variables x samples numeric matrix
  transform.method = "none",           # input already relative abundance, no extra
  r.threshold      = 0.7,               # |r| cutoff for keeping an edge
  p.threshold      = 0.05,              # adjusted p-value cutoff
  method           = "WGCNA",          # correlation backend: WGCNA::corAndPvalue
  cor.method       = "pearson",         # Pearson correlation
  proc             = "bonferroni",      # multiple-testing correction
  module.method    = "Fast_greedy",     # community detection algorithm
  node_annotation  = tax_tab,           # taxonomy joined onto each node by name
  top_modules      = 15,                # keep top-15 modules; rest -> "Others"
  seed            = 1115                # fix RNG for reproducibility
)

graph_obj
```

```
## # A tbl_graph: 213 nodes and 844 edges
```

```
## #
```

```
## # An undirected simple graph with 29 components
```

```
## #
```

```
## # Node Data: 213 x 14 (active)
```

```
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <dbl>        <dbl>        <dbl>        <dbl>    <dbl>    <dbl>  <chr>
```

```
## 1 ASV_649 5 5 5 27 26.5 Bacter~
## 2 ASV_705 5 5 5 27 26.5 Bacter~
## 3 ASV_12~ 5 5 5 27 26.5 Bacter~
## 4 ASV_13~ 5 5 5 27 26.5 Bacter~
## 5 ASV_14~ 5 5 5 27 26.5 Bacter~
## 6 ASV_14~ 5 5 5 27 26.5 Bacter~
## 7 ASV_24~ 5 5 5 27 26.5 Bacter~
## 8 ASV_25~ 5 5 5 27 26.4 Bacter~
## 9 ASV_28~ 5 5 5 27 26.5 Bacter~
## 10 ASV_28~ 5 5 5 27 26.5 Bacter~
## # i 203 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## # Genus <chr>, Species <chr>
## #
## # Edge Data: 844 x 5
## from to weight correlation corr_direction
## <int> <int> <dbl> <dbl> <chr>
## 1 194 195 0.959 0.959 Positive
## 2 185 208 0.954 0.954 Positive
## 3 185 213 0.957 0.957 Positive
## # i 841 more rows
```

4.1 Full-network information

```
# ---- Retrieve node-level and edge-level info for the FULL graph ----
# Returns a list with `node_info` and `edge_info` tibbles.
graph_obj_info <- get_info_from_graph(graph_obj = graph_obj)
```

```
class(graph_obj_info) # "list"
```

```
## [1] "list"
```

```
names(graph_obj_info) # "node_info", "edge_info"
```

```
## [1] "node_info" "edge_info"
```

```
graph_obj_info$node_info # per-node attributes: name, Modularity, Degree, Strength, taxonomy
```

```
## # A tibble: 213 x 11
## name Modularity Degree Strength Kingdom Phylum Class Order Family Genus
## <chr> <ord> <dbl> <dbl> <chr> <chr> <chr> <chr> <chr> <chr>
```

```
## 1 ASV_649 5 27 26.5 Bacteria Proteo~ Alph~ Rhiz~ Unass~ Unas~
## 2 ASV_705 5 27 26.5 Bacteria Actino~ Acti~ Acti~ Cellu~ Cell~
## 3 ASV_1234 5 27 26.5 Bacteria Gemmat~ Gemm~ Gemm~ Gemma~ Gemm~
## 4 ASV_1373 5 27 26.5 Bacteria Actino~ Acti~ Soli~ Solir~ Soli~
## 5 ASV_1479 5 27 26.5 Bacteria Verruc~ Spar~ Unas~ Unass~ Spar~
## 6 ASV_1481 5 27 26.5 Bacteria Proteo~ Gamm~ Unas~ Unass~ Unas~
## 7 ASV_2453 5 27 26.5 Bacteria Bacter~ Sphi~ Sphi~ Chiti~ Unas~
## 8 ASV_2562 5 27 26.4 Bacteria Bacter~ Sphi~ Sphi~ Chiti~ Unas~
## 9 ASV_2848 5 27 26.5 Bacteria Planct~ Plan~ Plan~ Planc~ Unas~
## 10 ASV_2844 5 27 26.5 Bacteria Proteo~ Unas~ Unas~ Unass~ Unas~
## # i 203 more rows
## # i 1 more variable: Species <chr>
```

```
graph_obj_info$edge_info # per-edge attributes: from, to, weight, ...
```

```
## # A tibble: 844 x 5
##   from to weight correlation corr_direction
##   <chr> <chr> <dbl> <dbl> <chr>
## 1 ASV_6 ASV_39 0.959 0.959 Positive
## 2 ASV_20 ASV_1852 0.954 0.954 Positive
## 3 ASV_20 ASV_2911 0.957 0.957 Positive
## 4 ASV_172 ASV_43 0.969 0.969 Positive
## 5 ASV_614 ASV_65 0.943 0.943 Positive
## 6 ASV_69 ASV_173 0.995 0.995 Positive
## 7 ASV_69 ASV_206 0.994 0.994 Positive
## 8 ASV_69 ASV_744 0.976 0.976 Positive
## 9 ASV_945 ASV_69 0.985 0.985 Positive
## 10 ASV_69 ASV_1418 0.975 0.975 Positive
## # i 834 more rows
```

4.2 Sub-network (modularity) information

```
# ---- Split the graph by `Modularity` (one subgraph per module) ----
# `select_module = NULL` -> no merged subgraph; only per-module split + stats.
subgraph_info <- get_subgraph(
  graph_obj = graph_obj, # input tbl_graph (must have a `Modularity` column on n
  select_module = NULL # NULL -> don't build a merged subgraph
)
```

```
##   Module Number
## 1      5      31
## 2      2      24
```

```
## 3      8      22
## 4     21     15
## 5      1     13
## 6      3     11
## 7      4     11
## 8      6     11
## 9     25      8
## 10     7      6
## 11    12      5
## 12    22      5
## 13    10      4
## 14    11      4
## 15    15      4
## 16 Others    39
```

```
class(subgraph_info)      # "list"
```

```
## [1] "list"
```

```
names(subgraph_info)      # "sub_graph_all", "stat_module", "sub_graph_select"
```

```
## [1] "sub_graph_all"      "stat_module"      "sub_graph_select"
```

```
# Per-module node counts (data.frame with columns: Module, Number).
subgraph_info$stat_module
```

```
##      Module Number
## 1         5      31
## 2         2      24
## 3         8      22
## 4        21      15
## 5         1      13
## 6         3      11
## 7         4      11
## 8         6      11
## 9        25       8
## 10        7       6
## 11       12       5
## 12       22       5
## 13       10       4
## 14       11       4
## 15       15       4
## 16 Others    39
```

We select modularity 5

```
# ---- Build a merged subgraph for a chosen module ----
# `select_module = "5"` keeps nodes whose Modularity == "5"
# (and the edges between them) in `sub_graph_select`.
graph_module5 <- get_subgraph(
  graph_obj      = graph_obj,
  select_module = "5"          # character vector of module labels to retain
)

##      Module Number
## 1         5      31
## 2         2      24
## 3         8      22
## 4        21      15
## 5         1      13
## 6         3      11
## 7         4      11
## 8         6      11
## 9        25       8
## 10        7       6
## 11       12       5
## 12       22       5
## 13       10       4
## 14       11       4
## 15       15       4
## 16 Others      39

# ---- Inspect the module-5 subgraph using the same info helper ----
graph_obj_info <- get_info_from_graph(
  graph_obj = graph_module5$sub_graph_select  # the merged tbl_graph for module 5
)

class(graph_obj_info)          # "list"

## [1] "list"

names(graph_obj_info)          # "node_info", "edge_info"

## [1] "node_info" "edge_info"

graph_obj_info$node_info      # nodes belonging to module 5
```

```
## # A tibble: 31 x 11
##   name      Modularity Degree Strength Kingdom  Phylum  Class Order Family Genus
##   <chr>      <ord>      <dbl>    <dbl> <chr>    <chr>    <chr> <chr> <chr> <chr>
## 1 ASV_649    5          27      26.5 Bacteria Proteo~ Alph~ Rhiz~ Unass~ Unas~
## 2 ASV_705    5          27      26.5 Bacteria Actino~ Acti~ Acti~ Cellu~ Cell~
## 3 ASV_1234   5          27      26.5 Bacteria Gemmat~ Gemm~ Gemm~ Gemma~ Gemm~
## 4 ASV_1373   5          27      26.5 Bacteria Actino~ Acti~ Soli~ Solir~ Soli~
## 5 ASV_1479   5          27      26.5 Bacteria Verruc~ Spar~ Unas~ Unass~ Spar~
## 6 ASV_1481   5          27      26.5 Bacteria Proteo~ Gamm~ Unas~ Unass~ Unas~
## 7 ASV_2453   5          27      26.5 Bacteria Bacter~ Sphi~ Sphi~ Chiti~ Unas~
## 8 ASV_2562   5          27      26.4 Bacteria Bacter~ Sphi~ Sphi~ Chiti~ Unas~
## 9 ASV_2848   5          27      26.5 Bacteria Planct~ Plan~ Plan~ Planc~ Unas~
## 10 ASV_2844  5          27      26.5 Bacteria Proteo~ Unas~ Unas~ Unass~ Unas~
## # i 21 more rows
## # i 1 more variable: Species <chr>
```

```
graph_obj_info$edge_info      # edges between module-5 nodes
```

```
## # A tibble: 330 x 5
##   from      to      weight correlation corr_direction
##   <chr>    <chr>    <dbl>    <dbl> <chr>
## 1 ASV_244 ASV_367  0.949      0.949 Positive
## 2 ASV_1119 ASV_244  0.952      0.952 Positive
## 3 ASV_1531 ASV_244  0.965      0.965 Positive
## 4 ASV_649 ASV_322  0.958      0.958 Positive
## 5 ASV_705 ASV_322  0.954      0.954 Positive
## 6 ASV_913 ASV_322  0.954      0.954 Positive
## 7 ASV_996 ASV_322  0.949      0.949 Positive
## 8 ASV_1234 ASV_322  0.962      0.962 Positive
## 9 ASV_1373 ASV_322  0.958      0.958 Positive
## 10 ASV_1479 ASV_322  0.958      0.958 Positive
## # i 320 more rows
```

4.3 Sub-network (sample) information

```
# ---- Extract sample-level subgraphs -----
# For each sample (column of `mat`), an OTU is treated as "present" when
# its abundance is > `min_abundance` (default 0). The induced subgraph of
# `graph_obj` on those present OTUs is returned per sample. With
# `select_sample` + `combine`, a single merged subgraph is also returned.
res <- get_sample_subgraph(
  graph_obj      = graph_obj,                # the network built above
```

```

    mat          = otu_rare_relative,          # same matrix used to judge "present"
    select_sample = colnames(otu_rare_relative)[1:3], # sample IDs to merge into one subgraph
    combine       = "union"                    # "union" = OTUs present in ANY sample
  )
  names(res)     # "sub_graph_all", "stat_sample", "sub_graph_select"

## [1] "sub_graph_all"      "stat_sample"        "sub_graph_select"

# ---- Inspect the merged 3-sample subgraph ----
res$sub_graph_select # tbl_graph: nodes = union/intersect of present OTUs across the three samples

## # A tbl_graph: 181 nodes and 602 edges
## #
## # An undirected simple graph with 34 components
## #
## # Node Data: 181 x 16 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>      <fct>      <ord>      <chr>      <ord>      <dbl>   <dbl> <chr>
## 1 ASV_649 5          5          5          5          27      26.5 Bacter~
## 2 ASV_705 5          5          5          5          27      26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 4 ASV_13~ 5          5          5          5          27      26.5 Bacter~
## 5 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 6 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 7 ASV_24~ 5          5          5          5          27      26.5 Bacter~
## 8 ASV_25~ 5          5          5          5          27      26.4 Bacter~
## 9 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## 10 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## # i 171 more rows
## # i 8 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>, n_present_samples <int>,
## #   present_in_samples <chr>
## #
## # Edge Data: 602 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1    164   165 0.959        0.959 Positive
## 2    158   181 0.957        0.957 Positive
## 3    145   147 0.969        0.969 Positive
## # i 599 more rows

graph_obj_info <- get_info_from_graph(graph_obj = res$sub_graph_select)

graph_obj_info$node_info # node attributes + n_present_samples / present_in_samples

```

```
## # A tibble: 181 x 13
##   name      Modularity Degree Strength Kingdom  Phylum  Class Order Family Genus
##   <chr>      <ord>      <dbl>    <dbl> <chr>    <chr>    <chr> <chr> <chr> <chr>
## 1 ASV_649    5          27      26.5 Bacteria Proteo~ Alph~ Rhiz~ Unass~ Unas~
## 2 ASV_705    5          27      26.5 Bacteria Actino~ Acti~ Acti~ Cellu~ Cell~
## 3 ASV_1234   5          27      26.5 Bacteria Gemmat~ Gemm~ Gemm~ Gemma~ Gemm~
## 4 ASV_1373   5          27      26.5 Bacteria Actino~ Acti~ Soli~ Solir~ Soli~
## 5 ASV_1479   5          27      26.5 Bacteria Verruc~ Spar~ Unas~ Unass~ Spar~
## 6 ASV_1481   5          27      26.5 Bacteria Proteo~ Gamm~ Unas~ Unass~ Unas~
## 7 ASV_2453   5          27      26.5 Bacteria Bacter~ Sphi~ Sphi~ Chiti~ Unas~
## 8 ASV_2562   5          27      26.4 Bacteria Bacter~ Sphi~ Sphi~ Chiti~ Unas~
## 9 ASV_2848   5          27      26.5 Bacteria Planct~ Plan~ Plan~ Planc~ Unas~
## 10 ASV_2844  5          27      26.5 Bacteria Proteo~ Unas~ Unas~ Unass~ Unas~
## # i 171 more rows
## # i 3 more variables: Species <chr>, n_present_samples <int>,
## #   present_in_samples <chr>
```

```
graph_obj_info$edge_info  # induced edges between surviving nodes
```

```
## # A tibble: 602 x 5
##   from    to      weight correlation corr_direction
##   <chr>  <chr>    <dbl>    <dbl> <chr>
## 1 ASV_6   ASV_39    0.959      0.959 Positive
## 2 ASV_20  ASV_2911  0.957      0.957 Positive
## 3 ASV_172 ASV_43    0.969      0.969 Positive
## 4 ASV_614 ASV_65    0.943      0.943 Positive
## 5 ASV_69  ASV_173   0.995      0.995 Positive
## 6 ASV_69  ASV_206   0.994      0.994 Positive
## 7 ASV_69  ASV_744   0.976      0.976 Positive
## 8 ASV_945 ASV_69    0.985      0.985 Positive
## 9 ASV_69  ASV_1418  0.975      0.975 Positive
## 10 ASV_69  ASV_2350  0.967      0.967 Positive
## # i 592 more rows
```


Chapter 5

Extract subgraph

Load R Package

```
library(tidyverse) # data manipulation / piping utilities
library(ggNetView) # graph builders + info-extraction helpers
```

Example data

```
# Access built-in example datasets in ggNetView

# ---- Dataset 1: Relative abundance table of rarefied ASVs/OTUs ----
# Rows = ASVs/OTUs (features / variables), Columns = samples
data("otu_rare_relative")
dim(otu_rare_relative) # Check matrix dimensions: n_features x n_samples
```

```
## [1] 2859  18
```

```
otu_rare_relative[1:5, 1:5] # Preview first 5 rows x 5 cols to confirm data layout
```

```
##           K01           K02           K03           K04           K05
## ASV_1 0.03306667 0.05453333 0.02013333 0.03613333 0.02686667
## ASV_2 0.05750000 0.03393333 0.06046667 0.05810000 0.07320000
## ASV_3 0.01733333 0.01296667 0.02290000 0.02336667 0.03106667
## ASV_4 0.04266667 0.01093333 0.01416667 0.01933333 0.03346667
## ASV_6 0.02646667 0.01856667 0.02110000 0.02353333 0.03806667
```

```
# ---- Dataset 2: Taxonomic annotation table for ASVs/OTUs ----
# Rows = ASVs/OTUs (row names should match the abundance table above)
# Columns = taxonomic ranks (Kingdom ~ Species, etc.)
data("tax_tab")
dim(tax_tab)
```

```
## [1] 2859      8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
##   OTUID Kingdom Phylum      Class      Order
##   <chr>   <chr>   <chr>      <chr>      <chr>
## 1 ASV_2  Archaea Thaumarchaeota Unassigned Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia Spartobacteria Unassigned
## 3 ASV_31 Bacteria Actinobacteria Actinobacteria Actinomycetales
## 4 ASV_27 Archaea Thaumarchaeota Unassigned Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned      Unassigned Unassigned
```

Build graph object

```
# Build a correlation-based graph object (tbl_graph) from the abundance matrix.
# This graph object is the foundation for downstream subgraph extraction,
# layout computation, and visualization.
```

```
graph_obj <- build_graph_from_mat(
  mat          = otu_rare_relative, # Input matrix: features (rows) x samples (columns)
  transform.method = "none",        # Data transformation before correlation; Other options: "log", "sqrt", "abs"
  r.threshold    = 0.7,             # Correlation cutoff: keep edges only if |r| >= r.threshold
  p.threshold    = 0.05,            # Significance cutoff: keep edges only if p <= p.threshold
  method         = "WGCNA",         # Association method: Other options: "SpiecEasi", "correlation"
  cor.method     = "pearson",        # Correlation type: Pearson (also: "spearman", "kendall")
  proc           = "bonferroni",     # Multiple-testing correction for p-values. Other options: "BH" / "fdr" / "holm" / "hochberg"
  module.method  = "Fast_greedy",    # Community / module detection algorithm. Other options: "louvain", "leiden", "infomap"
  node_annotation = tax_tab,         # Node metadata (taxonomy) attached to each node
  top_modules    = 15,              # Keep only the top 15 largest modules for downstream analysis
  seed           = 1115,            # Random seed for reproducible module detection
)

graph_obj # Print the graph object: node/edge counts and other summary statistics
```

```
## # A tbl_graph: 213 nodes and 844 edges
```



```
##      Module Number
## 1         5      31
## 2         2      24
## 3         8      22
## 4        21      15
## 5         1      13
## 6         3      11
## 7         4      11
## 8         6      11
## 9        25       8
## 10        7       6
## 11       12       5
## 12       22       5
## 13       10       4
## 14       11       4
## 15       15       4
## 16 Others     39
```

```
class(module5_subgraph_info)
```

```
## [1] "list"
```

```
names(module5_subgraph_info)
```

```
## [1] "sub_graph_all"      "stat_module"        "sub_graph_select"
```

```
module5_subgraph_info$stat_module      # Per-module node count summary (Module, Number)
```

```
##      Module Number
## 1         5      31
## 2         2      24
## 3         8      22
## 4        21      15
## 5         1      13
## 6         3      11
## 7         4      11
## 8         6      11
## 9        25       8
## 10        7       6
## 11       12       5
## 12       22       5
## 13       10       4
## 14       11       4
## 15       15       4
## 16 Others     39
```

```
# module5_subgraph_info$sub_graph_all      # Named list of tbl_graphs, one per module
module5_subgraph_info$sub_graph_select     # Single tbl_graph containing only module "5"
```

```
## # A tbl_graph: 31 nodes and 330 edges
## #
## # An undirected simple graph with 1 component
## #
## # Node Data: 31 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>      <fct>      <ord>      <chr>      <ord>      <dbl>      <dbl> <chr>
## 1 ASV_649 5          5          5          5          27        26.5 Bacter~
## 2 ASV_705 5          5          5          5          27        26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27        26.5 Bacter~
## 4 ASV_13~ 5          5          5          5          27        26.5 Bacter~
## 5 ASV_14~ 5          5          5          5          27        26.5 Bacter~
## 6 ASV_14~ 5          5          5          5          27        26.5 Bacter~
## 7 ASV_24~ 5          5          5          5          27        26.5 Bacter~
## 8 ASV_25~ 5          5          5          5          27        26.4 Bacter~
## 9 ASV_28~ 5          5          5          5          27        26.5 Bacter~
## 10 ASV_28~ 5          5          5          5          27        26.5 Bacter~
## # i 21 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 330 x 5
##   from      to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1    29    30 0.949        0.949 Positive
## 2    21    29 0.952        0.952 Positive
## 3    25    29 0.965        0.965 Positive
## # i 327 more rows
```

Select multi-modularity

```
# Extract a combined subgraph for multiple modules at once (modules 5, 2, and 8).
# `sub_graph_select` will contain all nodes from these three modules
# plus the edges among them.
```

```
module5_2_8_subgraph_info <- get_subgraph(
  graph_obj      = graph_obj,
  select_module = c("5", "2", "8")) # Multiple module IDs
```

```
##   Module Number
## 1         5    31
```

```
## 2      2      24
## 3      8      22
## 4     21      15
## 5      1      13
## 6      3      11
## 7      4      11
## 8      6      11
## 9     25       8
## 10     7       6
## 11    12       5
## 12    22       5
## 13    10       4
## 14    11       4
## 15    15       4
## 16 Others    39
```

```
class(module5_2_8_subgraph_info)
```

```
## [1] "list"
```

```
names(module5_2_8_subgraph_info)
```

```
## [1] "sub_graph_all"      "stat_module"        "sub_graph_select"
```

```
module5_2_8_subgraph_info$stat_module      # Global module stats (same as before; not c
```

```
##      Module Number
## 1      5      31
## 2      2      24
## 3      8      22
## 4     21      15
## 5      1      13
## 6      3      11
## 7      4      11
## 8      6      11
## 9     25       8
## 10     7       6
## 11    12       5
## 12    22       5
## 13    10       4
## 14    11       4
## 15    15       4
## 16 Others    39
```

```
# module5_2_8_subgraph_info$sub_graph_all    # Full per-module subgraph list (unchanged)

module5_2_8_subgraph_info$sub_graph_select # Combined subgraph filtered to modules 5, 2, and 8

## # A tbl_graph: 77 nodes and 582 edges
## #
## # An undirected simple graph with 3 components
## #
## # Node Data: 77 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_649 5          5          5          5          27      26.5 Bacter~
## 2 ASV_705 5          5          5          5          27      26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 4 ASV_13~ 5          5          5          5          27      26.5 Bacter~
## 5 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 6 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 7 ASV_24~ 5          5          5          5          27      26.5 Bacter~
## 8 ASV_25~ 5          5          5          5          27      26.4 Bacter~
## 9 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## 10 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## # i 67 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 582 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1     56    74 0.952      0.952 Positive
## 2     57    74 0.954      0.954 Positive
## 3     58    74 0.947      0.947 Positive
## # i 579 more rows
```

5.2 Extract subgraph by sample

```
# ---- Load example datasets shipped with ggNetView -----
# Relative-abundance table of rarefied ASVs/OTUs (rows = features, cols = samples)

data("otu_rare_relative")
# Taxonomic annotation table for those ASVs/OTUs (used as node metadata)
data("tax_tab")
```

```

# ---- Build the correlation-based network -----
# This produces a tbl_graph that downstream subgraph / layout / plotting
# functions all consume.
graph_obj <- build_graph_from_mat(
  mat           = otu_rare_relative, # Feature-by-sample matrix
  transform.method = "none",         # No transform: input is already relative abundance
  r.threshold    = 0.7,              # Keep edges with |r| >= 0.7
  p.threshold    = 0.05,             # Keep edges with p < 0.05
  method         = "WGCNA",         # WGCNA correlation + p-values
  cor.method     = "pearson",        # Pearson correlation
  proc           = "bonferroni",     # Bonferroni multiple-testing correction
  module.method  = "Fast_greedy",    # Community detection algorithm
  node_annotation = tax_tab,         # Attach taxonomy to each node
  top_modules    = 15,              # Keep the 15 largest modules
  seed           = 1115              # Reproducible module assignment
)

graph_obj

```

```

## # A tbl_graph: 213 nodes and 844 edges
## #
## # An undirected simple graph with 29 components
## #
## # Node Data: 213 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <dbl>      <dbl>      <dbl>      <dbl>      <dbl>  <dbl>  <chr>
## 1 ASV_649  5          5          5          5          27     26.5 Bacter~
## 2 ASV_705  5          5          5          5          27     26.5 Bacter~
## 3 ASV_12~  5          5          5          5          27     26.5 Bacter~
## 4 ASV_13~  5          5          5          5          27     26.5 Bacter~
## 5 ASV_14~  5          5          5          5          27     26.5 Bacter~
## 6 ASV_14~  5          5          5          5          27     26.5 Bacter~
## 7 ASV_24~  5          5          5          5          27     26.5 Bacter~
## 8 ASV_25~  5          5          5          5          27     26.4 Bacter~
## 9 ASV_28~  5          5          5          5          27     26.5 Bacter~
## 10 ASV_28~ 5          5          5          5          27     26.5 Bacter~
## # i 203 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 844 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1   194   195 0.959      0.959 Positive
## 2   185   208 0.954      0.954 Positive

```

```
## 3    185    213    0.957      0.957 Positive
## # i 841 more rows
```

5.2.1 select subgraph by sample name

```
# ---- Extract sample-level subgraphs ----
# For each sample (column of `mat`), an OTU is treated as "present" when
# its abundance is > `min_abundance` (default 0). The induced subgraph of
# `graph_obj` on those present OTUs is returned per sample. With
# `select_sample` + `combine`, a single merged subgraph is also returned.
res <- get_sample_subgraph(
  graph_obj = graph_obj,           # The network built above
  mat       = otu_rare_relative,   # Same matrix used to judge "presence"
  select_sample = colnames(otu_rare_relative)[1:3], # Merge the first 3 samples
  combine     = "union"           # Keep OTUs present in ANY of the 3 samples (v
)

class(res)
```

```
## [1] "list"
```

```
names(res)
```

```
## [1] "sub_graph_all"      "stat_sample"        "sub_graph_select"
```

```
# ---- Inspect outputs ----
# Per-sample summary table: Sample / Node / Edge / Status.
# Includes every sample in `mat`, even those whose subgraph is empty.
res$stat_sample
```

```
##      Sample Node Edge Status
## 1      K01   113   161     OK
## 2      K02   122   164     OK
## 3      K03   102   379     OK
## 4      K04    94   107     OK
## 5      K05    71    63     OK
## 6      K06    97   119     OK
## 7      OE1   138   308     OK
## 8      OE2   135   230     OK
## 9      OE3   119   160     OK
## 10     OE4   118   171     OK
## 11     OE5   111   165     OK
```

```
## 12    OE6   140   234    OK
## 13    WT1   105   130    OK
## 14    WT2    83    59    OK
## 15    WT3   117   171    OK
## 16    WT4   111   222    OK
## 17    WT5    90   103    OK
## 18    WT6   111   162    OK
```

```
# Merged subgraph for the 3 selected samples (a single tbl_graph).
# Its node table carries two extra columns:
#   - n_present_samples : how many of the 3 selected samples a node appears in
#   - present_in_samples: comma-separated sample IDs, ordered by `select_sample`
res$sub_graph_select
```

```
## # A tbl_graph: 181 nodes and 602 edges
## #
## # An undirected simple graph with 34 components
## #
## # Node Data: 181 x 16 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_649 5          5          5          5          27      26.5 Bacter~
## 2 ASV_705 5          5          5          5          27      26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 4 ASV_13~ 5          5          5          5          27      26.5 Bacter~
## 5 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 6 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 7 ASV_24~ 5          5          5          5          27      26.5 Bacter~
## 8 ASV_25~ 5          5          5          5          27      26.4 Bacter~
## 9 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## 10 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## # i 171 more rows
## # i 8 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>, n_present_samples <int>,
## #   present_in_samples <chr>
## #
## # Edge Data: 602 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1   164   165 0.959      0.959 Positive
## 2   158   181 0.957      0.957 Positive
## 3   145   147 0.969      0.969 Positive
## # i 599 more rows
```

```
# detail information
res$sub_graph_select %>%
  tidygraph::activate(., "nodes") %>%
  tibble::as_tibble()
```

```
## # A tibble: 181 x 16
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <dbl>         <dbl>         <dbl>         <dbl>   <dbl>   <dbl>   <chr>
## 1 ASV_649 5           5           5           5       27     26.5 Bacter~
## 2 ASV_705 5           5           5           5       27     26.5 Bacter~
## 3 ASV_12~ 5           5           5           5       27     26.5 Bacter~
## 4 ASV_13~ 5           5           5           5       27     26.5 Bacter~
## 5 ASV_14~ 5           5           5           5       27     26.5 Bacter~
## 6 ASV_14~ 5           5           5           5       27     26.5 Bacter~
## 7 ASV_24~ 5           5           5           5       27     26.5 Bacter~
## 8 ASV_25~ 5           5           5           5       27     26.4 Bacter~
## 9 ASV_28~ 5           5           5           5       27     26.5 Bacter~
## 10 ASV_28~ 5           5           5           5       27     26.5 Bacter~
## # i 171 more rows
## # i 8 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>, n_present_samples <int>,
## #   present_in_samples <chr>
```


Chapter 6

Network layout

Load R Package

```
library(tidyverse) # data manipulation / piping utilities
library(ggNetView) # graph builders + layout/plotting frontend
```

Example data

```
# ---- Built-in example datasets ----

# Relative abundance table of rarefied ASVs/OTUs (rows = ASVs, cols = samples).
data("otu_rare_relative")
dim(otu_rare_relative)
```

```
## [1] 2859  18
```

```
otu_rare_relative[1:5, 1:5]
```

```
##           K01           K02           K03           K04           K05
## ASV_1 0.03306667 0.05453333 0.02013333 0.03613333 0.02686667
## ASV_2 0.05750000 0.03393333 0.06046667 0.05810000 0.07320000
## ASV_3 0.01733333 0.01296667 0.02290000 0.02336667 0.03106667
## ASV_4 0.04266667 0.01093333 0.01416667 0.01933333 0.03346667
## ASV_6 0.02646667 0.01856667 0.02110000 0.02353333 0.03806667
```

```
# Taxonomic annotation; first column must be the ASV/OTU ID.
data("tax_tab")
dim(tax_tab)
```

```
## [1] 2859      8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
##   OTUID Kingdom Phylum      Class      Order
##   <chr>  <chr>    <chr>      <chr>      <chr>
## 1 ASV_2  Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia Spartobacteria Unassigned
## 3 ASV_31 Bacteria Actinobacteria Actinobacteria Actinomycetales
## 4 ASV_27 Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned     Unassigned  Unassigned
```

Build graph object

```
# ---- Build a co-occurrence network used by every layout demo below ----
graph_obj <- build_graph_from_mat(
  mat           = otu_rare_relative, # variables x samples numeric matrix
  transform.method = "none",         # already relative abundance, no extra transform
  r.threshold    = 0.7,              # |r| cutoff for keeping an edge
  p.threshold    = 0.05,             # adjusted p-value cutoff
  method         = "WGCNA",         # correlation backend: WGCNA::corAndPvalue
  cor.method     = "pearson",        # Pearson correlation
  proc           = "BH",             # multiple-testing correction (Benjamini-Hochberg)
  module.method  = "Fast_greedy",    # community detection algorithm
  node_annotation = tax_tab,         # taxonomy joined onto nodes by name
  top_modules    = 15,              # keep top-15 modules; rest -> "Others"
  seed           = 1115             # fix RNG for reproducibility
)

graph_obj
```

```
## # A tbl_graph: 2049 nodes and 9602 edges
```

```
## #
```

```
## # An undirected simple graph with 100 components
```

```
## #
```

```
## # Node Data: 2,049 x 14 (active)
```

	name	modularity	modularity2	modularity3	Modularity	Degree	Strength	Kingdom
	<chr>	<fct>	<ord>	<chr>	<ord>	<dbl>	<dbl>	<chr>
## 1	ASV_916	1	1	1	1	58	50.5	Bacter~
## 2	ASV_777	1	1	1	1	58	48.7	Bacter~
## 3	ASV_606	1	1	1	1	55	45.8	Bacter~
## 4	ASV_740	1	1	1	1	54	47.2	Bacter~
## 5	ASV_14~	1	1	1	1	54	44.5	Bacter~

```
## 6 ASV_23~ 1 1 1 1 54 47.4 Bacter~
## 7 ASV_15~ 1 1 1 1 52 45.3 Bacter~
## 8 ASV_24~ 1 1 1 1 52 43.0 Bacter~
## 9 ASV_19~ 1 1 1 1 52 43.0 Bacter~
## 10 ASV_568 1 1 1 1 51 45.1 Bacter~
## # i 2,039 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## # Genus <chr>, Species <chr>
## #
## # Edge Data: 9,602 x 5
## from to weight correlation corr_direction
## <int> <int> <dbl> <dbl> <chr>
## 1 1771 1825 0.793 0.793 Positive
## 2 594 597 0.895 0.895 Positive
## 3 588 597 0.864 0.864 Positive
## # i 9,599 more rows
```

6.1 Full graph layout

6.1.1 Full network

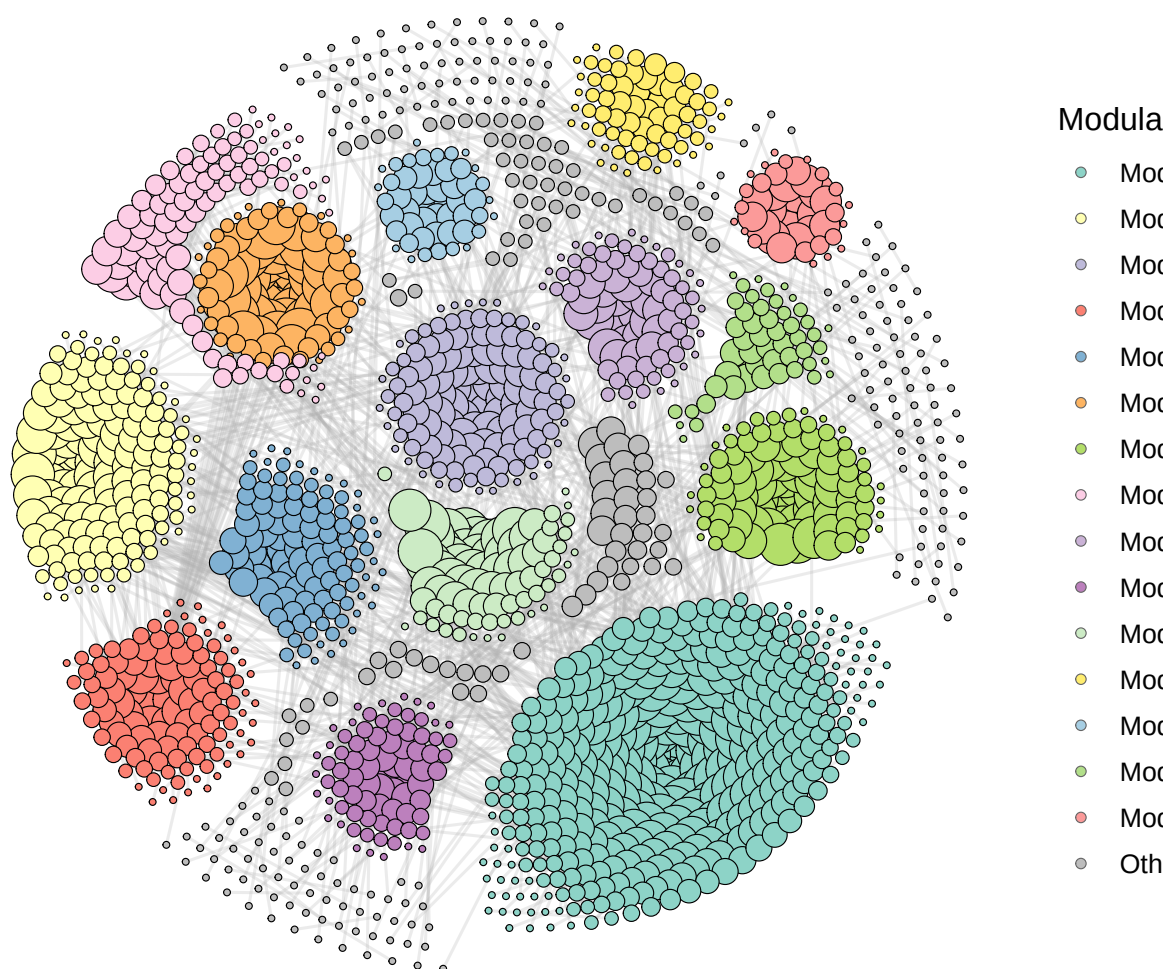
6.1.1.1 Gephi layout

layout.module = "random"

```
# Gephi-style ForceAtlas2-like layout: good general-purpose layout for
# scale-free / community-rich biological networks.
p <- ggNetView(
  graph_obj = graph_obj, # the tbl_graph to draw
  layout = "gephi", # global node-positioning algorithm
  center = F, # don't pull all modules toward the plot center
  shrink = 0.8, # scale each module inward (smaller = tighter modules)
  layout.module = "random", # how modules are placed relative to each other: "random" / "adjacency"
  group.by = "Modularity", # grouping variable used for per-module layout
  fill.by = "Modularity" # node fill colour mapped to module ID
)
p
```

Node = 2049

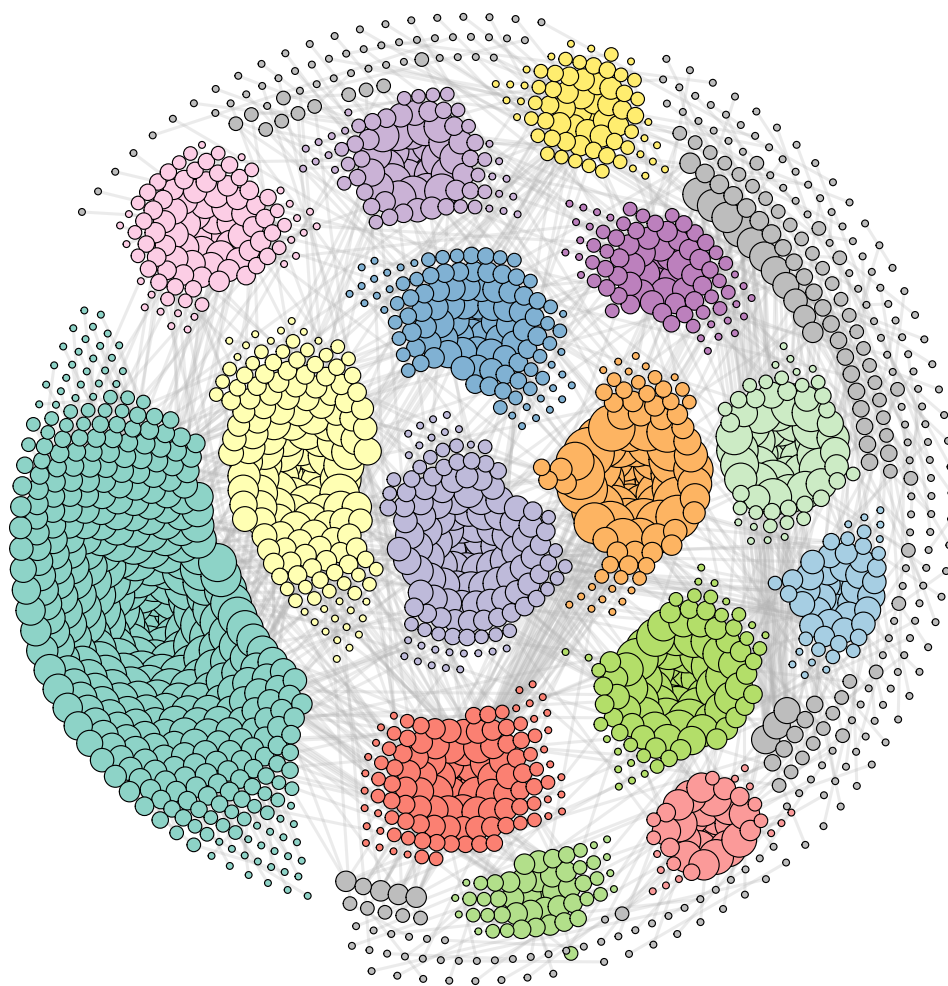
Edge = 9602



```
layout.module = "adjacent"
```

```
# Gephi-style ForceAtlas2-like layout: good general-purpose layout for  
# scale-free / community-rich biological networks.  
p <- ggNetView(  
  graph_obj = graph_obj,    # the tbl_graph to draw  
  layout     = "gephi",      # global node-positioning algorithm  
  center     = F,            # don't pull all modules toward the plot center  
  shrink     = 0.8,          # scale each module inward (smaller = tighter modules)  
  layout.module = "adjacent", # how modules are placed relative to each other: "random" / "adjacent"  
  group.by   = "Modularity", # grouping variable used for per-module layout  
  fill.by    = "Modularity"  # node fill colour mapped to module ID  
)  
p
```

Node = 2049
Edge = 9602



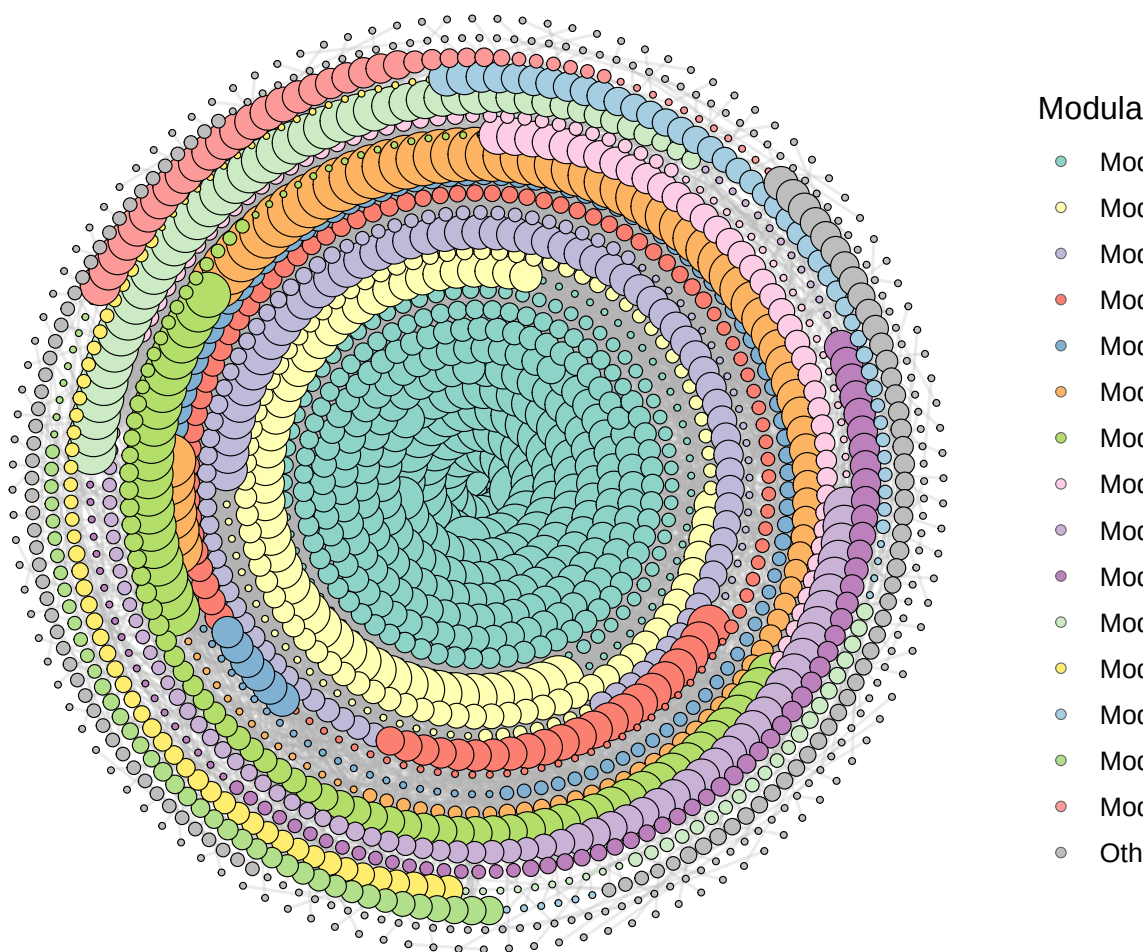
Modula

- Modula 1
- Modula 2
- Modula 3
- Modula 4
- Modula 5
- Modula 6
- Modula 7
- Modula 8
- Modula 9
- Modula 10
- Modula 11
- Modula 12
- Modula 13
- Modula 14
- Modula 15
- Other

```
layout.module = "order"
```

```
# Gephi-style ForceAtlas2-like layout: good general-purpose layout for
# scale-free / community-rich biological networks.
p <- ggNetView(
  graph_obj = graph_obj,    # the tbl_graph to draw
  layout     = "gephi",     # global node-positioning algorithm
  center     = F,           # don't pull all modules toward the plot center
  shrink     = 1,           # scale each module inward (smaller = tighter modules)
  layout.module = "order",  # how modules are placed relative to each other: "random" / "adjac
  group.by   = "Modularity", # grouping variable used for per-module layout
  fill.by    = "Modularity" # node fill colour mapped to module ID
)
p
```

Node = 2049
Edge = 9602



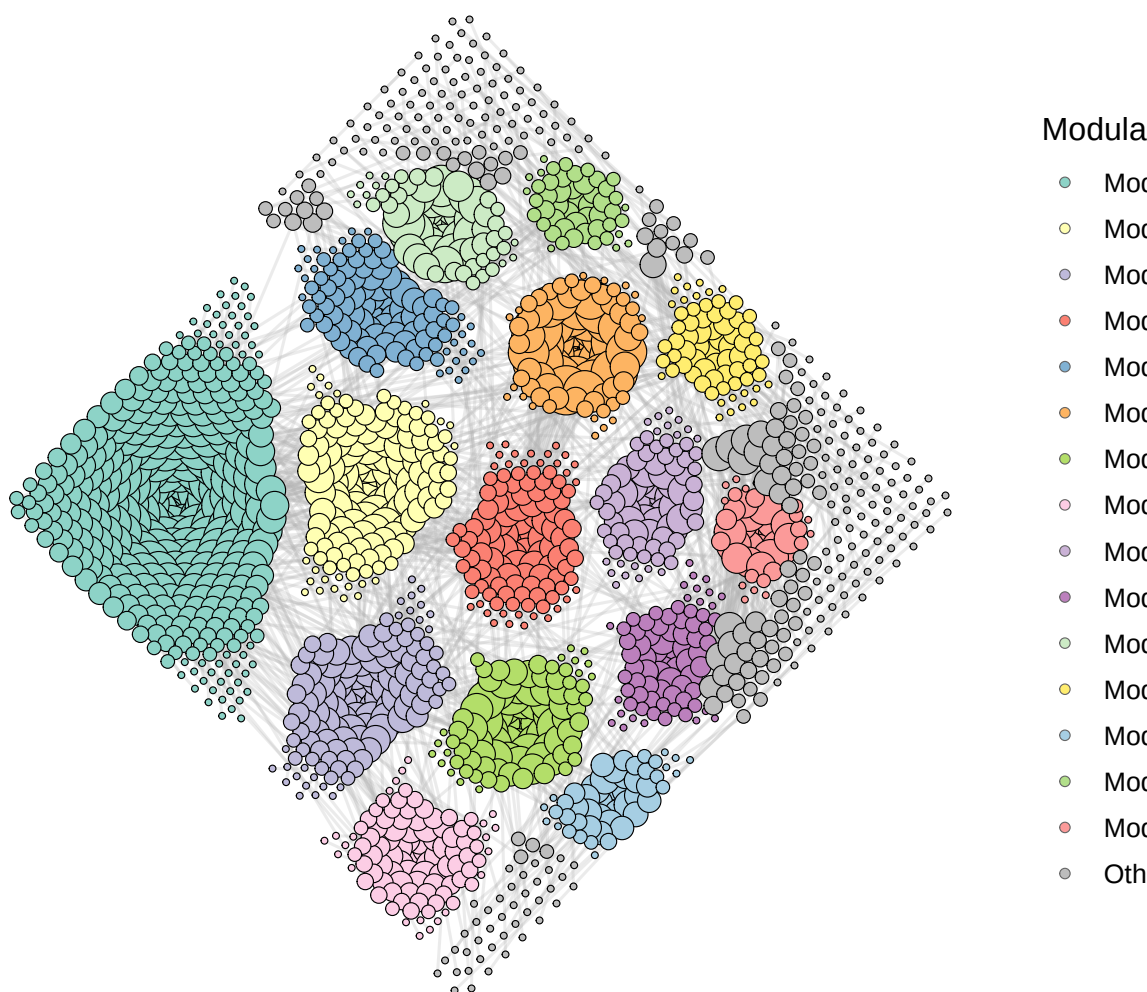
6.1.1.2 Fruchterman-Reingold force-directed layout

```
# FR2: a force-directed layout (springs pull connected nodes, repulsion pushes
# all others apart). Classic choice; reveals dense vs. sparse regions clearly.
p <- ggNetView(
  graph_obj = graph_obj,
  layout    = "fr2",          # Fruchterman-Reingold variant
  center    = F,
  shrink    = 0.8,
  layout.module = "random",  # modules placed independently from each other
  group.by   = "Modularity",
  fill.by    = "Modularity"
)
p
```


6.1.1.3 Diamond layout

```
# "diamond": modules arranged on a rhombic / diamond-shaped lattice.  
# Good for showing many modules side-by-side without overlap.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "diamond",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent", # neighbouring modules placed close together to minimise gaps  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

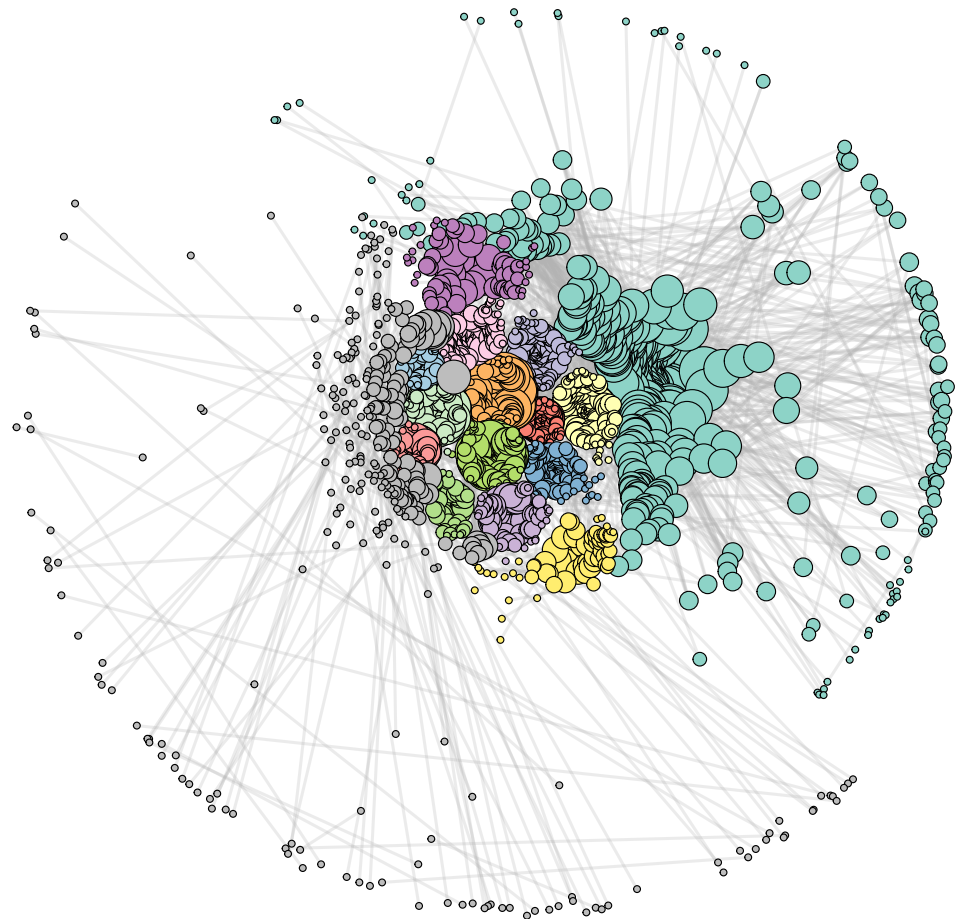
Node = 2049
Edge = 9602



6.1.1.4 KK layout

```
# Kamada-Kawai force-directed layout: like FR but optimises a stress function;  
# tends to give smoother, more even spacing for medium-sized networks.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "kk",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

Node = 2049
Edge = 9602



6.1.1.5 Nicley layout

*# "nicely": igraph's auto-pick layout - chooses a reasonable algorithm based on
graph size and structure. Safe default when you don't want to pick yourself.*

```
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "nicely",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by   = "Modularity",  
  fill.by    = "Modularity"  
)
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 12; retrying with k_nn = 32.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 32; retrying with k_nn = 52.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 52; retrying with k_nn = 72.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 72; retrying with k_nn = 92.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 92; retrying with k_nn = 115.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 115; retrying with k_nn = 144.
```

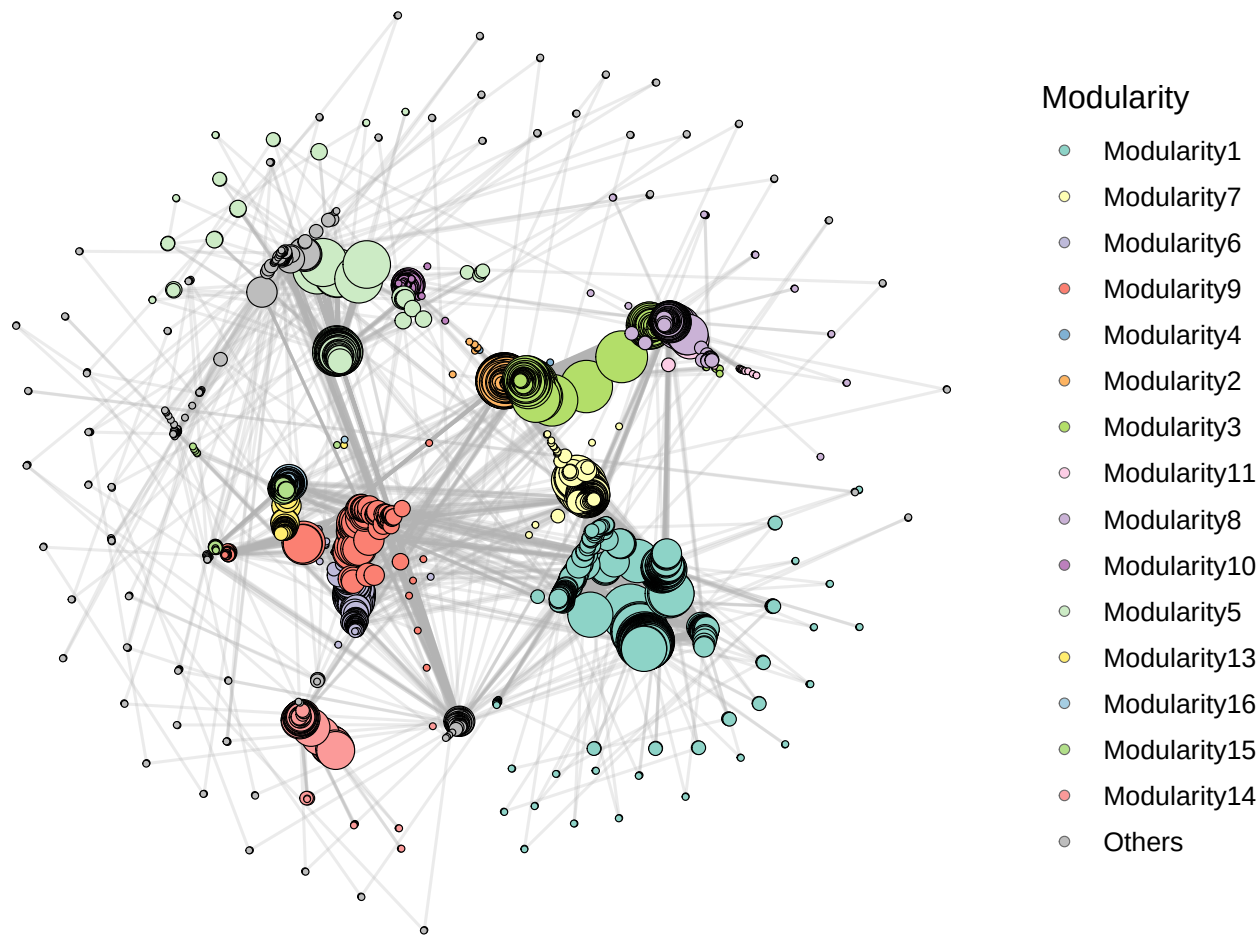
```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 144; retrying with k_nn = 180.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 180; retrying with k_nn = 225.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 225; retrying with k_nn = 282.
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "nicely", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 282; retrying with k_nn = 353.
```

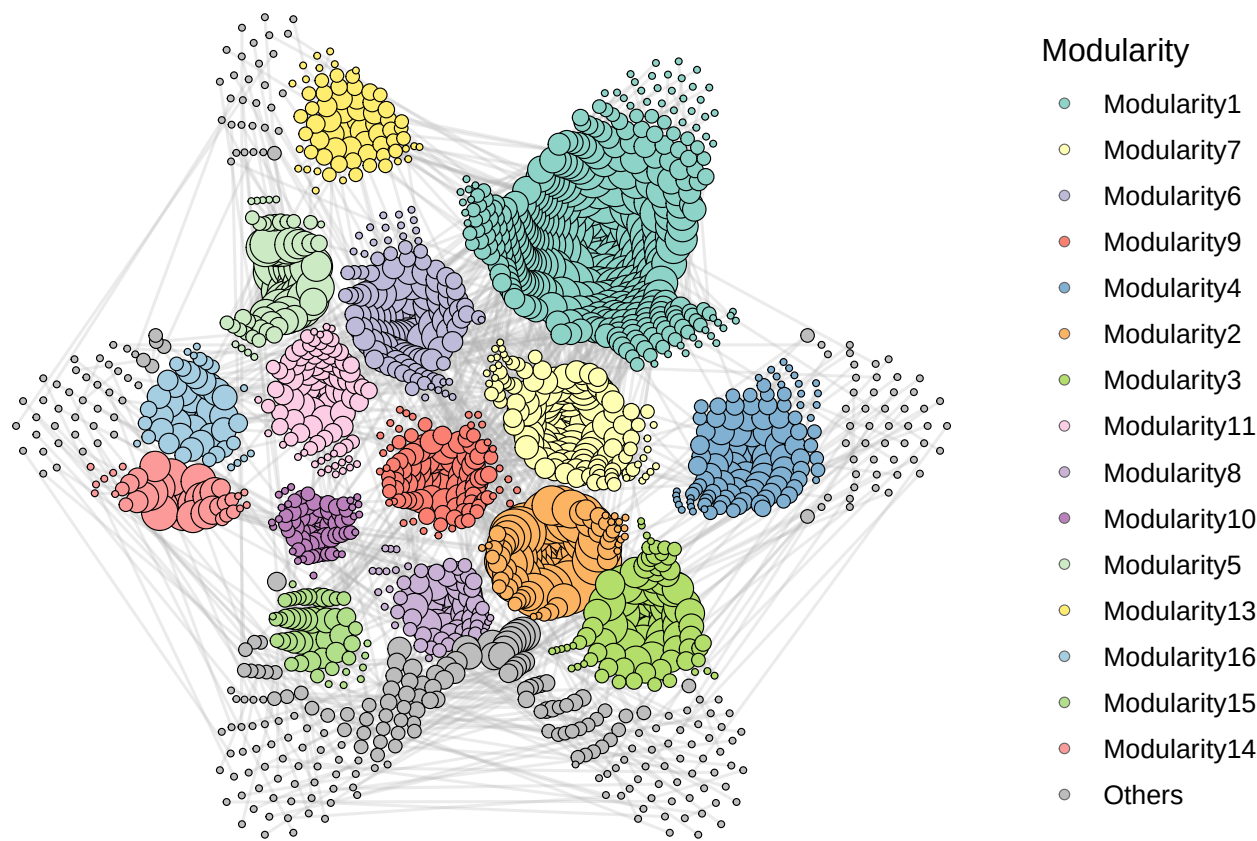

Node = 2049
Edge = 9602



6.1.1.6 Petal layout

```
# "petal": each module is laid out as a circular petal radiating from the  
# global centre. Visually emphasises modular structure.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout    = "petal",  
  center    = F,  
  shrink    = 0.8,  
  layout.module = "adjacent",  
  group.by   = "Modularity",  
  fill.by    = "Modularity"  
)  
p
```

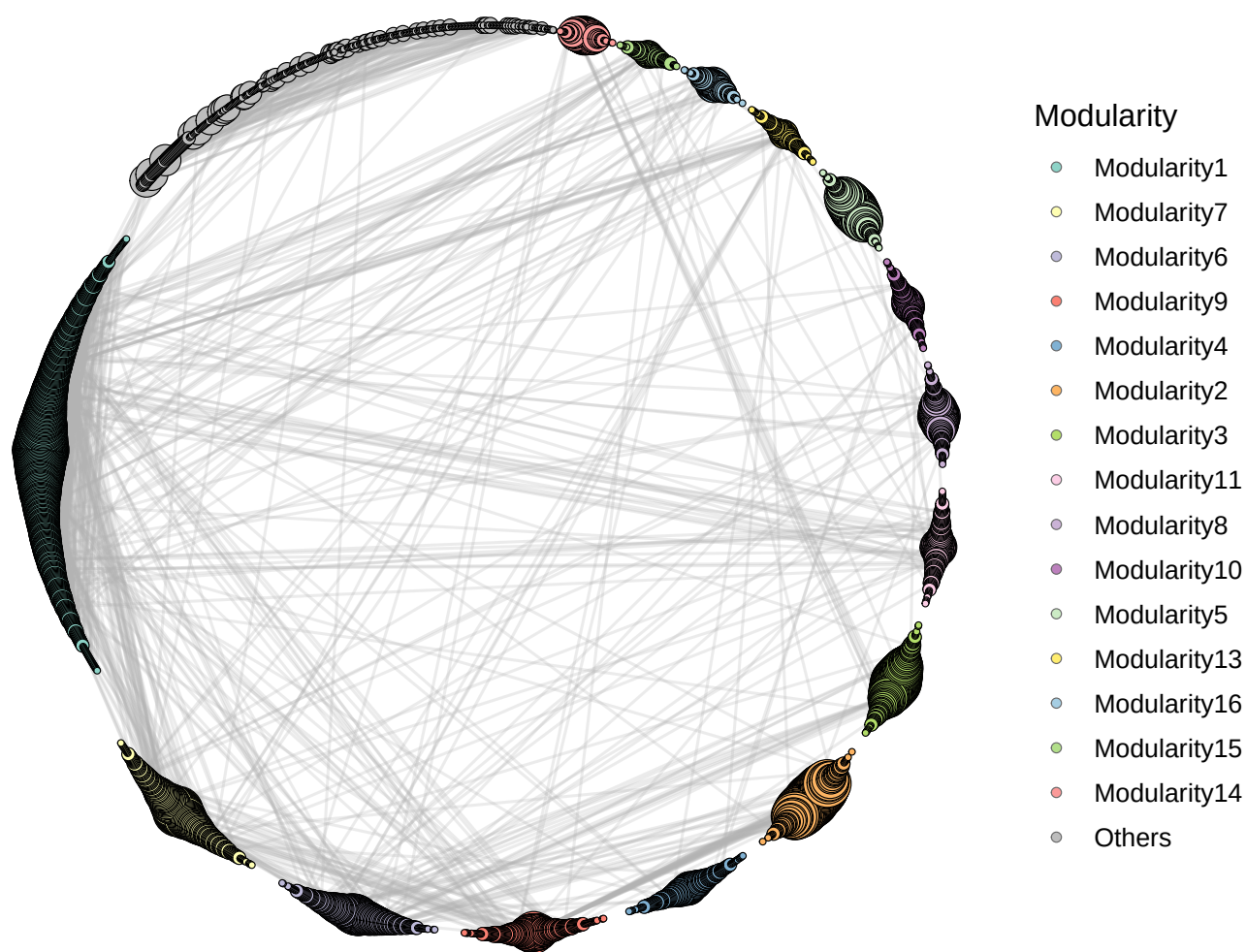
Node = 2049
Edge = 9602



6.1.1.7 Circle layout

```
# "circle": all nodes placed on a single ring. Useful for small networks or for  
# emphasising edges over node positions.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout    = "circle",  
  center    = F,  
  shrink    = 0.8,  
  layout.module = "adjacent",  
  group.by   = "Modularity",  
  fill.by    = "Modularity"  
)  
p
```

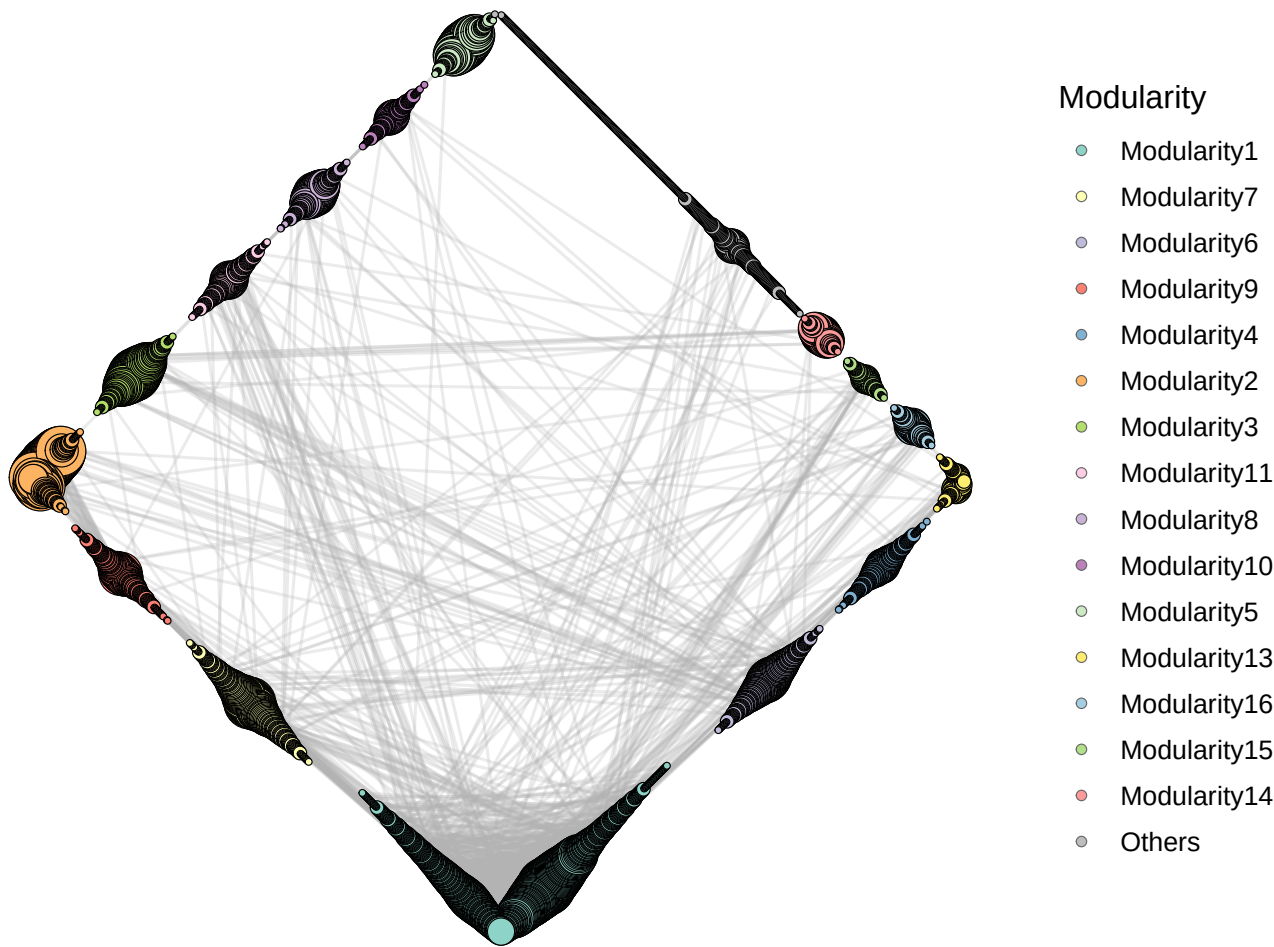
Node = 2049
Edge = 9602



6.1.1.8 Diamond outline layout

```
# "diamond_outline": diamond layout but nodes pushed toward the module border -  
# hollow-looking modules, easier to read edge connections inside each module.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "diamond_outline",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

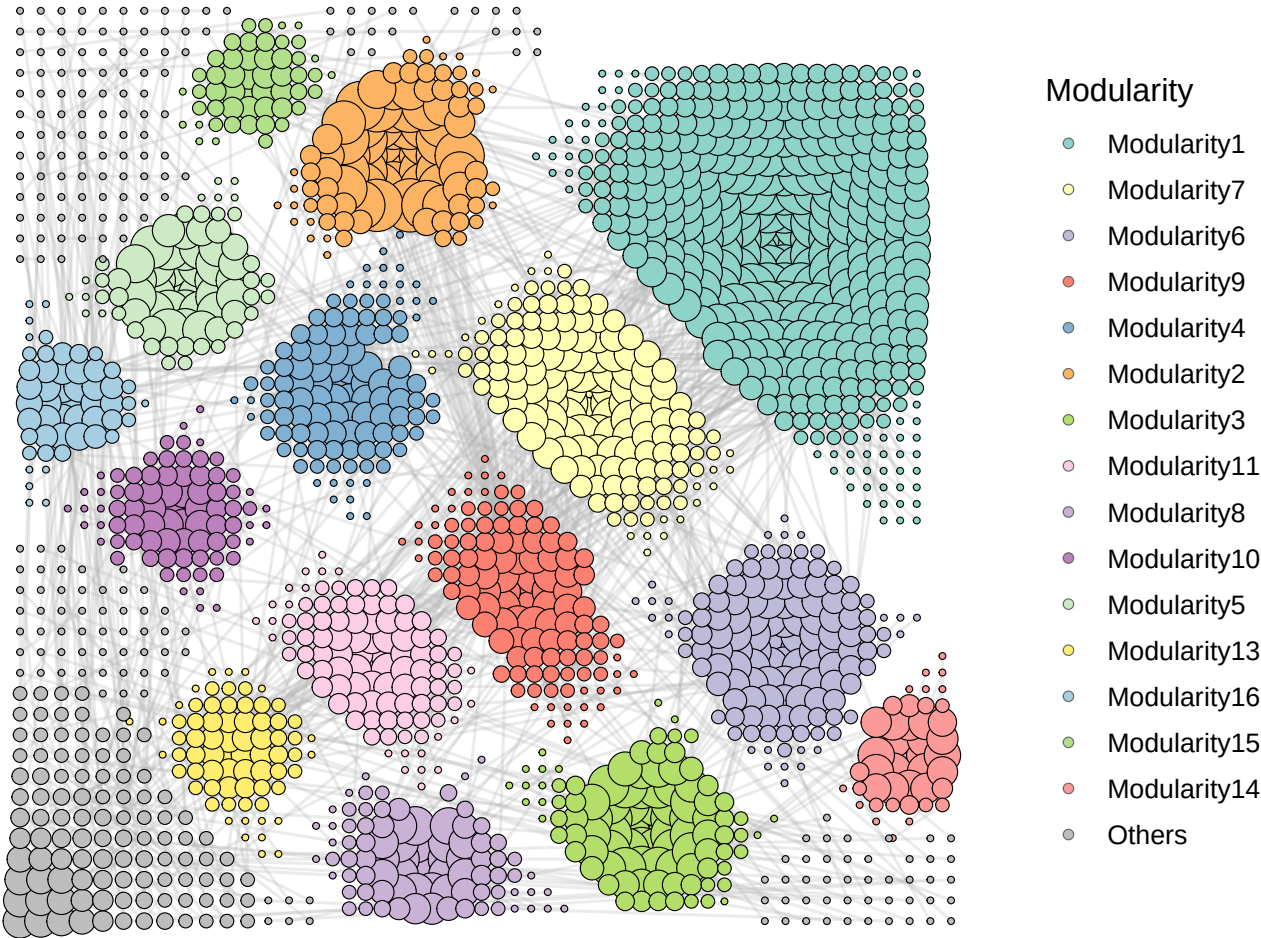
Node = 2049
Edge = 9602



6.1.1.9 Grid layout

```
# "grid": nodes snapped to a regular rectangular grid.  
# Edges may cross a lot, but useful for systematic comparison plots.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout    = "grid",  
  center    = F,  
  shrink    = 0.8,  
  layout.module = "adjacent",  
  group.by   = "Modularity",  
  fill.by    = "Modularity"  
)  
p
```

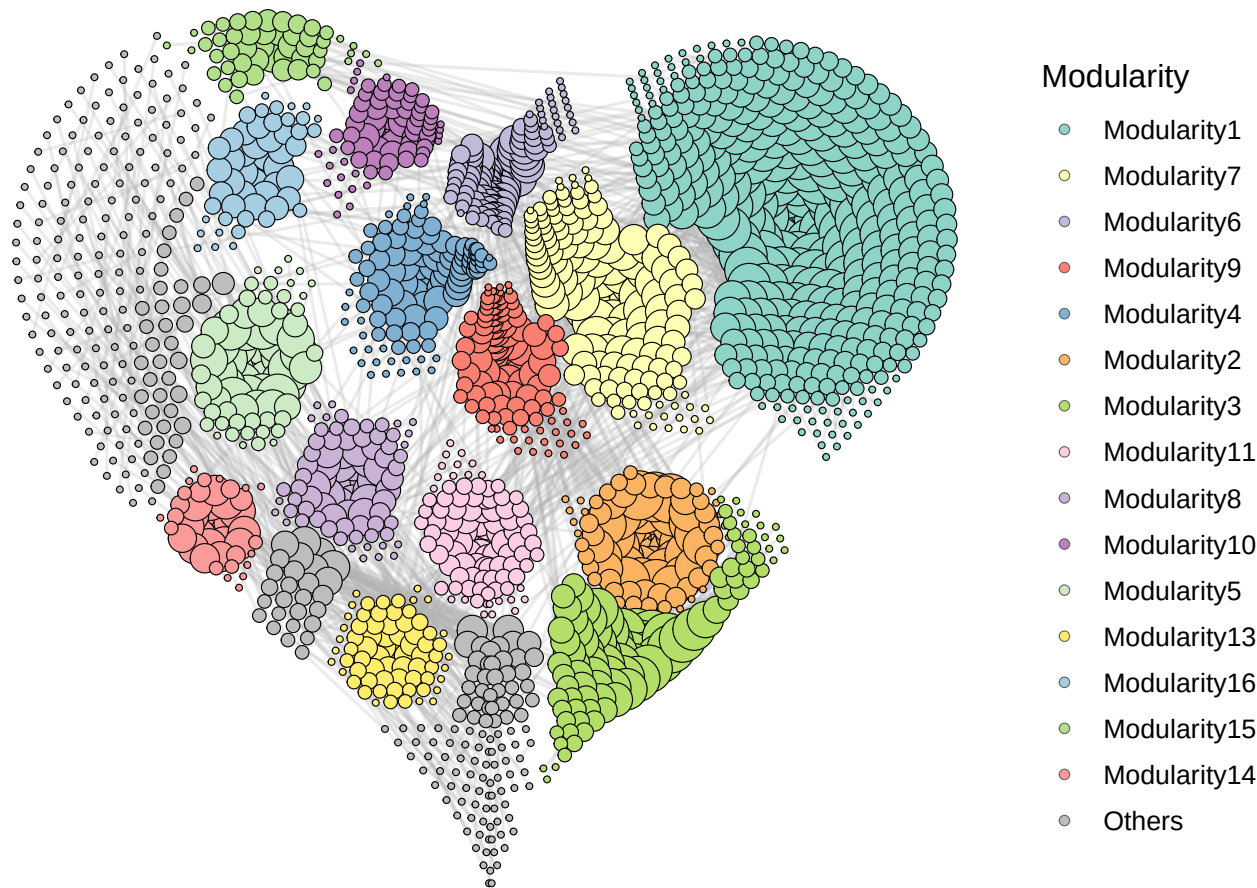
Node = 2049
Edge = 9602



6.1.1.10 Heart_centered layout

```
# "heart_centered": stylised heart-shaped frame with nodes pulled toward the  
# centre. Mostly cosmetic - handy for talks / decorative figures.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "heart_centered",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

Node = 2049
Edge = 9602



6.1.1.11 Lgl layout

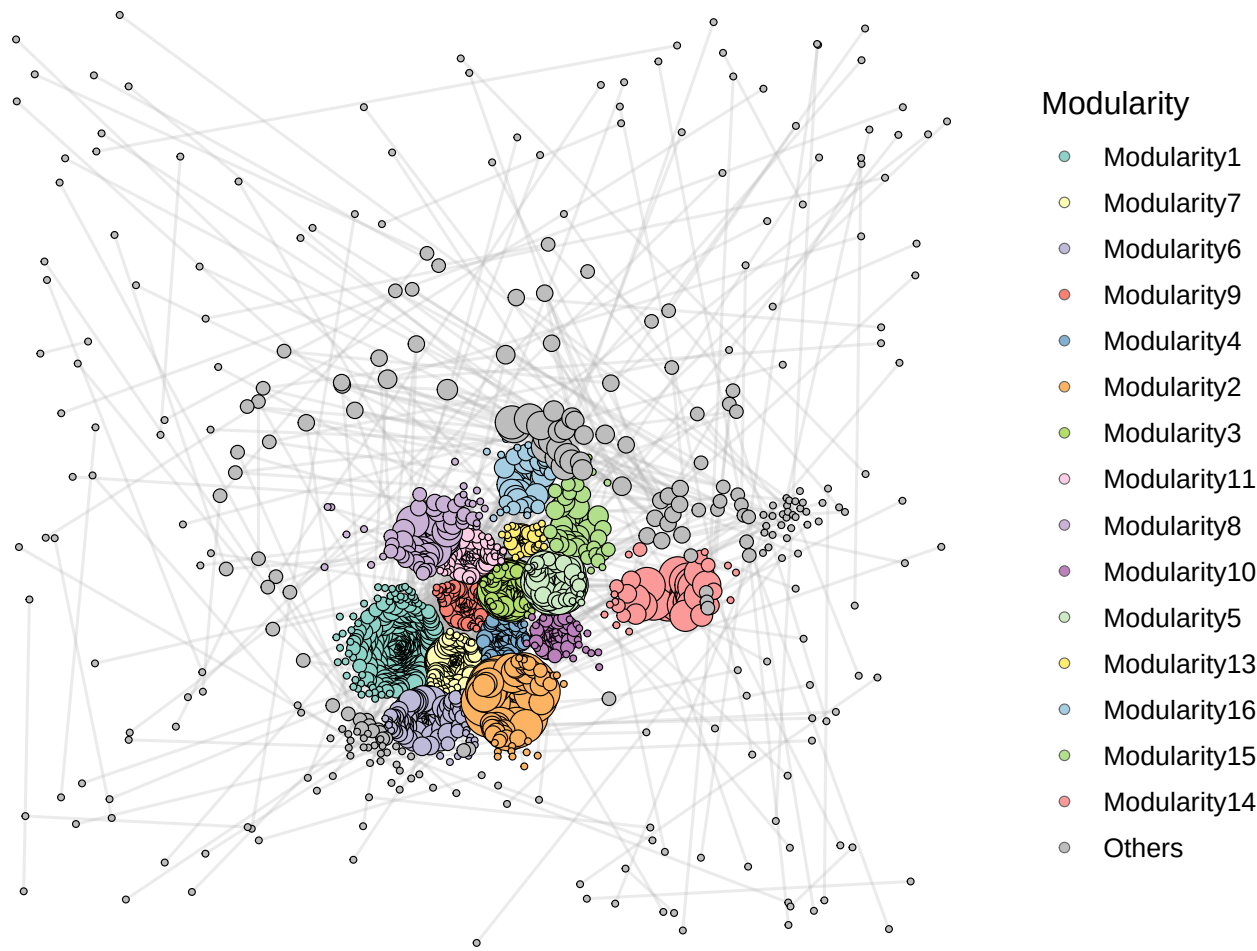
```
# "lgl" (Large Graph Layout): force-directed layout designed for large graphs;  
# spreads nodes radially around dense cores. Scales better than FR/KK.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "lgl",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)
```

```
## Warning in alg_fun(graph): LGL layout does not support disconnected graphs yet.  
## Source: layout/large_graph.c:179
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "lgl", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 12; retrying with k_nn = 32.
```

```
p
```

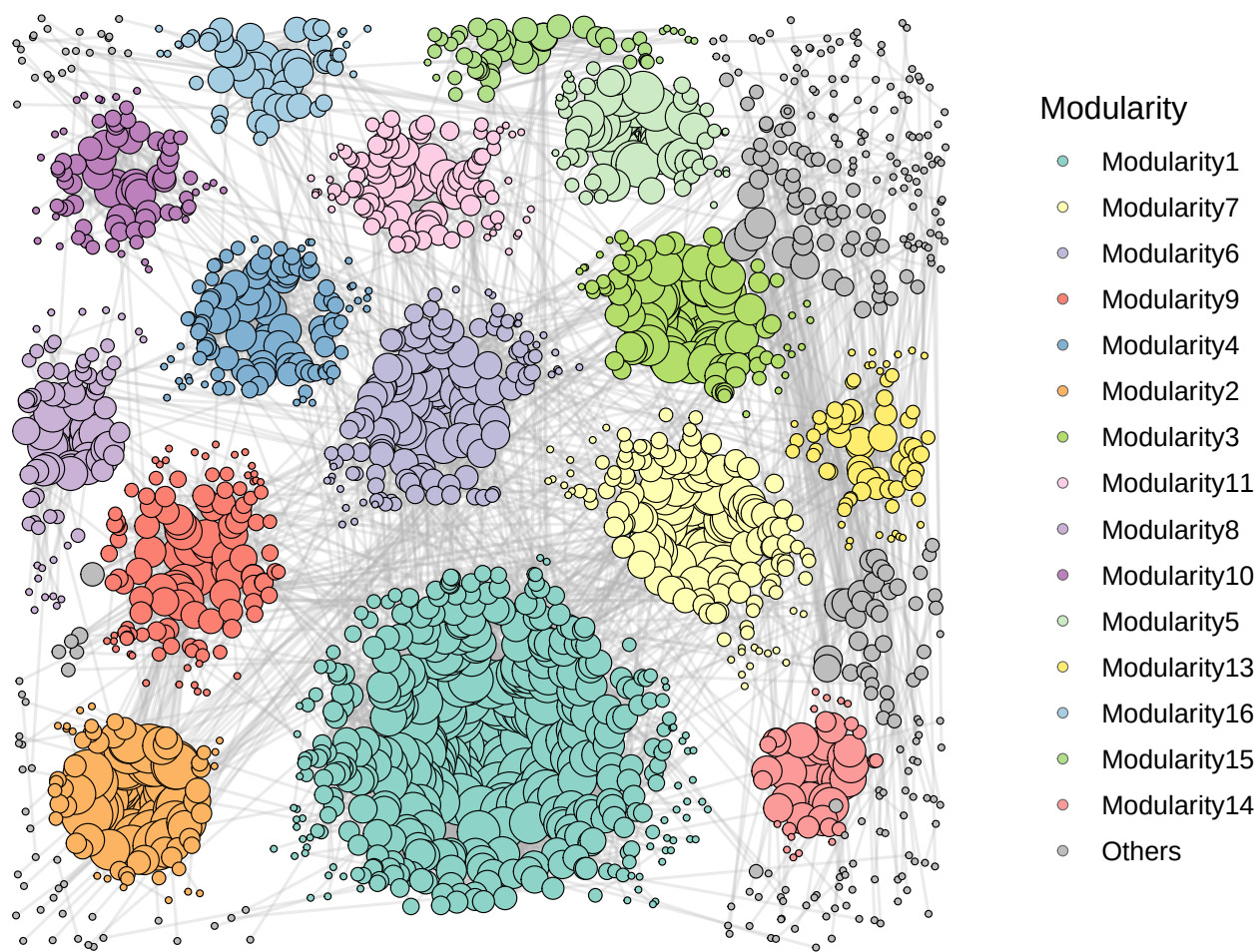
Node = 2049
Edge = 9602



6.1.1.12 Randomly layout

```
# "randomly": nodes placed at uniformly random positions inside the canvas.  
# Baseline / sanity-check layout - not informative on its own.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "randomly",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

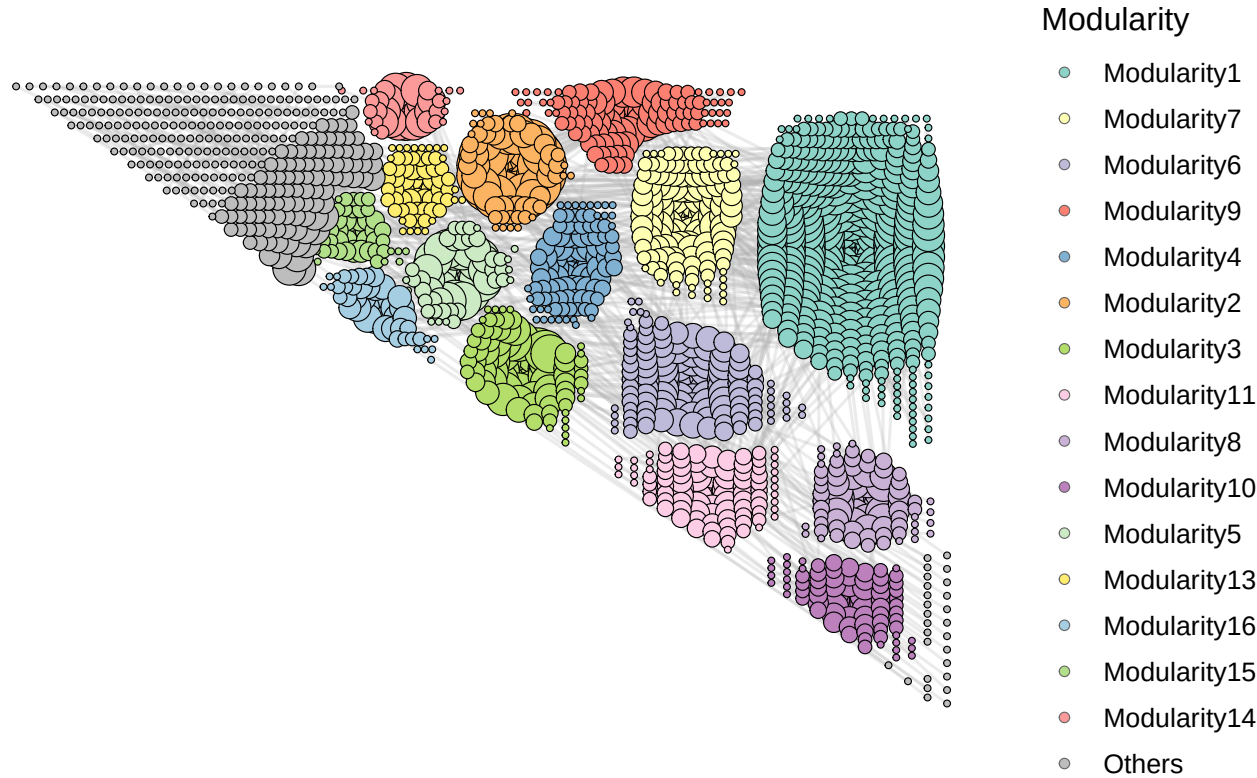
Node = 2049
Edge = 9602



6.1.1.13 Rectangle layout

```
# "rectangle": nodes laid out along a rectangular frame.  
# Good when you want to free up the centre for annotations / edge bundles.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "rectangle",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

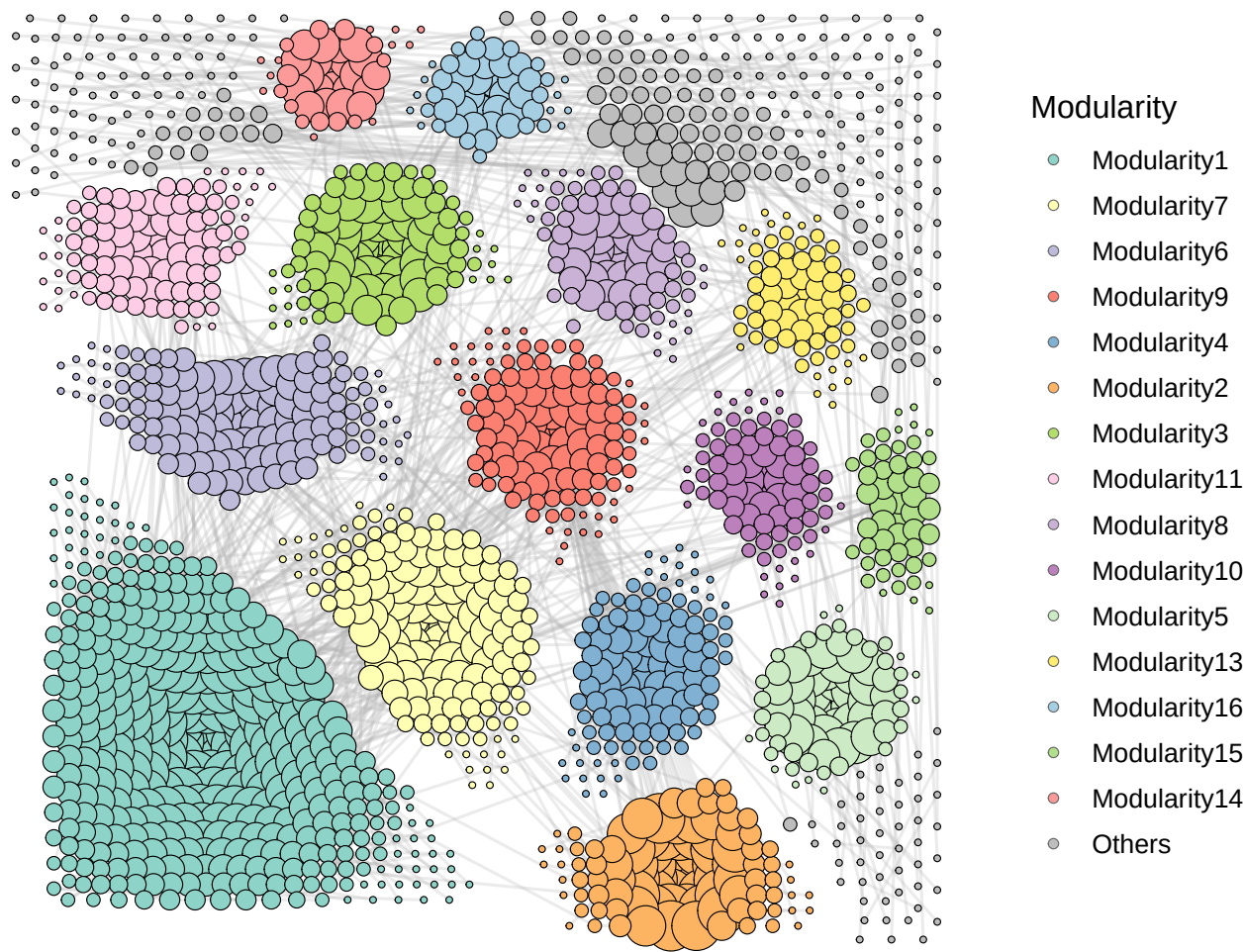
Node = 2049
Edge = 9602



6.1.1.14 Square layout

```
# "square": like rectangle but constrained to a square aspect ratio.
p <- ggNetView(
  graph_obj    = graph_obj,
  layout       = "square",
  center       = F,
  shrink       = 0.8,
  layout.module = "adjacent",
  group.by     = "Modularity",
  fill.by      = "Modularity"
)
p
```

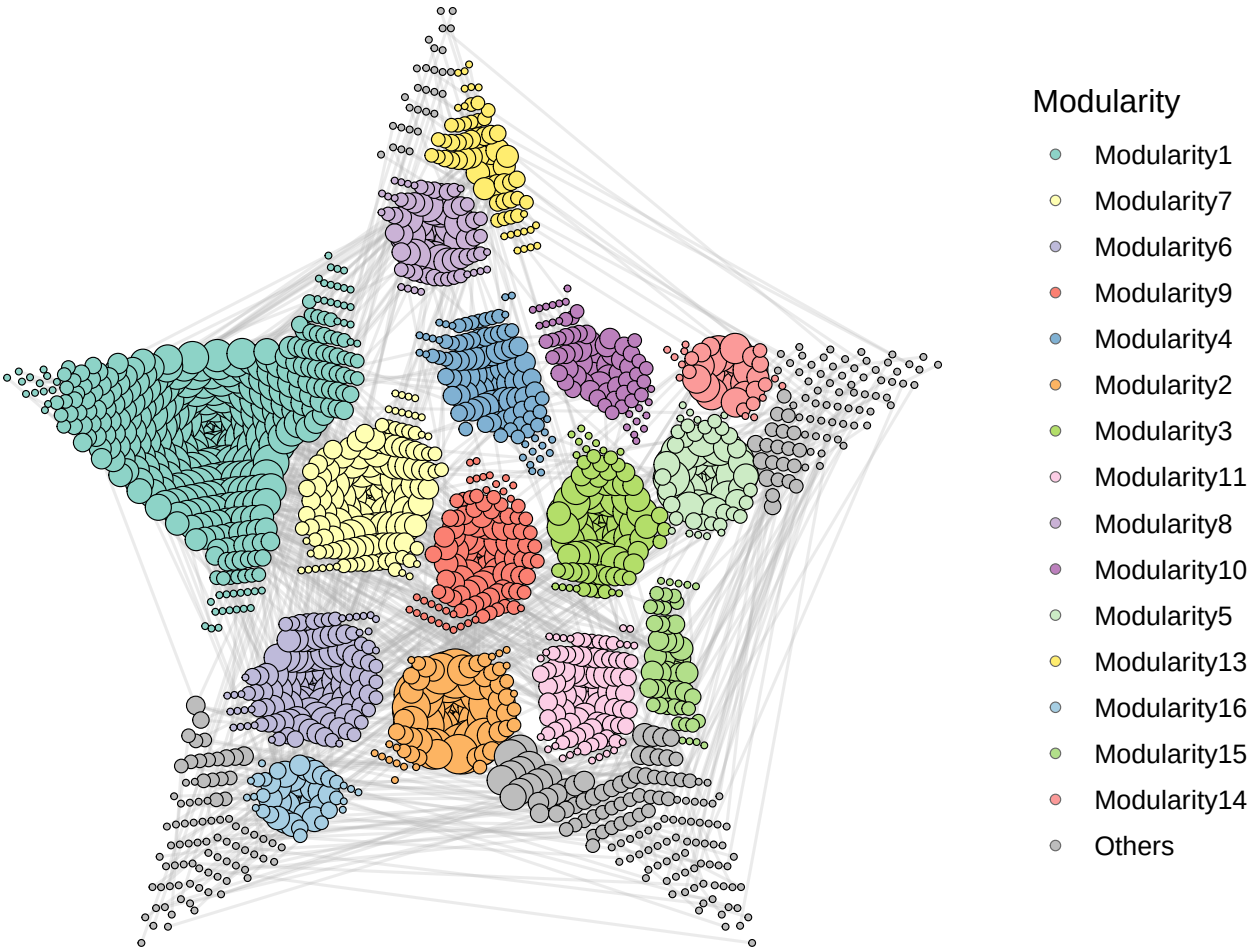
Node = 2049
Edge = 9602



6.1.1.15 Star layout

```
# "star": one central node + others arranged as rays radiating outward.  
# Best when there is a clear hub node; less useful for community-rich graphs.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "star",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

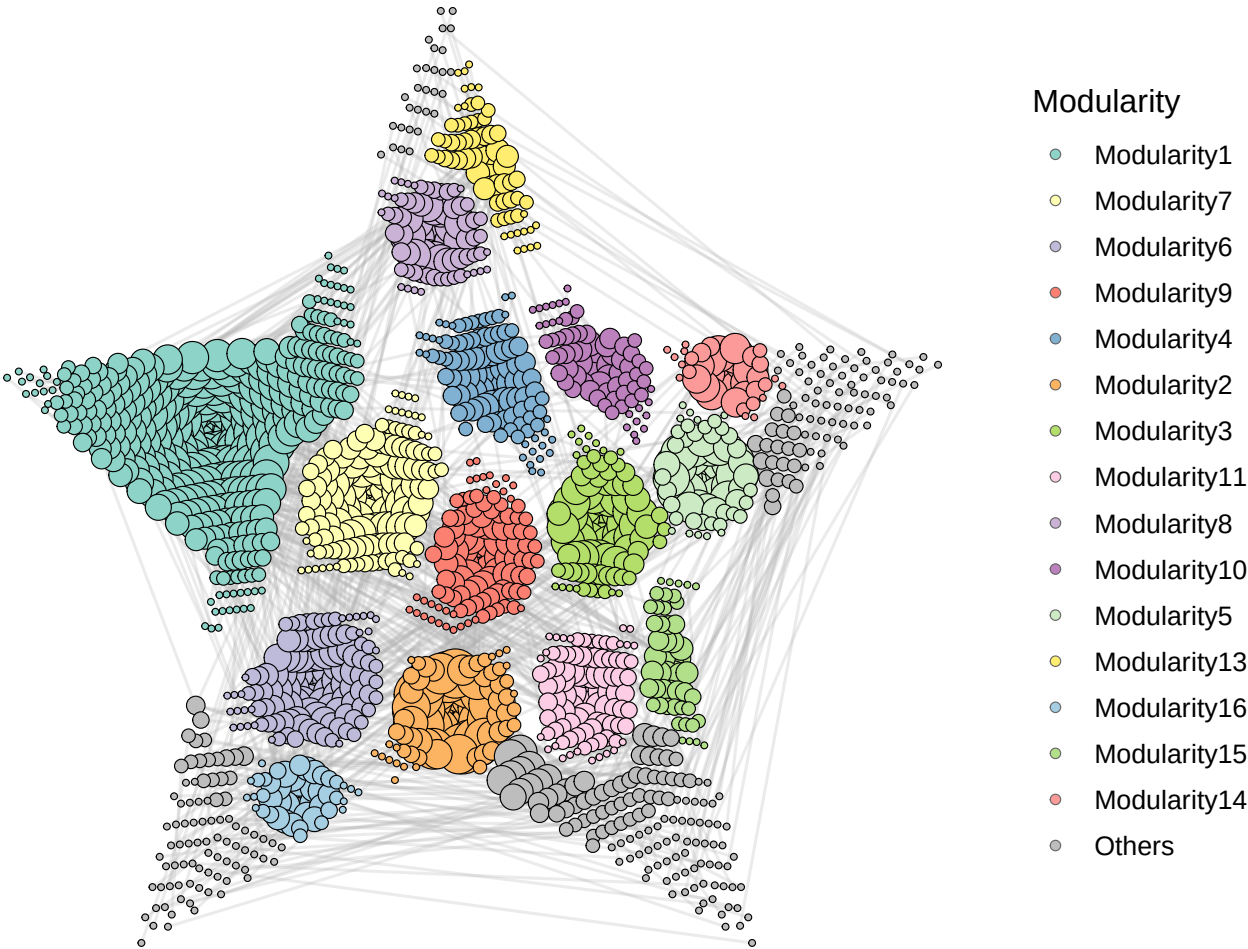
Node = 2049
Edge = 9602



6.1.1.16 Star_concentric layout

```
# "star_concentric": star layout with multiple concentric rings - nodes ranked  
# by some centrality measure end up closer to the centre.  
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout     = "star_concentric",  
  center     = F,  
  shrink     = 0.8,  
  layout.module = "adjacent",  
  group.by    = "Modularity",  
  fill.by     = "Modularity"  
)  
p
```

Node = 2049
Edge = 9602



6.1.1.17 Stress layout

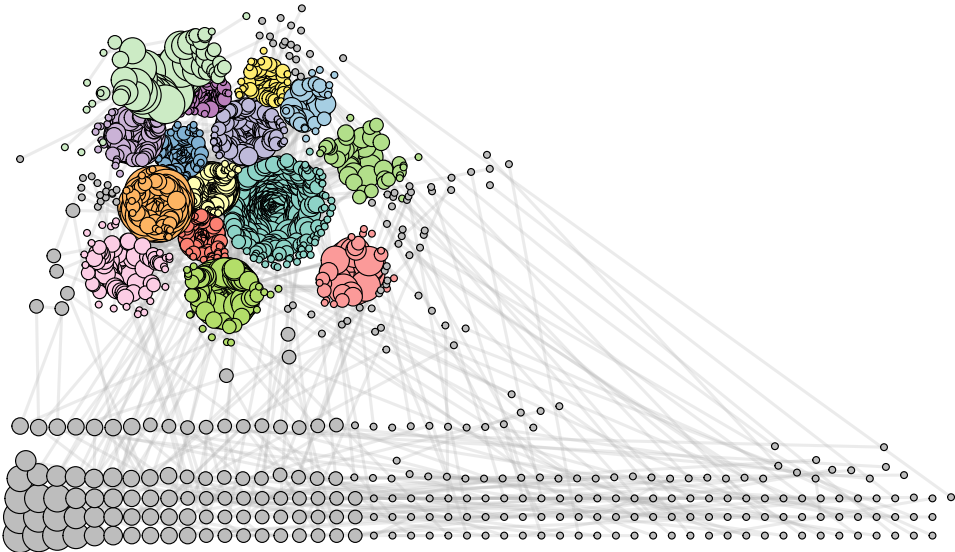
```
# "stress": stress-majorisation layout (from graphlayouts). Generally produces  
# the cleanest "FR/KK-like" results and is reproducible across runs - a strong  
# default for publication figures.
```

```
p <- ggNetView(  
  graph_obj = graph_obj,  
  layout    = "stress",  
  center    = F,  
  shrink    = 0.8,  
  layout.module = "adjacent",  
  group.by   = "Modularity",  
  fill.by    = "Modularity"  
)
```

```
## Warning in ggNetView(graph_obj = graph_obj, layout = "stress", center = F, :  
## `layout.module = 'adjacent'` failed at k_nn = 12; retrying with k_nn = 32.
```

```
p
```

Node = 2049
Edge = 9602

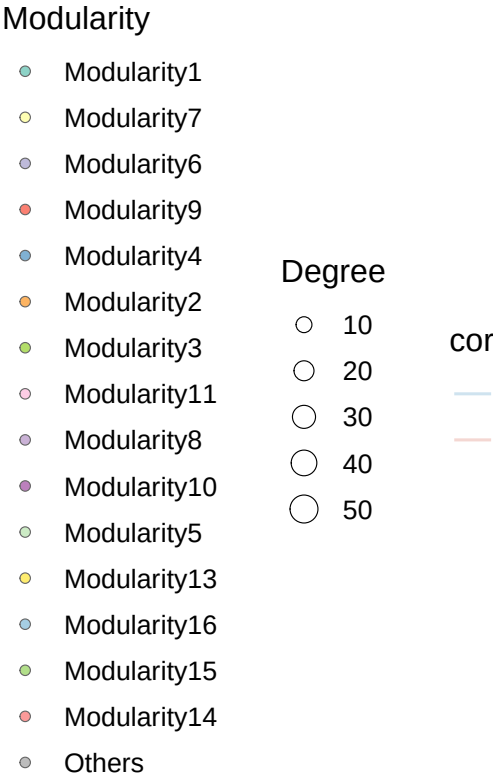
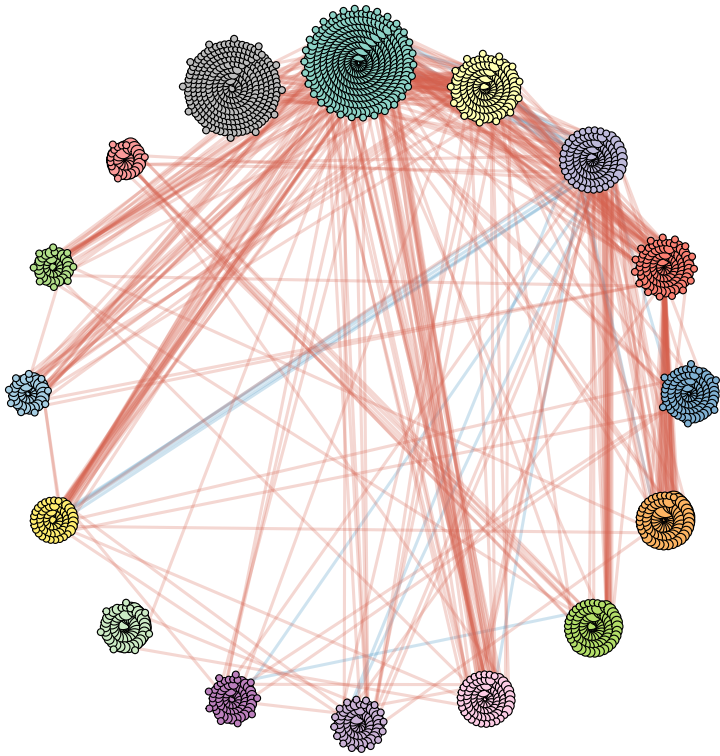


- Modularity
- Modularity1
 - Modularity7
 - Modularity6
 - Modularity9
 - Modularity4
 - Modularity2
 - Modularity3
 - Modularity11
 - Modularity8
 - Modularity10
 - Modularity5
 - Modularity13
 - Modularity16
 - Modularity15
 - Modularity14
 - Others

6.1.1.18 circular_modules_gephi_layout

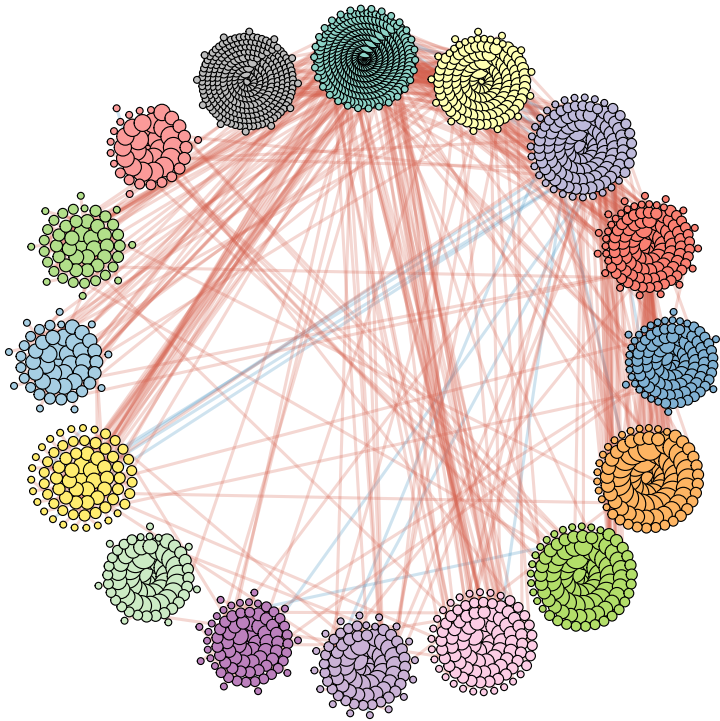
```
p_modules_gephi <- ggNetView(  
  graph_obj = graph_obj,  
  layout = "circular_modules_gephi_layout",  
  center = F,  
  shrink = 0.9,  
  pointsize = c(1, 5),  
  layout.module = "order",  
  group.by = "Modularity",  
  fill.by = "Modularity",  
  anchor_dist = 60,  
  mapping_line = T,  
  r = 1  
)  
  
p_modules_gephi
```

Node = 2049
Edge = 9602
Positive = 9521
Negative = 81



```
p_modules_equal_gephi <- ggNetView(  
  graph_obj = graph_obj,  
  layout = "circular_modules_equal_gephi_layout",  
  center = F,  
  shrink = 0.9,  
  pointsize = c(1, 5),  
  layout.module = "order",  
  group.by = "Modularity",  
  fill.by = "Modularity",  
  anchor_dist = 60,  
  mapping_line = T,  
  r = 1  
)  
  
p_modules_equal_gephi
```

Node = 2049
Edge = 9602
Positive = 9521
Negative = 81



Modularity

- Modularity1
- Modularity7
- Modularity6
- Modularity9
- Modularity4
- Modularity2
- Modularity3
- Modularity11
- Modularity8
- Modularity10
- Modularity5
- Modularity13
- Modularity16
- Modularity15
- Modularity14
- Others

Degree

- 10
- 20
- 30
- 40
- 50

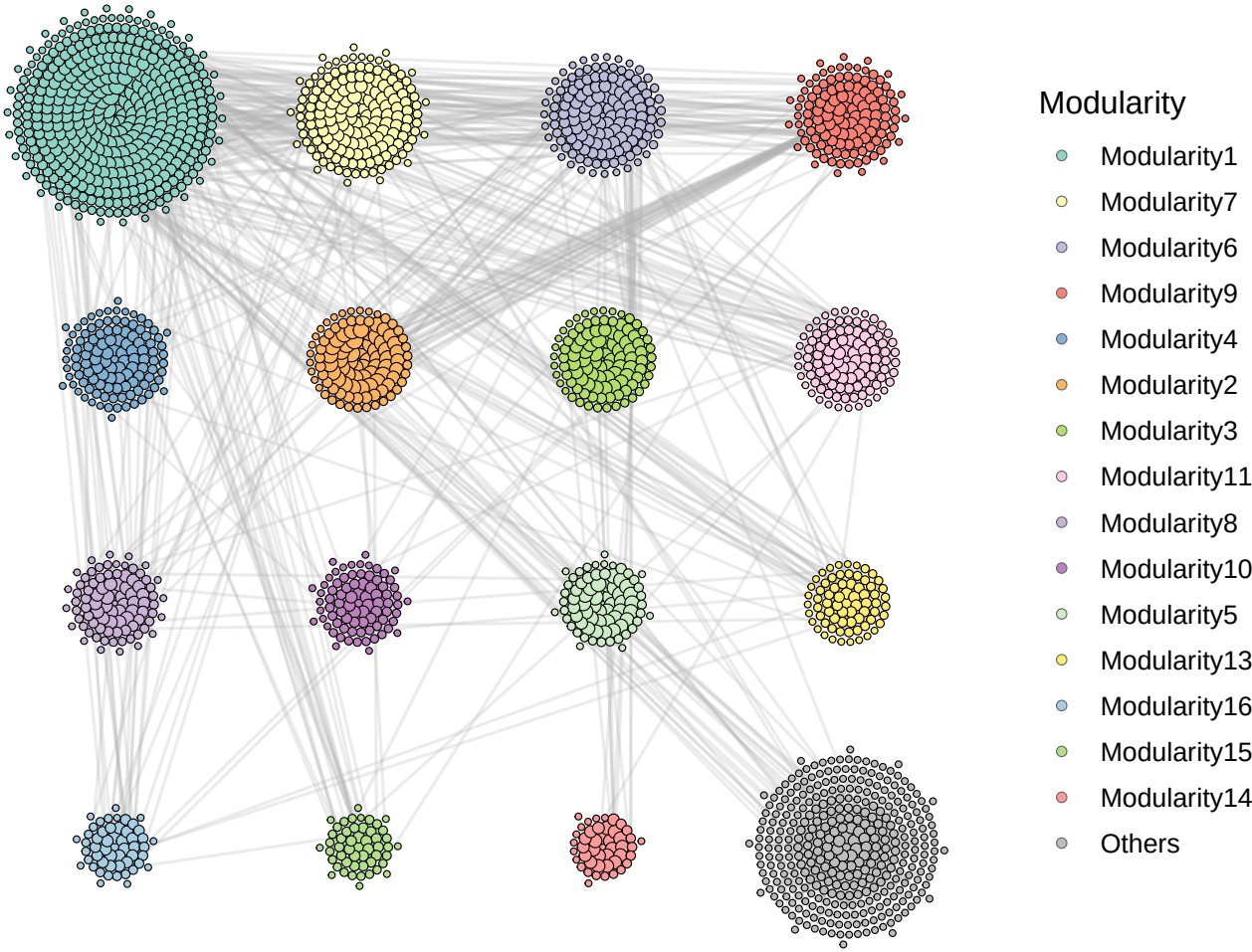
con

—

6.1.1.19 consensus_module_gephi

```
p_consensus_module <- ggNetView(  
  graph_obj = graph_obj,  
  layout = "consensus_module_gephi",  
  center = F,  
  r = 2,  
  node_add = 7,  
  anchor_dist = 50,  
  pointsize = c(1,3),  
  layout.module = "order"  
)  
  
p_consensus_module
```

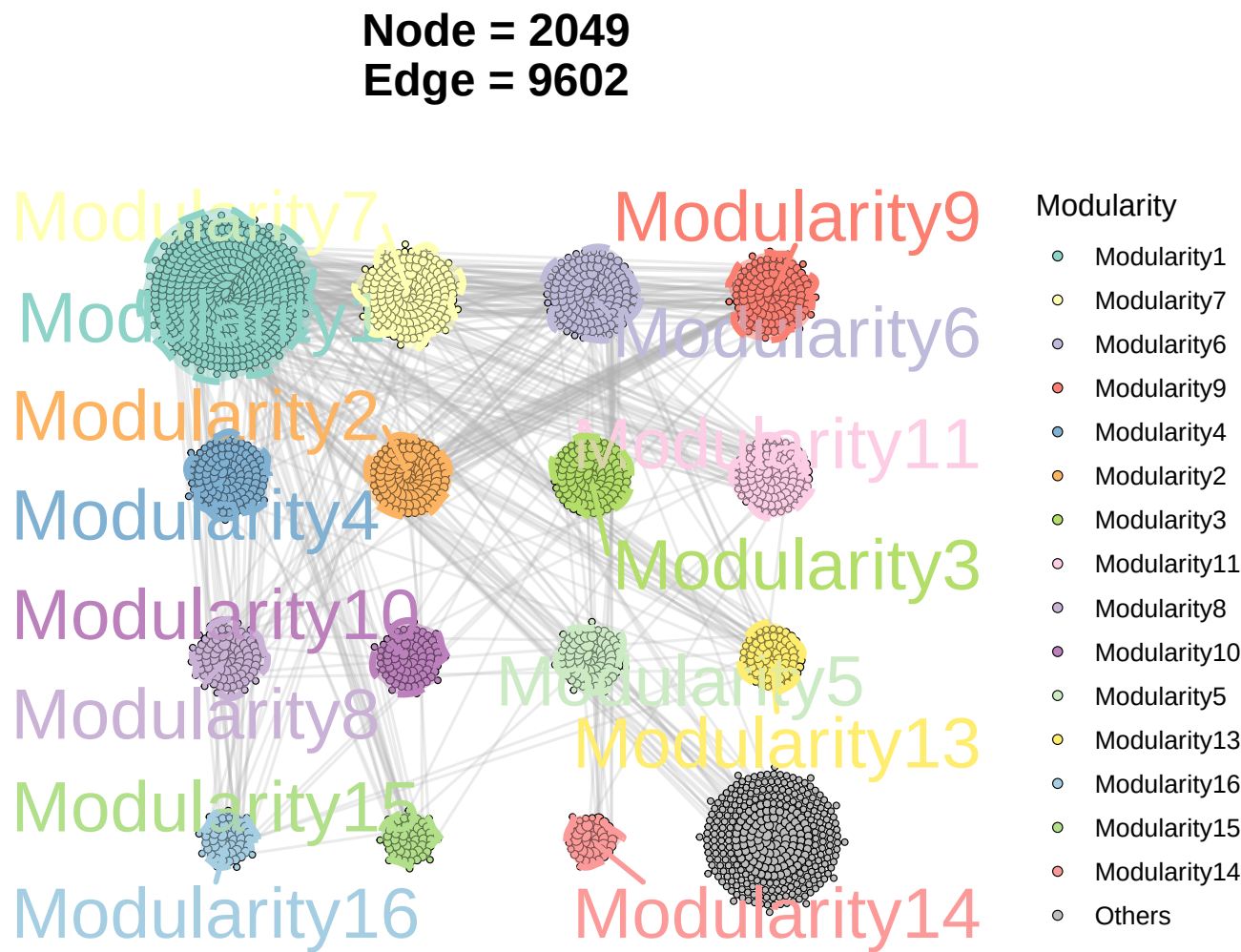
Node = 2049
Edge = 9602



```
p_consensus_module2 <- ggNetView(  
  graph_obj = graph_obj,  
  layout = "consensus_module_gephi",  
  center = F,  
  r = 2,  
  node_add = 7,  
  anchor_dist = 50,  
  pointsize = c(1,3),  
  add_outer = T,  
  expand_outer = 1.3,  
  label = T,  
  layout.module = "order"  
)
```

```
## Coordinate system already present.  
## i Adding new coordinate system, which will replace the existing one.
```

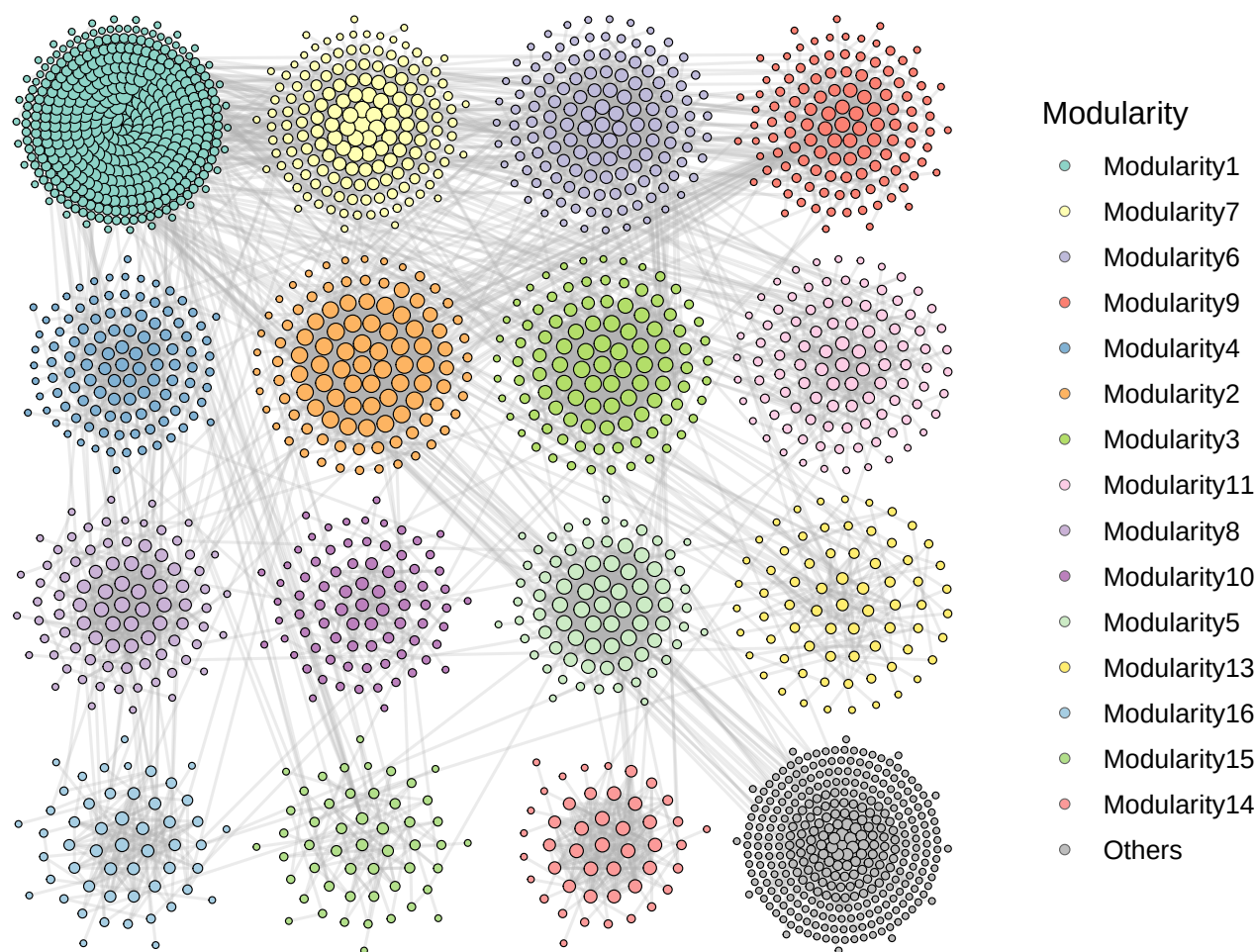
```
p_consensus_module2
```



6.1.1.20 consensus_module_equal_gephi

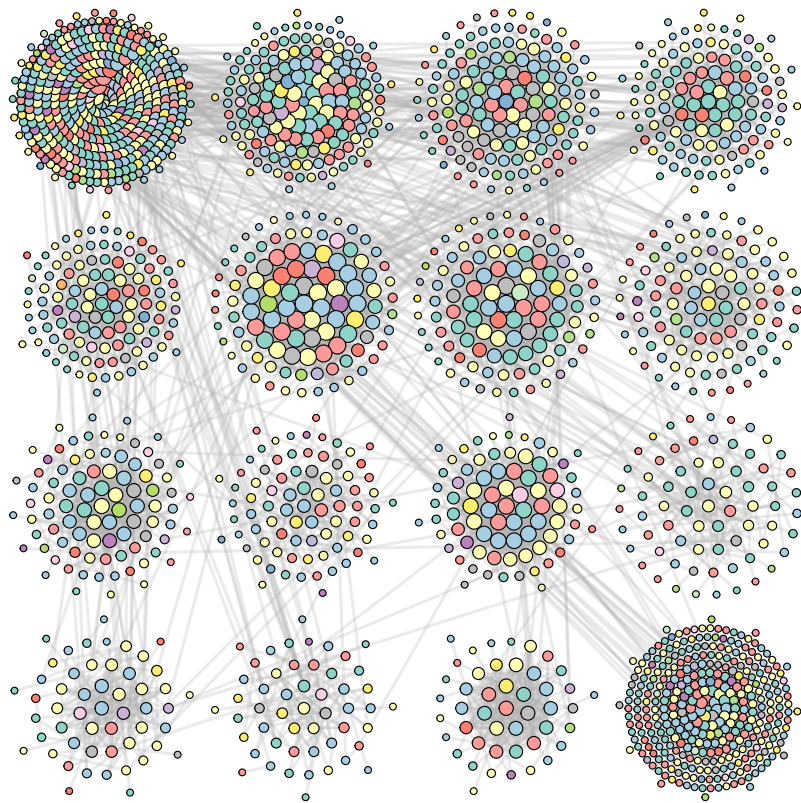
```
p_consensus_module3 <- ggNetView(  
  graph_obj = graph_obj,  
  layout = "consensus_module_equal_gephi",  
  center = F,  
  r = 2,  
  node_add = 7,  
  anchor_dist = 50,  
  pointsize = c(1,3),  
  layout.module = "order"  
)  
  
p_consensus_module3
```

Node = 2049
Edge = 9602



```
p_consensus_module4 <- ggNetView(  
  graph_obj = graph_obj,  
  layout = "consensus_module_equal_gephi",  
  fill.by = "Phylum",  
  center = F,  
  r = 2,  
  node_add = 7,  
  anchor_dist = 50,  
  pointsize = c(1,3),  
  layout.module = "order"  
)  
  
p_consensus_module4
```

Node = 2049
Edge = 9602

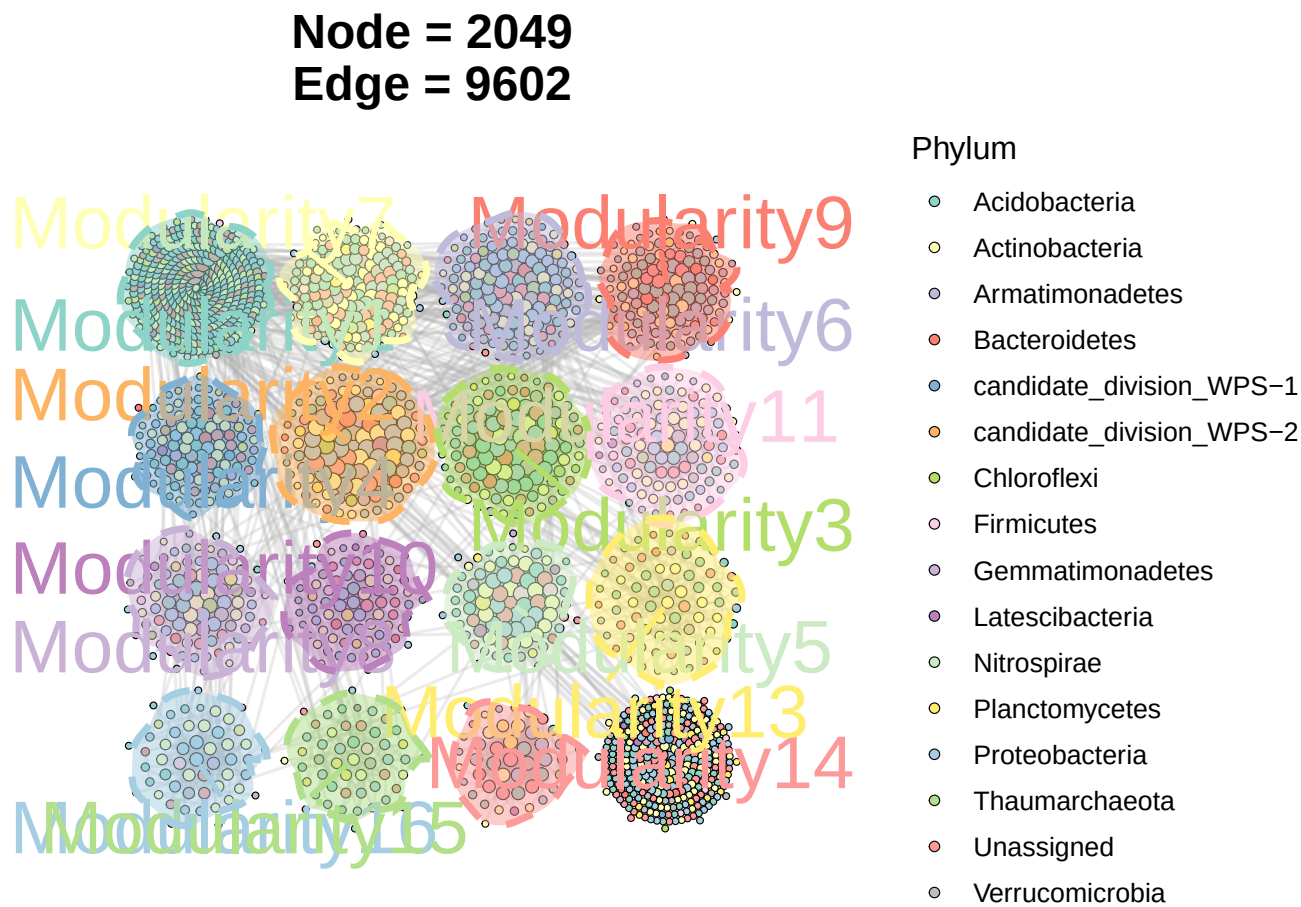


- Phylum
- Acidobacteria
 - Actinobacteria
 - Armatimonadetes
 - Bacteroidetes
 - candidate_division_WPS-1
 - candidate_division_WPS-2
 - Chloroflexi
 - Firmicutes
 - Gemmatimonadetes
 - Latescibacteria
 - Nitrospirae
 - Planctomycetes
 - Proteobacteria
 - Thaumarchaeota
 - Unassigned
 - Verrucomicrobia

```
p_consensus_module5 <- ggNetView(  
  graph_obj = graph_obj,  
  layout = "consensus_module_equal_gephi",  
  fill.by = "Phylum",  
  center = F,  
  r = 2,  
  node_add = 7,  
  anchor_dist = 50,  
  pointsize = c(1,3),  
  add_outer = T,  
  expand_outer = 1.3,  
  label = T,  
  layout.module = "order"  
)
```

```
## Coordinate system already present.  
## i Adding new coordinate system, which will replace the existing one.
```

```
p_consensus_module5
```



6.2 Sub graph layout

6.2.1 bipartite network

```
# Extract the subgraph corresponding to module "1" and "7".
# Internally, get_subgraph():
# 1) splits all nodes by the `Modularity` attribute,
# 2) builds one subgraph per module and stores them in `sub_graph_all`,
# 3) if `select_module` is provided, also returns `sub_graph_select`
# (the graph filtered to those modules via tidygraph::filter),
# 4) returns `stat_module`: a table of node counts per module across the full graph.
graph_sub <- get_subgraph(graph_obj = graph_obj,
                          select_module = c("1", "7")
                          )
```

```
##      Module Number
## 1         1     416
## 2         7     161
## 3         6     137
## 4         9     121
## 5         4     112
## 6         2     105
## 7         3     104
## 8        11     101
## 9         8      87
## 10        10     80
## 11         5     78
## 12        13     70
## 13        16     52
## 14        15     51
## 15        14     46
## 16 Others    328
```

```
graph_sub$sub_graph_select
```

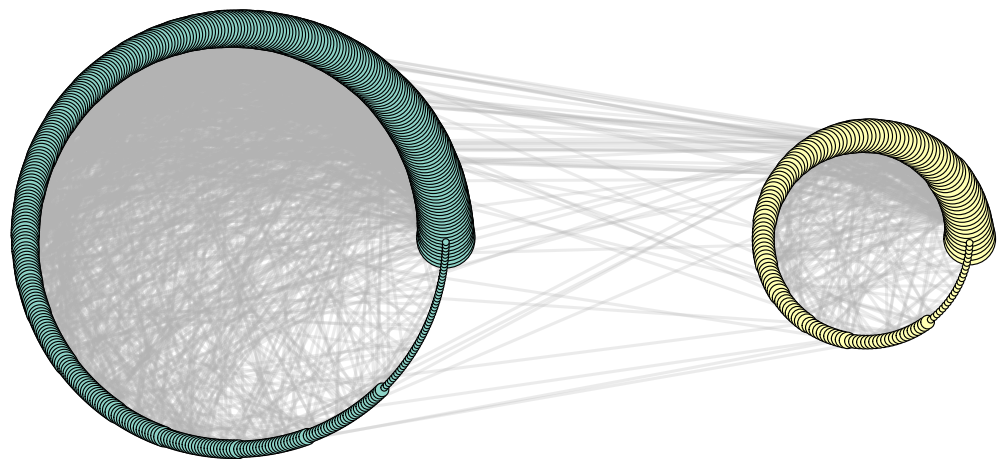
```
## # A tbl_graph: 577 nodes and 3865 edges
## #
## # An undirected simple graph with 1 component
## #
## # Node Data: 577 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_916 1          1          1          1          58      50.5 Bacter~
```

```
## 2 ASV_777 1 1 1 1 58 48.7 Bacter~
## 3 ASV_606 1 1 1 1 55 45.8 Bacter~
## 4 ASV_740 1 1 1 1 54 47.2 Bacter~
## 5 ASV_14~ 1 1 1 1 54 44.5 Bacter~
## 6 ASV_23~ 1 1 1 1 54 47.4 Bacter~
## 7 ASV_15~ 1 1 1 1 52 45.3 Bacter~
## 8 ASV_24~ 1 1 1 1 52 43.0 Bacter~
## 9 ASV_19~ 1 1 1 1 52 43.0 Bacter~
## 10 ASV_568 1 1 1 1 51 45.1 Bacter~
## # i 567 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## # Genus <chr>, Species <chr>
## #
## # Edge Data: 3,865 x 5
## from to weight correlation corr_direction
## <int> <int> <dbl> <dbl> <chr>
## 1 197 337 0.798 -0.798 Negative
## 2 210 337 0.801 -0.801 Negative
## 3 35 196 0.828 0.828 Positive
## # i 3,862 more rows
```

6.2.1.1 bipartite_layout

```
p1 <- ggNetView(
  graph_obj = graph_sub$sub_graph_select,
  layout = "bipartite_layout",
  layout.module = "order",
  center = F,
  scale = T
)
p1
```

Node = 577
Edge = 3865



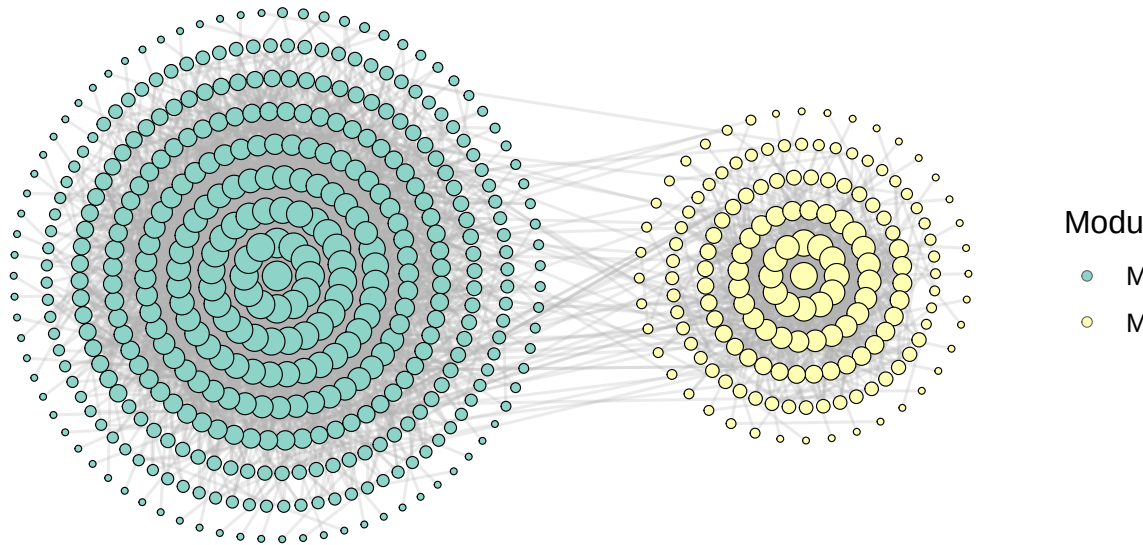
Modul

- Mo
- Mo

6.2.1.2 bipartite_gephi_layout

```
p2 <- ggNetView(  
  graph_obj = graph_sub$sub_graph_select,  
  layout = "bipartite_gephi_layout",  
  layout.module = "order",  
  node_add = 12,  
  anchor_dist = 8,  
  pointsize = c(1, 5),  
  center = F,  
  scale = F  
)  
p2
```

Node = 577
Edge = 3865



6.2.2 tripartite network

```
graph_sub <- get_subgraph(graph_obj = graph_obj,
                          select_module = c("1", "7", "6")
                          )
```

```
##      Module Number
## 1         1      416
## 2         7      161
## 3         6      137
## 4         9      121
## 5         4      112
## 6         2      105
## 7         3      104
## 8        11      101
## 9         8       87
## 10        10       80
## 11         5       78
## 12        13       70
## 13        16       52
## 14        15       51
## 15        14       46
## 16 Others      328
```

```
graph_sub$sub_graph_select
```

```
## # A tbl_graph: 714 nodes and 4448 edges
## #
## # An undirected simple graph with 1 component
## #
## # Node Data: 714 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_916 1          1          1          1          58      50.5 Bacter~
## 2 ASV_777 1          1          1          1          58      48.7 Bacter~
## 3 ASV_606 1          1          1          1          55      45.8 Bacter~
## 4 ASV_740 1          1          1          1          54      47.2 Bacter~
## 5 ASV_14~ 1          1          1          1          54      44.5 Bacter~
## 6 ASV_23~ 1          1          1          1          54      47.4 Bacter~
## 7 ASV_15~ 1          1          1          1          52      45.3 Bacter~
## 8 ASV_24~ 1          1          1          1          52      43.0 Bacter~
## 9 ASV_19~ 1          1          1          1          52      43.0 Bacter~
## 10 ASV_568 1          1          1          1          51      45.1 Bacter~
```

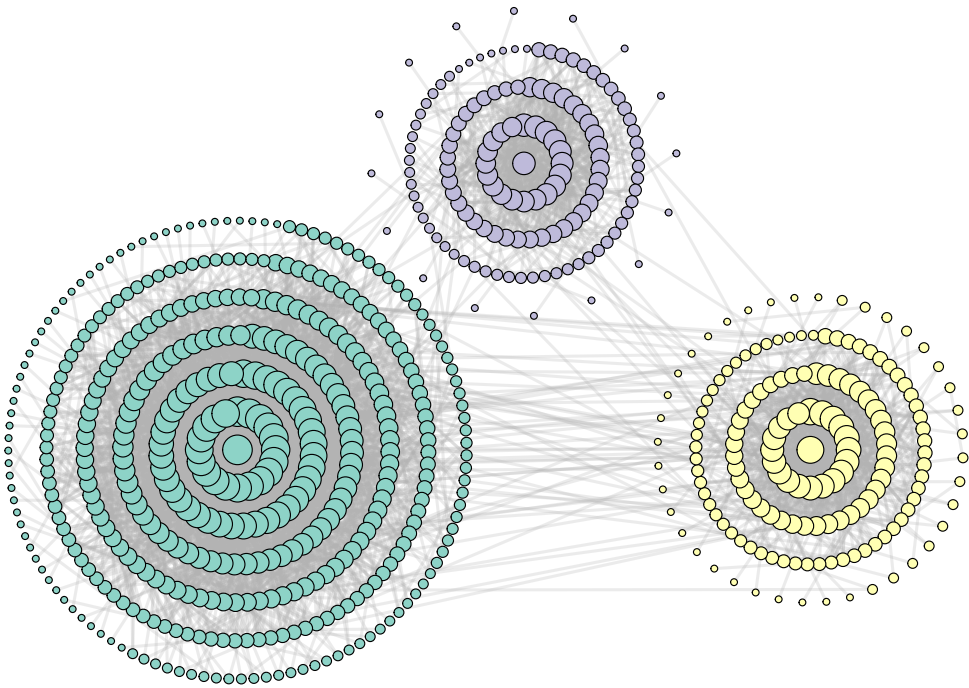
```
## # i 704 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 4,448 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>         <dbl> <chr>
## 1  594    597  0.895          0.895 Positive
## 2  588    597  0.864          0.864 Positive
## 3  589    597  0.800          0.800 Positive
## # i 4,445 more rows
```

6.2.2.1 tripartite_layout

```
p3 <- ggNetView(
  graph_obj = graph_sub$sub_graph_select,
  layout = "tripartite_gephi_layout",
  layout.module = "order",
  node_add = 20,
  r = 10,
  pointsize = c(1, 5),
  mapping_line = F,
  anchor_dist = 75,
  center = F,
  scale = F
)

p3
```

Node = 714
Edge = 4448

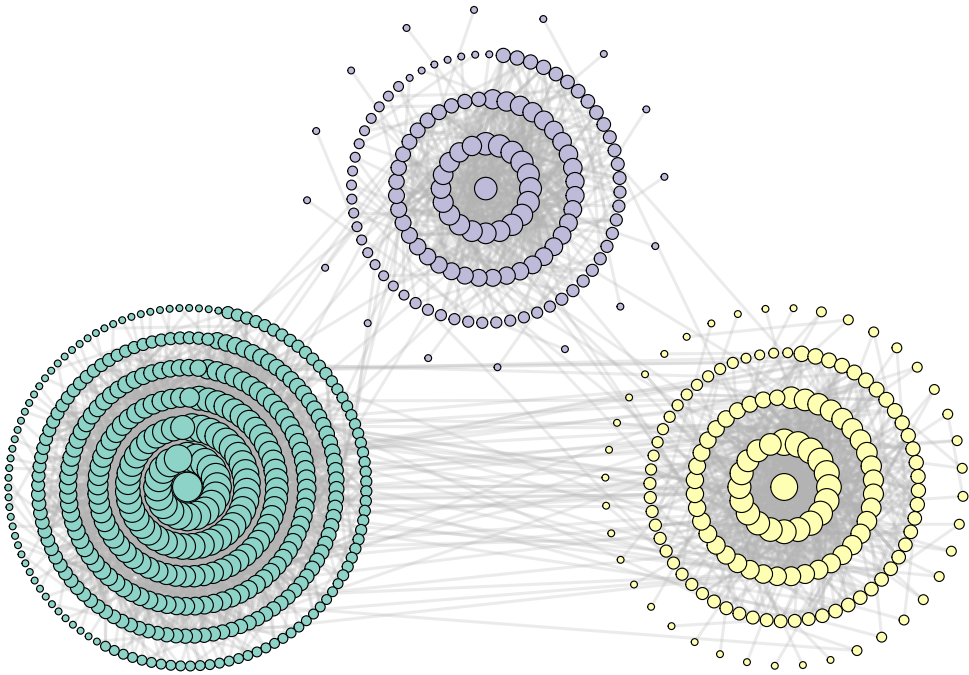


- Modularity
- Modularity1
 - Modularity7
 - Modularity6

6.2.2.2 tripartite_gephi_layout

```
p4 <- ggNetView(  
  graph_obj = graph_sub$sub_graph_select,  
  layout = "tripartite_equal_gephi_layout",  
  layout.module = "order",  
  node_add = 20,  
  r = 10,  
  pointsize = c(1, 5),  
  mapping_line = F,  
  anchor_dist = 100,  
  center = F,  
  scale = F  
)  
p4
```

Node = 714
Edge = 4448



Modularity

- Modularity1
- Modularity7
- Modularity6

6.2.3 quadripartite network

```
graph_sub <- get_subgraph(graph_obj = graph_obj,
                          select_module = c("1", "7", "6", "9")
                          )
```

```
##      Module Number
## 1         1      416
## 2         7      161
## 3         6      137
## 4         9      121
## 5         4      112
## 6         2      105
## 7         3      104
## 8        11      101
## 9         8       87
## 10        10       80
## 11         5       78
## 12        13       70
## 13        16       52
## 14        15       51
## 15        14       46
## 16 Others      328
```

```
graph_sub$sub_graph_select
```

```
## # A tbl_graph: 835 nodes and 4826 edges
## #
## # An undirected simple graph with 1 component
## #
## # Node Data: 835 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_916 1          1          1          1          58      50.5 Bacter~
## 2 ASV_777 1          1          1          1          58      48.7 Bacter~
## 3 ASV_606 1          1          1          1          55      45.8 Bacter~
## 4 ASV_740 1          1          1          1          54      47.2 Bacter~
## 5 ASV_14~ 1          1          1          1          54      44.5 Bacter~
## 6 ASV_23~ 1          1          1          1          54      47.4 Bacter~
## 7 ASV_15~ 1          1          1          1          52      45.3 Bacter~
## 8 ASV_24~ 1          1          1          1          52      43.0 Bacter~
## 9 ASV_19~ 1          1          1          1          52      43.0 Bacter~
## 10 ASV_568 1          1          1          1          51      45.1 Bacter~
```

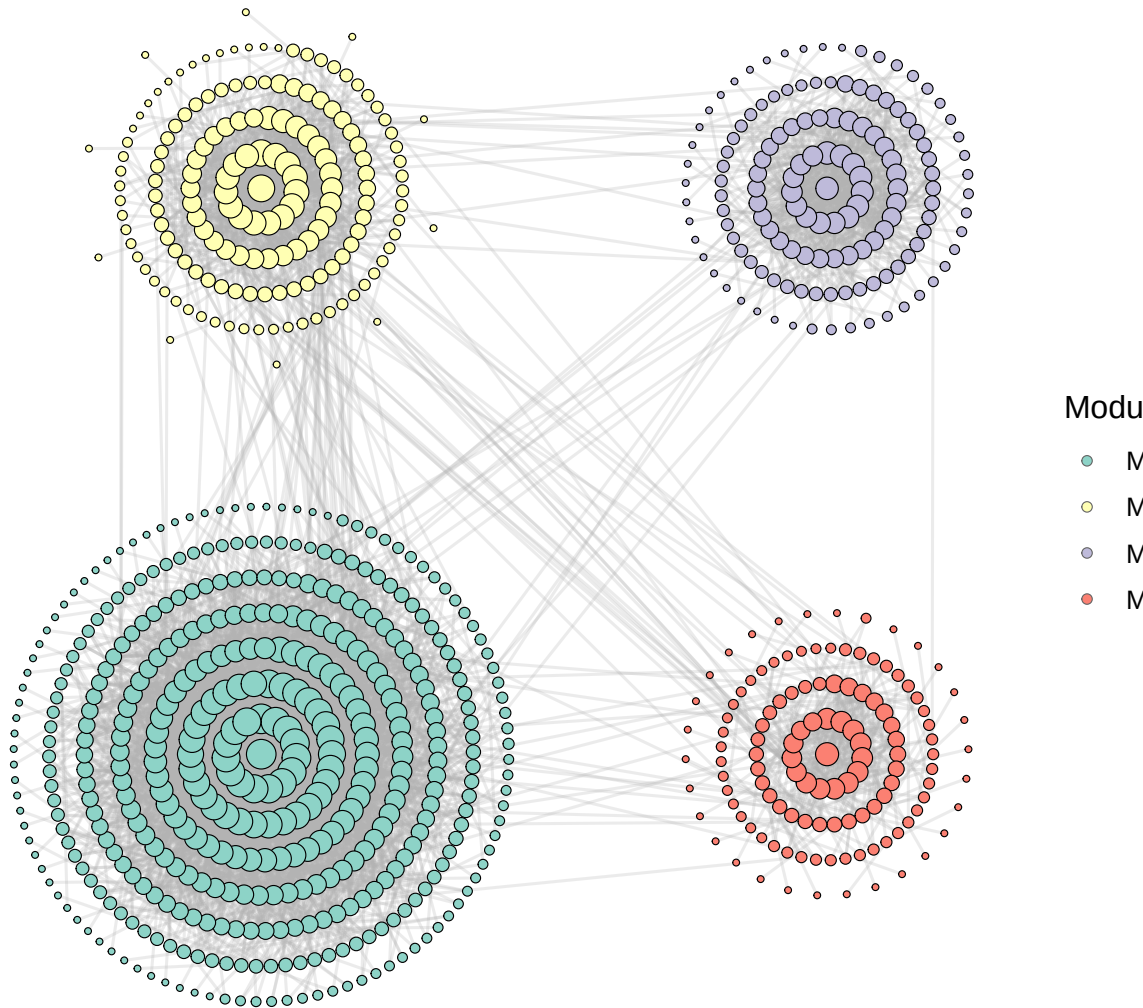
```
## # i 825 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 4,826 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>         <dbl> <chr>
## 1   594   597  0.895          0.895 Positive
## 2   588   597  0.864          0.864 Positive
## 3   589   597  0.800          0.800 Positive
## # i 4,823 more rows
```

6.2.3.1 quadripartite_layout

```
p5 <- ggNetView(
  graph_obj = graph_sub$sub_graph_select,
  layout = "quadripartite_gephi_layout",
  layout.module = "order",
  node_add = 15,
  anchor_dist = 8,
  r = 1,
  pointsize = c(1, 5),
  mapping_line = F,
  center = F,
  scale = F
)

p5
```

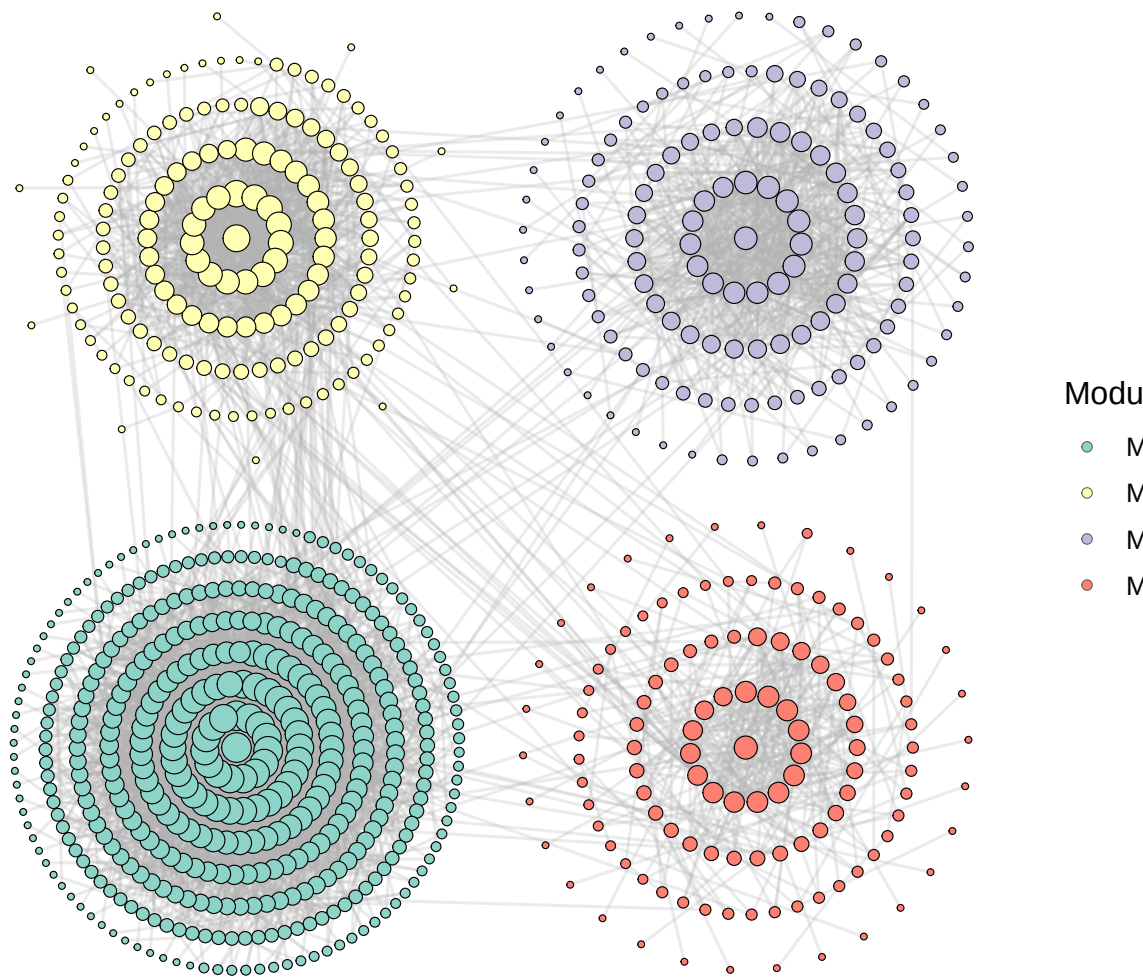
Node = 835
Edge = 4826



6.2.3.2 quadripartite_gephi_layout

```
p6 <- ggNetView(  
  graph_obj = graph_sub$sub_graph_select,  
  layout = "quadripartite_equal_gephi_layout",  
  layout.module = "order",  
  node_add = 15,  
  anchor_dist = 8,  
  r = 1,  
  pointsize = c(1, 5),  
  mapping_line = F,  
  center = F,  
  scale = F  
)  
p6
```

Node = 835
Edge = 4826



6.2.4 pentapartite network

```
graph_sub <- get_subgraph(graph_obj = graph_obj,
                          select_module = c("1", "7", "6", "9", "4")
                          )
```

```
##      Module Number
## 1         1      416
## 2         7      161
## 3         6      137
## 4         9      121
## 5         4      112
## 6         2      105
## 7         3      104
## 8        11      101
## 9         8       87
## 10        10       80
## 11         5       78
## 12        13       70
## 13        16       52
## 14        15       51
## 15        14       46
## 16 Others      328
```

```
graph_sub$sub_graph_select
```

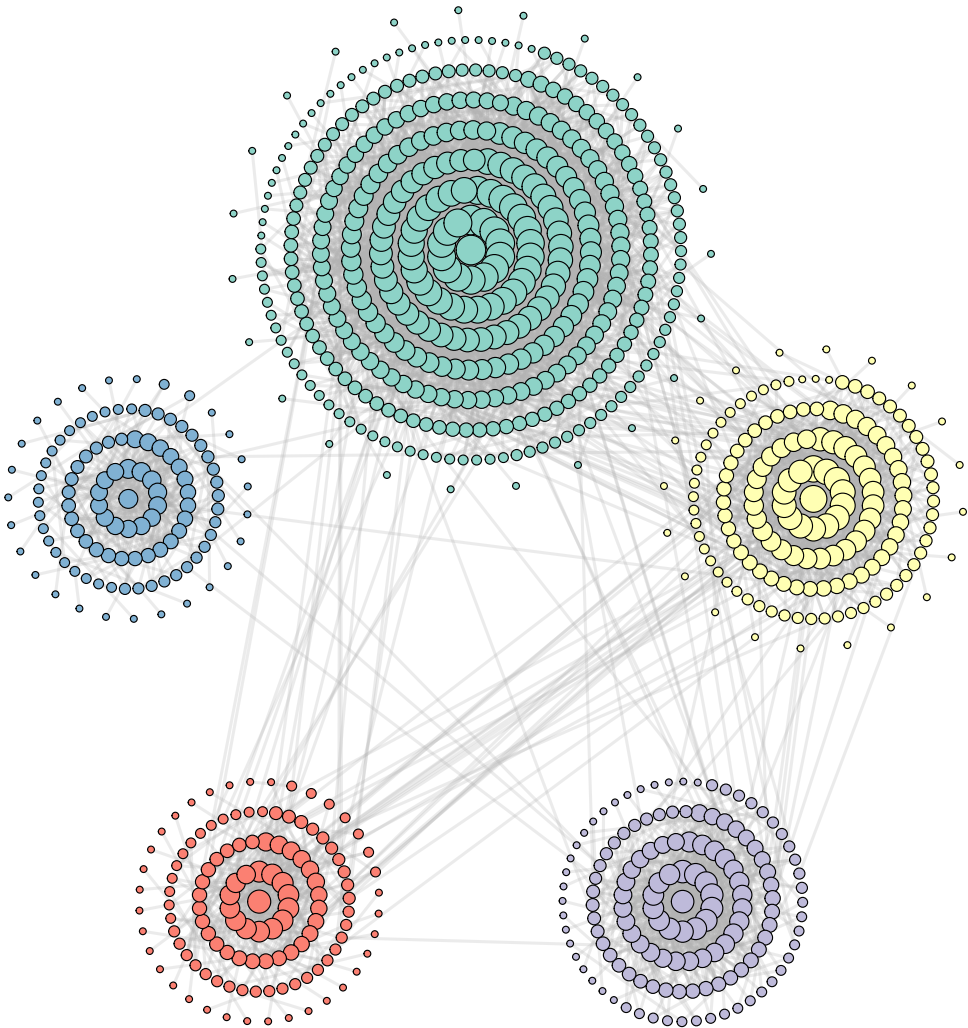
```
## # A tbl_graph: 947 nodes and 5108 edges
## #
## # An undirected simple graph with 1 component
## #
## # Node Data: 947 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_916 1          1          1          1          58      50.5 Bacter~
## 2 ASV_777 1          1          1          1          58      48.7 Bacter~
## 3 ASV_606 1          1          1          1          55      45.8 Bacter~
## 4 ASV_740 1          1          1          1          54      47.2 Bacter~
## 5 ASV_14~ 1          1          1          1          54      44.5 Bacter~
## 6 ASV_23~ 1          1          1          1          54      47.4 Bacter~
## 7 ASV_15~ 1          1          1          1          52      45.3 Bacter~
## 8 ASV_24~ 1          1          1          1          52      43.0 Bacter~
## 9 ASV_19~ 1          1          1          1          52      43.0 Bacter~
## 10 ASV_568 1          1          1          1          51      45.1 Bacter~
```

```
## # i 937 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 5,108 x 5
##   from      to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1    594    597  0.895        0.895 Positive
## 2    588    597  0.864        0.864 Positive
## 3    589    597  0.800        0.800 Positive
## # i 5,105 more rows
```

6.2.4.1 pentapartite_layout

```
p7 <- ggNetView(
  graph_obj = graph_sub$sub_graph_select,
  layout = "pentapartite_gephi_layout",
  layout.module = "order",
  node_add = 14,
  anchor_dist = 12,
  r = 1,
  pointsize = c(1, 5),
  mapping_line = F,
  center = F,
  scale = F
)
p7
```

Node = 947
Edge = 5108

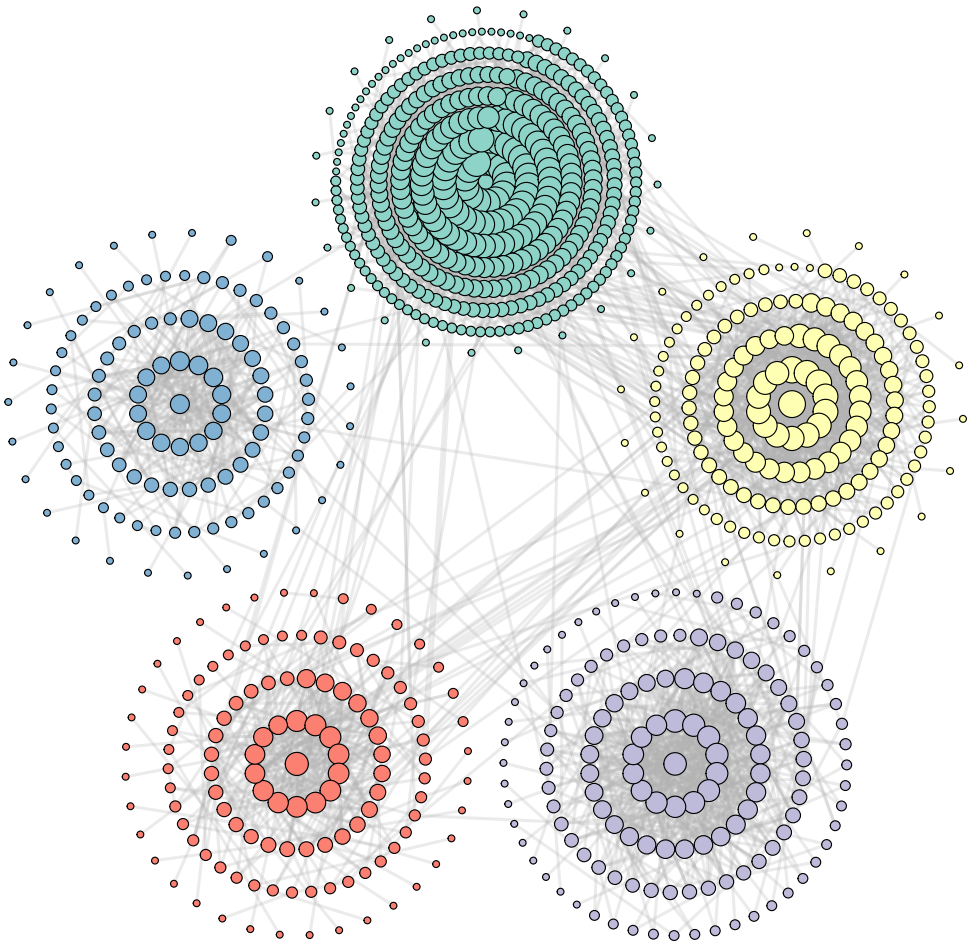


- Modularity
- Modularity1
 - Modularity7
 - Modularity6
 - Modularity9
 - Modularity4

6.2.4.2 pentapartite_gephi_layout

```
p8 <- ggNetView(  
  graph_obj = graph_sub$sub_graph_select,  
  layout = "pentapartite_equal_gephi_layout",  
  layout.module = "order",  
  node_add = 14,  
  anchor_dist = 15,  
  r = 1,  
  pointsize = c(1, 5),  
  mapping_line = F,  
  center = F,  
  scale = F  
)  
p8
```

Node = 947
Edge = 5108



- Modularity
- Modularity1
 - Modularity7
 - Modularity6
 - Modularity9
 - Modularity4

6.2.5 circular network

```
graph_sub <- get_subgraph(graph_obj = graph_obj,
                          select_module = c("1", "7", "6", "9", "4", "2")
                          )
```

```
##      Module Number
## 1         1      416
## 2         7      161
## 3         6      137
## 4         9      121
## 5         4      112
## 6         2      105
## 7         3      104
## 8        11      101
## 9         8       87
## 10        10       80
## 11         5       78
## 12        13       70
## 13        16       52
## 14        15       51
## 15        14       46
## 16 Others      328
```

```
graph_sub$sub_graph_select
```

```
## # A tbl_graph: 1052 nodes and 6353 edges
## #
## # An undirected simple graph with 1 component
## #
## # Node Data: 1,052 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_916 1          1          1          1          58      50.5 Bacter~
## 2 ASV_777 1          1          1          1          58      48.7 Bacter~
## 3 ASV_606 1          1          1          1          55      45.8 Bacter~
## 4 ASV_740 1          1          1          1          54      47.2 Bacter~
## 5 ASV_14~ 1          1          1          1          54      44.5 Bacter~
## 6 ASV_23~ 1          1          1          1          54      47.4 Bacter~
## 7 ASV_15~ 1          1          1          1          52      45.3 Bacter~
## 8 ASV_24~ 1          1          1          1          52      43.0 Bacter~
## 9 ASV_19~ 1          1          1          1          52      43.0 Bacter~
## 10 ASV_568 1          1          1          1          51      45.1 Bacter~
```

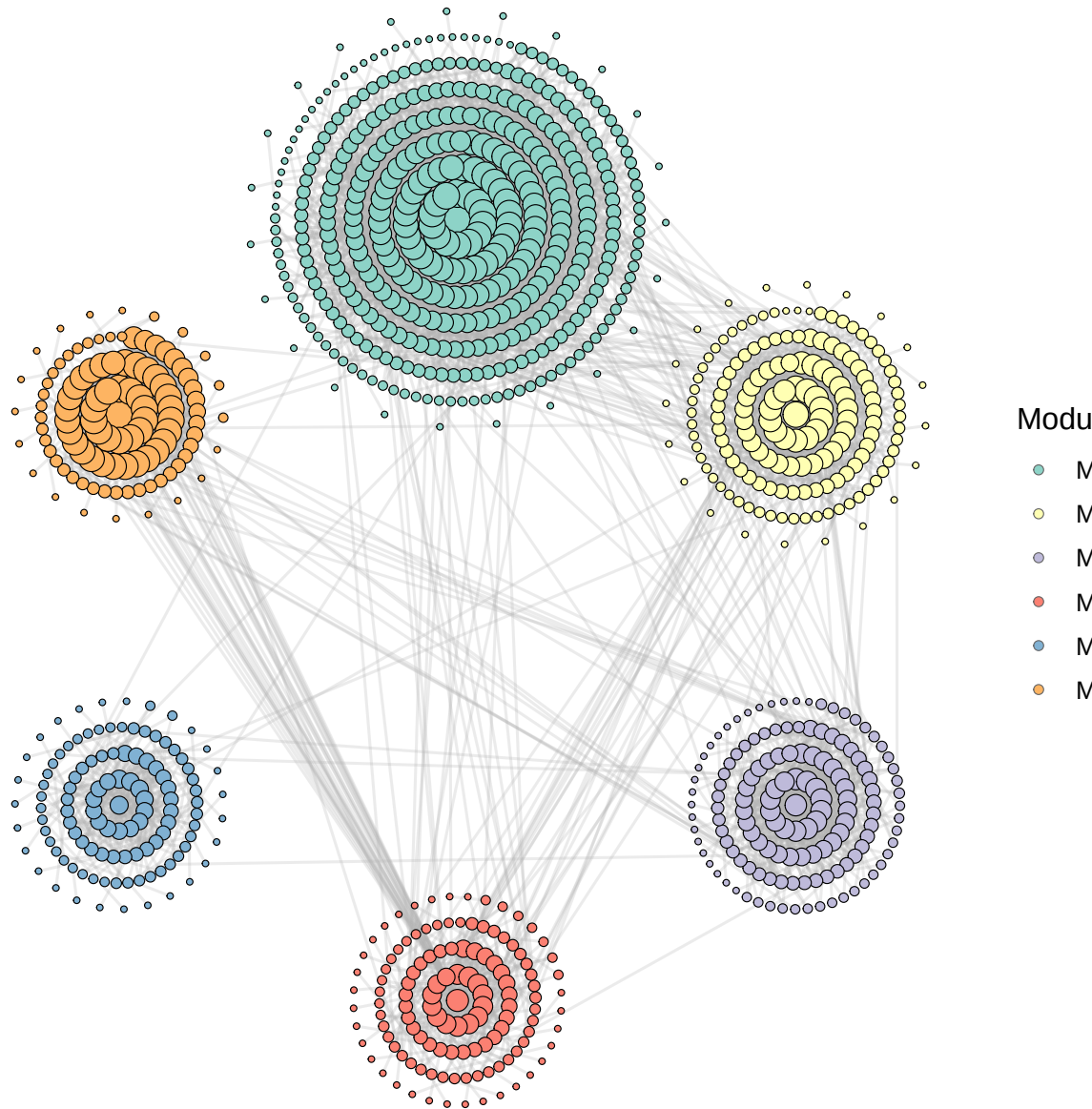
```
## # i 1,042 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 6,353 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>         <dbl> <chr>
## 1   594   597  0.895         0.895 Positive
## 2   588   597  0.864         0.864 Positive
## 3   589   597  0.800         0.800 Positive
## # i 6,350 more rows
```

6.2.5.1 circular_modules_gephi_layout (six group)

```
p9 <- ggNetView(
  graph_obj = graph_sub$sub_graph_select,
  layout = "circular_modules_gephi_layout",
  layout.module = "order",
  node_add = 14,
  anchor_dist = 15,
  r = 1,
  pointsize = c(1, 5),
  mapping_line = F,
  center = F,
  scale = F
)

p9
```

Node = 1052
Edge = 6353



6.2.5.2 circular_modules_equal_gephi_layout (six group)

```
p10 <- ggNetView(  
  graph_obj = graph_sub$sub_graph_select,  
  layout = "circular_modules_equal_gephi_layout",  
  layout.module = "order",  
  node_add = 14,  
  anchor_dist = 15,  
  r = 1,  
  pointsize = c(1, 5),  
  mapping_line = F,  
  center = F,  
  scale = F  
)  
p10
```

Node = 1052
Edge = 6353



```
graph_sub <- get_subgraph(graph_obj = graph_obj,
  select_module = c("1", "7", "6", "9", "4", "2", "11")
)
```

```
##      Module Number
## 1         1      416
## 2         7      161
## 3         6      137
## 4         9      121
## 5         4      112
## 6         2      105
## 7         3      104
## 8        11      101
## 9         8       87
## 10        10       80
## 11         5       78
## 12        13       70
## 13        16       52
## 14        15       51
## 15        14       46
## 16 Others     328
```

```
graph_sub$sub_graph_select
```

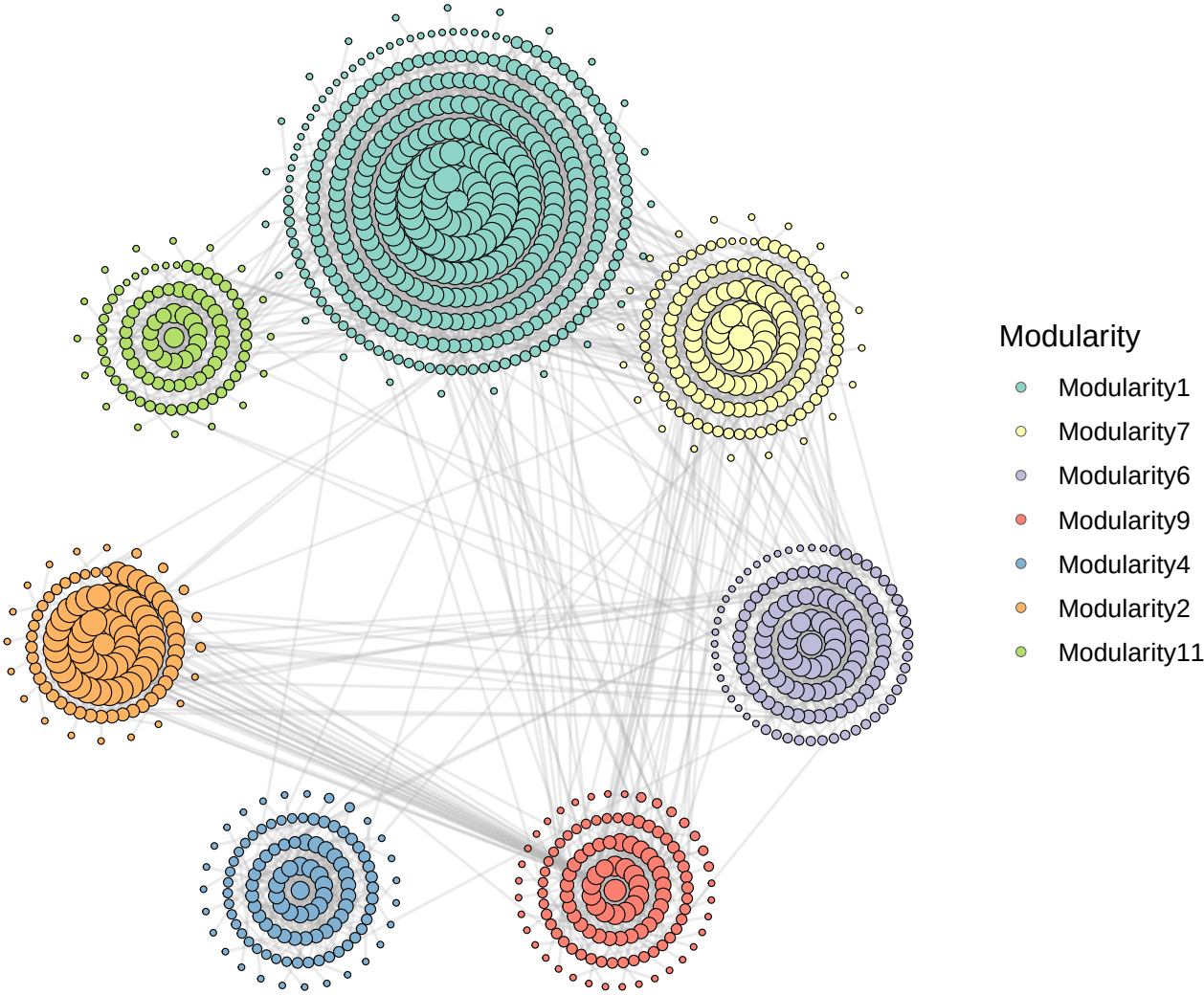
```
## # A tbl_graph: 1153 nodes and 6630 edges
## #
## # An undirected simple graph with 1 component
## #
## # Node Data: 1,153 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_916 1          1          1          1          58      50.5 Bacter~
## 2 ASV_777 1          1          1          1          58      48.7 Bacter~
## 3 ASV_606 1          1          1          1          55      45.8 Bacter~
## 4 ASV_740 1          1          1          1          54      47.2 Bacter~
## 5 ASV_14~ 1          1          1          1          54      44.5 Bacter~
## 6 ASV_23~ 1          1          1          1          54      47.4 Bacter~
## 7 ASV_15~ 1          1          1          1          52      45.3 Bacter~
## 8 ASV_24~ 1          1          1          1          52      43.0 Bacter~
## 9 ASV_19~ 1          1          1          1          52      43.0 Bacter~
## 10 ASV_568 1          1          1          1          51      45.1 Bacter~
## # i 1,143 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
```

```
## #  
## # Edge Data: 6,630 x 5  
##   from    to weight correlation corr_direction  
##   <int> <int> <dbl>      <dbl> <chr>  
## 1   594   597  0.895        0.895 Positive  
## 2   588   597  0.864        0.864 Positive  
## 3   589   597  0.800        0.800 Positive  
## # i 6,627 more rows
```

6.2.5.3 circular_modules_gephi_layout (seven group)

```
p10 <- ggNetView(  
  graph_obj = graph_sub$sub_graph_select,  
  layout = "circular_modules_gephi_layout",  
  layout.module = "order",  
  node_add = 14,  
  anchor_dist = 15,  
  r = 1,  
  pointsize = c(1, 5),  
  mapping_line = F,  
  center = F,  
  scale = F  
)  
  
p10
```

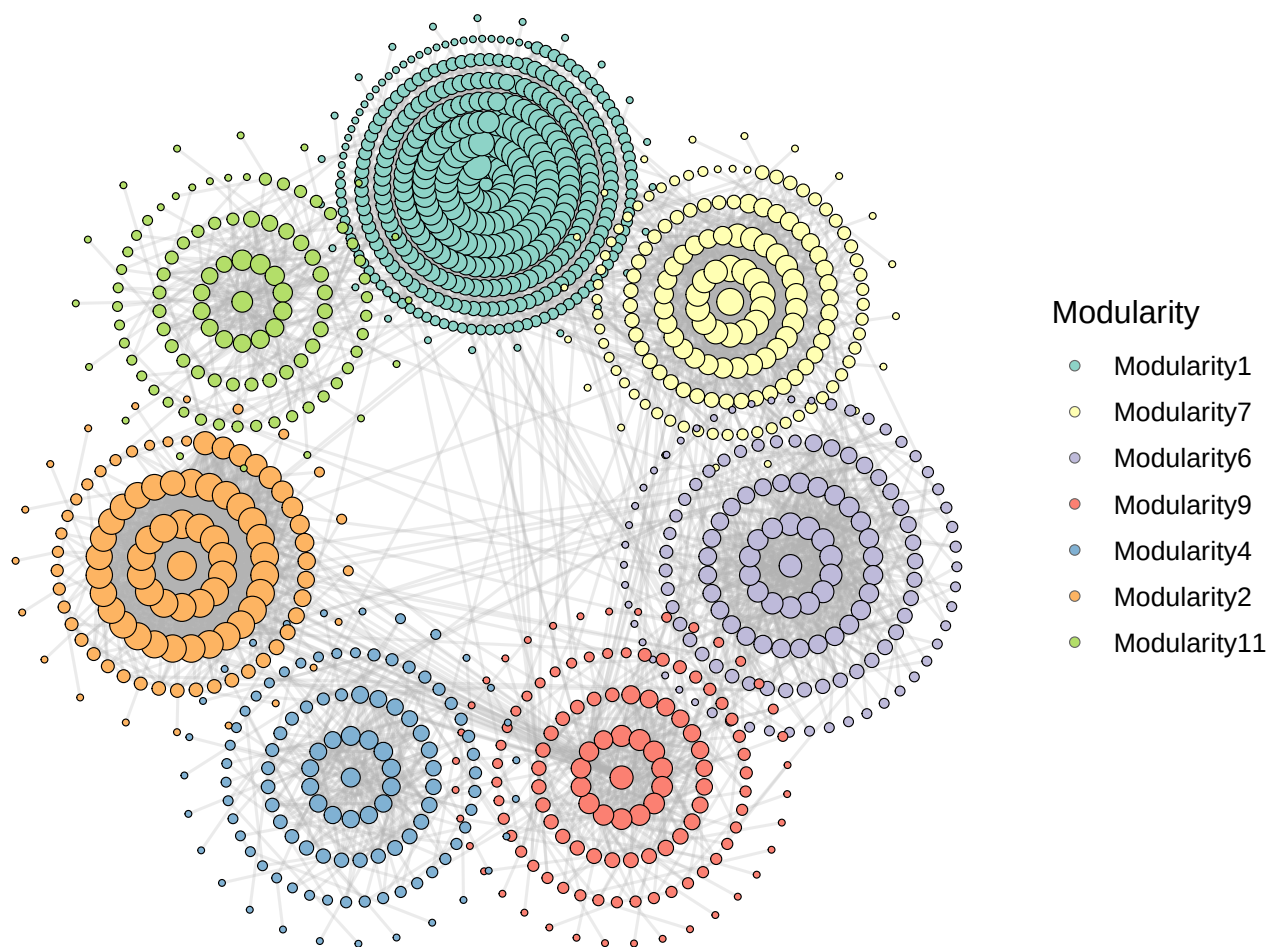
Node = 1153
Edge = 6630



6.2.5.4 circular_modules_equal_gephi_layout (seven group)

```
p11 <- ggNetView(  
  graph_obj = graph_sub$sub_graph_select,  
  layout = "circular_modules_equal_gephi_layout",  
  layout.module = "order",  
  node_add = 14,  
  anchor_dist = 15,  
  r = 1,  
  pointsize = c(1, 5),  
  mapping_line = F,  
  center = F,  
  scale = F  
)  
p11
```

Node = 1153
Edge = 6630



Chapter 7

Network topology information

Load R Package

```
library(tidyverse) # data manipulation / piping utilities
library(ggNetView) # graph builders + info-extraction helpers
```

Example data

```
# Access built-in example datasets in ggNetView

# ---- Dataset 1: Relative abundance table of rarefied ASVs/OTUs ----
# Rows = ASVs/OTUs (features / variables), Columns = samples
data("otu_rare_relative")
dim(otu_rare_relative) # Check matrix dimensions: n_features x n_samples

## [1] 2859    18

otu_rare_relative[1:5, 1:5] # Preview first 5 rows x 5 cols to confirm data layout

##           K01      K02      K03      K04      K05
## ASV_1 0.03306667 0.05453333 0.02013333 0.03613333 0.02686667
## ASV_2 0.05750000 0.03393333 0.06046667 0.05810000 0.07320000
## ASV_3 0.01733333 0.01296667 0.02290000 0.02336667 0.03106667
## ASV_4 0.04266667 0.01093333 0.01416667 0.01933333 0.03346667
## ASV_6 0.02646667 0.01856667 0.02110000 0.02353333 0.03806667
```

```
# ---- Dataset 2: Taxonomic annotation table for ASVs/OTUs ----
# Rows = ASVs/OTUs (row names should match the abundance table above)
# Columns = taxonomic ranks (Kingdom ~ Species, etc.)
data("tax_tab")
dim(tax_tab)
```

```
## [1] 2859      8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
##   OTUID Kingdom Phylum      Class      Order
##   <chr>   <chr>   <chr>      <chr>      <chr>
## 1 ASV_2  Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia Spartobacteria Unassigned
## 3 ASV_31 Bacteria Actinobacteria Actinobacteria Actinomycetales
## 4 ASV_27 Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned      Unassigned  Unassigned
```

Build graph object

```
# ---- Build a co-occurrence network from an abundance matrix ----
# Internal pipeline: transform -> correlation -> p-value adjustment ->
# edge filtering -> community detection -> attach taxonomy.
graph_obj <- build_graph_from_mat(
  mat          = otu_rare_relative, # variables x samples numeric matrix
  transform.method = "none",        # input already relative abundance, no extra
  method        = "WGCNA",         # correlation backend: WGCNA::corAndPvalue
  proc          = "bonferroni",     # multiple-testing correction
  r.threshold   = 0.7,              # |r| cutoff for keeping an edge
  p.threshold   = 0.05,            # adjusted p-value cutoff
  node_annotation = tax_tab,        # taxonomy joined onto each node by name
  top_modules   = 15,              # keep top-15 modules; rest -> "Others"
  seed          = 1115             # fix RNG for reproducibility
)

graph_obj      # tbl_graph with Modularity / Degree / Strength + taxonomy columns
```

```
## # A tbl_graph: 213 nodes and 844 edges
## #
## # An undirected simple graph with 29 components
## #
## # Node Data: 213 x 14 (active)
```

```
##      name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##      <chr>      <fct>          <ord>          <chr>          <ord>          <dbl>      <dbl> <chr>
##  1 ASV_649 5              5              5              5              27      26.5 Bacter~
##  2 ASV_705 5              5              5              5              27      26.5 Bacter~
##  3 ASV_12~ 5              5              5              5              27      26.5 Bacter~
##  4 ASV_13~ 5              5              5              5              27      26.5 Bacter~
##  5 ASV_14~ 5              5              5              5              27      26.5 Bacter~
##  6 ASV_14~ 5              5              5              5              27      26.5 Bacter~
##  7 ASV_24~ 5              5              5              5              27      26.5 Bacter~
##  8 ASV_25~ 5              5              5              5              27      26.4 Bacter~
##  9 ASV_28~ 5              5              5              5              27      26.5 Bacter~
## 10 ASV_28~ 5              5              5              5              27      26.5 Bacter~
## # i 203 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 844 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>          <dbl> <chr>
##  1    194   195  0.959          0.959 Positive
##  2    185   208  0.954          0.954 Positive
##  3    185   213  0.957          0.957 Positive
## # i 841 more rows
```

7.1 Get network topology information

when `mat = NULL`

```
# ---- Compute network topology + robustness for a single graph ----
# When `mat = NULL`, the function still computes all *purely topological*
# metrics from the igraph object, but the following fields are returned as NA
# because they require the original abundance matrix:
#
# * Cohension_Positive - needs per-ASV mean abundance to weight positive r
# * Cohension_Negative - needs per-ASV mean abundance to weight negative r
# * Robustness_weight - abundance-weighted node-removal bootstrap (needs sp.ra)
# * Robustness_unweight - tied to the same `mat` branch in the code, so also NA
# * Stability - mean of the two Robustness_* fields, so also NA
# * out$Robustness - the whole 5%-100% removal curve is NA (mean/sd/se)
#
# All other fields (Node, Edge, Degree, Distance, Diameter, Density,
# Transitivity_global/local, Betweenness, Betweenness_edge, Closeness,
# Eigen_centrality, Modularity, K_core_mean/max/min, Network_efficiency,
# Network_info.centrality, Vulnerability) are computed normally from `graph_obj`.
```

```

topology_out <- get_network_topology(
  graph_obj      = graph_obj,      # input tbl_graph (required)
  mat            = NULL,           # raw abundance matrix; NULL -> skip cohesion + robustness
  transfrom.method = "none",       # pre-correlation transform (ignored when mat = NULL)
  method         = "WGCNA",       # correlation backend used to rebuild network.raw from graph
  proc           = "bonferroni",   # multiple-testing correction for p-values (used with method)
  r.threshold     = 0.7,           # |r| cutoff applied when rebuilding network.raw from graph
  p.threshold     = 0.05,         # adjusted p-value cutoff applied when rebuilding network.raw from graph
  bootstrap       = 100,           # # of random ER networks for `Random_network` column
                                   # also # of bootstrap iterations for robustness (only if bootstrap > 0)
)

class(topology_out)

```

```
## [1] "list"
```

```
names(topology_out)
```

```
## [1] "topology" "Robustness"
```

```
topology_out
```

```

## $topology
## # A tibble: 24 x 3
##   Topology      Target_network Random_network
##   <chr>          <dbl>          <dbl>
## 1 Node              213              213
## 2 Edge              844              844
## 3 Degree            7.92              7.92
## 4 Distance          1.45              2.81
## 5 Diameter          3.83              5.04
## 6 Density           0.0374             0.0374
## 7 Transitivity_global 0.851             0.0374
## 8 Transitivity_local  0.783             0.0373
## 9 Betweenness       3.47             191.
## 10 Betweenness_edge  2.58             75.0
## # i 14 more rows
##
## $Robustness
##   Proportion.removed remain.mean remain.sd remain.se weighted
## 1             0.05          NA          NA          NA weighted
## 2             0.10          NA          NA          NA weighted
## 3             0.15          NA          NA          NA weighted

```

## 4	0.20	NA	NA	NA	weighted
## 5	0.25	NA	NA	NA	weighted
## 6	0.30	NA	NA	NA	weighted
## 7	0.35	NA	NA	NA	weighted
## 8	0.40	NA	NA	NA	weighted
## 9	0.45	NA	NA	NA	weighted
## 10	0.50	NA	NA	NA	weighted
## 11	0.55	NA	NA	NA	weighted
## 12	0.60	NA	NA	NA	weighted
## 13	0.65	NA	NA	NA	weighted
## 14	0.70	NA	NA	NA	weighted
## 15	0.75	NA	NA	NA	weighted
## 16	0.80	NA	NA	NA	weighted
## 17	0.85	NA	NA	NA	weighted
## 18	0.90	NA	NA	NA	weighted
## 19	0.95	NA	NA	NA	weighted
## 20	1.00	NA	NA	NA	weighted
## 21	0.05	NA	NA	NA	unweighted
## 22	0.10	NA	NA	NA	unweighted
## 23	0.15	NA	NA	NA	unweighted
## 24	0.20	NA	NA	NA	unweighted
## 25	0.25	NA	NA	NA	unweighted
## 26	0.30	NA	NA	NA	unweighted
## 27	0.35	NA	NA	NA	unweighted
## 28	0.40	NA	NA	NA	unweighted
## 29	0.45	NA	NA	NA	unweighted
## 30	0.50	NA	NA	NA	unweighted
## 31	0.55	NA	NA	NA	unweighted
## 32	0.60	NA	NA	NA	unweighted
## 33	0.65	NA	NA	NA	unweighted
## 34	0.70	NA	NA	NA	unweighted
## 35	0.75	NA	NA	NA	unweighted
## 36	0.80	NA	NA	NA	unweighted
## 37	0.85	NA	NA	NA	unweighted
## 38	0.90	NA	NA	NA	unweighted
## 39	0.95	NA	NA	NA	unweighted
## 40	1.00	NA	NA	NA	unweighted

7.2 Get network topology information with matrix

When a matrix is passed to `mat`, all topology properties are computed.

```

# ---- Compute the FULL set of network topology + robustness metrics ----
# When `mat` is provided, the function recomputes the thresholded correlation
# matrix from `mat` (used as `network.raw`) and ALL topology fields are
# returned with real values - including the ones that are NA when mat = NULL:
#   Cohension_Positive / Cohension_Negative
#   Robustness_weight / Robustness_unweight
#   Stability
#   out$Robustness      (the 5%-100% node-removal curve)
topology_with_mat <- get_network_topology(
  graph_obj      = graph_obj,          # input tbl_graph (required)
  mat            = otu_rare_relative,  # raw abundance matrix used to derive sp.ra and
  transform.method = "none",           # pre-correlation transform on `mat`; "none" =
  method         = "WGCNA",           # correlation backend: WGCNA::corAndPvalue
  proc           = "bonferroni",       # multiple-testing correction
  r.threshold    = 0.7,               # |r| cutoff for keeping an edge in network.raw
  p.threshold    = 0.05,              # adjusted p-value cutoff for keeping an edge
  bootstrap      = 100                # # of bootstrap iterations for robustness AND
)

topology_with_mat

```

```

## $Topology
## # A tibble: 24 x 3
##   Topology      Target_network Random_network
##   <chr>          <dbl>          <dbl>
## 1 Node           213            213
## 2 Edge           844            844
## 3 Degree         7.92            7.92
## 4 Distance       1.45            2.81
## 5 Diameter       3.83            5.07
## 6 Density        0.0374          0.0374
## 7 Transitivity_global 0.851          0.0369
## 8 Transitivity_local  0.783          0.0369
## 9 Betweenness    3.47           191.
## 10 Betweenness_edge 2.58           75.0
## # i 14 more rows
##
## $Robustness
##   Proportion.removed remain.mean  remain.sd  remain.se  weighted
## 1      0.05  0.93605634  0.007211109  0.0007211109  weighted
## 2      0.10  0.88028169  0.009795834  0.0009795834  weighted
## 3      0.15  0.82098592  0.010068306  0.0010068306  weighted
## 4      0.20  0.75812207  0.014099990  0.0014099990  weighted
## 5      0.25  0.70136150  0.014709199  0.0014709199  weighted

```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 159

## 6	0.30	0.64497653	0.014509330	0.0014509330	weighted
## 7	0.35	0.58685446	0.016628903	0.0016628903	weighted
## 8	0.40	0.53352113	0.016501489	0.0016501489	weighted
## 9	0.45	0.47596244	0.015021992	0.0015021992	weighted
## 10	0.50	0.42164319	0.016598013	0.0016598013	weighted
## 11	0.55	0.36976526	0.017402457	0.0017402457	weighted
## 12	0.60	0.31342723	0.017882009	0.0017882009	weighted
## 13	0.65	0.26868545	0.016971541	0.0016971541	weighted
## 14	0.70	0.21671362	0.018246825	0.0018246825	weighted
## 15	0.75	0.16539906	0.017010851	0.0017010851	weighted
## 16	0.80	0.12633803	0.013887030	0.0013887030	weighted
## 17	0.85	0.08164319	0.015915997	0.0015915997	weighted
## 18	0.90	0.04460094	0.012313354	0.0012313354	weighted
## 19	0.95	0.01610329	0.009721802	0.0009721802	weighted
## 20	1.00	0.00000000	0.000000000	0.0000000000	weighted
## 21	0.05	0.93671362	0.007709494	0.0007709494	unweighted
## 22	0.10	0.88164319	0.009396664	0.0009396664	unweighted
## 23	0.15	0.81835681	0.012795630	0.0012795630	unweighted
## 24	0.20	0.76169014	0.013146217	0.0013146217	unweighted
## 25	0.25	0.70309859	0.013163142	0.0013163142	unweighted
## 26	0.30	0.64441315	0.013669288	0.0013669288	unweighted
## 27	0.35	0.58643192	0.015622327	0.0015622327	unweighted
## 28	0.40	0.53389671	0.013950773	0.0013950773	unweighted
## 29	0.45	0.47577465	0.015780078	0.0015780078	unweighted
## 30	0.50	0.42441315	0.017169071	0.0017169071	unweighted
## 31	0.55	0.36943662	0.017358583	0.0017358583	unweighted
## 32	0.60	0.31089202	0.015939550	0.0015939550	unweighted
## 33	0.65	0.26976526	0.017057446	0.0017057446	unweighted
## 34	0.70	0.21596244	0.019154967	0.0019154967	unweighted
## 35	0.75	0.16741784	0.017708100	0.0017708100	unweighted
## 36	0.80	0.12718310	0.016630175	0.0016630175	unweighted
## 37	0.85	0.08309859	0.014241705	0.0014241705	unweighted
## 38	0.90	0.04394366	0.013169906	0.0013169906	unweighted
## 39	0.95	0.01507042	0.010170393	0.0010170393	unweighted
## 40	1.00	0.00000000	0.000000000	0.0000000000	unweighted

7.3 Get network topology information from sample

```
# ---- Per-sample subgraph topology -----
# Step 1: split `graph_obj` into per-sample subgraphs by the abundance matrix
#         (an OTU is "present" in a sample when its abundance > min_abundance).
# Step 2: run the full `get_network_topology()` pipeline on EACH sample subgraph.
```

```

# When `mat` is supplied, every topology field (including cohesion, robustness,
# stability) is computed for each sample.
t3_group <- system.time(
  topology_from_sample <- get_sample_subgraph_topology(
    graph_obj      = graph_obj,          # input tbl_graph (the full network)
    mat            = otu_rare_relative,  # abundance matrix used for both presence-ca
    transform.method = "none",           # pre-correlation transform on `mat`; "none"
    method         = "WGCNA",           # correlation backend: WGCNA::corAndPvalue
    cor.method     = "pearson",          # Pearson correlation
    proc           = "bonferroni",       # multiple-testing correction
    bootstrap      = 100                 # # of bootstrap iterations for robustness;
  )
)

topology_from_sample # one topology table per sample, plus per-sample robustness cur

## $subgraph_list
## $subgraph_list$K01
## # A tbl_graph: 113 nodes and 161 edges
## #
## # An undirected simple graph with 35 components
## #
## # Node Data: 113 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_705  5          5          5          5          27      26.5 Bacter~
## 2 ASV_12~  5          5          5          5          27      26.5 Bacter~
## 3 ASV_913  5          5          5          5          26      25.2 Bacter~
## 4 ASV_767  5          5          5          5          22      21.1 Bacter~
## 5 ASV_11~  5          5          5          5          20      19.1 Bacter~
## 6 ASV_28~  5          5          5          5          20      19.1 Bacter~
## 7 ASV_322  5          5          5          5          18      17.2 Bacter~
## 8 ASV_15~  5          5          5          5          18      17.1 Bacter~
## 9 ASV_18~  5          5          5          5          17      16.3 Bacter~
## 10 ASV_25~ 5          5          5          5          12      11.3 Bacter~
## # i 103 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 161 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1    100   101 0.959      0.959 Positive
## 2     90    92 0.969      0.969 Positive
## 3     87    89 0.943      0.943 Positive

```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 161

```
## # i 158 more rows
##
## $subgraph_list$K02
## # A tbl_graph: 122 nodes and 164 edges
## #
## # An undirected simple graph with 37 components
## #
## # Node Data: 122 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 2 ASV_913 5          5          5          5          26      25.2 Bacter~
## 3 ASV_767 5          5          5          5          22      21.1 Bacter~
## 4 ASV_996 5          5          5          5          19      18.3 Bacter~
## 5 ASV_322 5          5          5          5          18      17.2 Bacter~
## 6 ASV_23~ 5          5          5          5          18      17.3 Bacter~
## 7 ASV_18~ 5          5          5          5          17      16.3 Bacter~
## 8 ASV_244 5          5          5          5          3       2.87 Bacter~
## 9 ASV_367 5          5          5          5          2       1.89 Bacter~
## 10 ASV_277 5          5          5          5          1       0.950 Bacter~
## # i 112 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 164 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1  108  109 0.959      0.959 Positive
## 2  104  122 0.957      0.957 Positive
## 3   91   93 0.969      0.969 Positive
## # i 161 more rows
##
## $subgraph_list$K03
## # A tbl_graph: 102 nodes and 379 edges
## #
## # An undirected simple graph with 36 components
## #
## # Node Data: 102 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_649 5          5          5          5          27      26.5 Bacter~
## 2 ASV_705 5          5          5          5          27      26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 4 ASV_13~ 5          5          5          5          27      26.5 Bacter~
## 5 ASV_14~ 5          5          5          5          27      26.5 Bacter~
## 6 ASV_14~ 5          5          5          5          27      26.5 Bacter~
```

```

## 7 ASV_24~ 5          5          5          5          27      26.5 Bacter~
## 8 ASV_25~ 5          5          5          5          27      26.4 Bacter~
## 9 ASV_28~ 5          5          5          5          27      26.5 Bacter~
## 10 ASV_28~ 5         5          5          5          27      26.5 Bacter~
## # i 92 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 379 x 5
##   from      to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1      1      2 0.996      0.996 Positive
## 2      1      3 0.997      0.997 Positive
## 3      2      3 0.994      0.994 Positive
## # i 376 more rows
##
## $subgraph_list$K04
## # A tbl_graph: 94 nodes and 107 edges
## #
## # An undirected simple graph with 36 components
## #
## # Node Data: 94 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_913 5          5          5          5          26      25.2 Bacter~
## 2 ASV_767 5          5          5          5          22      21.1 Bacter~
## 3 ASV_996 5          5          5          5          19      18.3 Bacter~
## 4 ASV_322 5          5          5          5          18      17.2 Bacter~
## 5 ASV_18~ 5          5          5          5          17      16.3 Bacter~
## 6 ASV_244 5          5          5          5          3       2.87 Bacter~
## 7 ASV_367 5          5          5          5          2       1.89 Bacter~
## 8 ASV_277 5          5          5          5          1       0.950 Bacter~
## 9 ASV_671 2          2          2          2          18      17.5 Bacter~
## 10 ASV_568 2         2          2          2          16      15.6 Bacter~
## # i 84 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 107 x 5
##   from      to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1      1      2 0.967      0.967 Positive
## 2      1      3 0.947      0.947 Positive
## 3      1      4 0.954      0.954 Positive
## # i 104 more rows
##

```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 163

```
## $subgraph_list$K05
## # A tbl_graph: 71 nodes and 63 edges
## #
## # An undirected simple graph with 32 components
## #
## # Node Data: 71 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_913 5          5          5          5          26      25.2  Bacter~
## 2 ASV_819 5          5          5          5          26      25.5  Bacter~
## 3 ASV_767 5          5          5          5          22      21.1  Bacter~
## 4 ASV_996 5          5          5          5          19      18.3  Bacter~
## 5 ASV_322 5          5          5          5          18      17.2  Bacter~
## 6 ASV_23~ 5          5          5          5          18      17.3  Bacter~
## 7 ASV_244 5          5          5          5          3       2.87  Bacter~
## 8 ASV_367 5          5          5          5          2       1.89  Bacter~
## 9 ASV_277 5          5          5          5          1       0.950 Bacter~
## 10 ASV_24~ 2         2          2          2          15      14.5  Bacter~
## # i 61 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 63 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1     1     2  0.974      0.974 Positive
## 2     1     3  0.967      0.967 Positive
## 3     2     3  0.969      0.969 Positive
## # i 60 more rows
##
## $subgraph_list$K06
## # A tbl_graph: 97 nodes and 119 edges
## #
## # An undirected simple graph with 35 components
## #
## # Node Data: 97 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_25~ 5          5          5          5          25      24.4  Bacter~
## 2 ASV_28~ 5          5          5          5          23      22.5  Bacter~
## 3 ASV_767 5          5          5          5          22      21.1  Bacter~
## 4 ASV_26~ 5          5          5          5          21      20.4  Bacter~
## 5 ASV_28~ 5          5          5          5          20      19.1  Bacter~
## 6 ASV_996 5          5          5          5          19      18.3  Bacter~
## 7 ASV_322 5          5          5          5          18      17.2  Bacter~
## 8 ASV_15~ 5          5          5          5          18      17.1  Bacter~
```

```

## 9 ASV_244 5          5          5          5          3      2.87 Bacter~
## 10 ASV_367 5          5          5          5          2      1.89 Bacter~
## # i 87 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 119 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1      1      2 0.989      0.989 Positive
## 2      1      3 0.949      0.949 Positive
## 3      2      3 0.943      0.943 Positive
## # i 116 more rows
##
## $subgraph_list$OE1
## # A tbl_graph: 138 nodes and 308 edges
## #
## # An undirected simple graph with 34 components
## #
## # Node Data: 138 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_705 5          5          5          5          27      26.5 Bacter~
## 2 ASV_913 5          5          5          5          26      25.2 Bacter~
## 3 ASV_819 5          5          5          5          26      25.5 Bacter~
## 4 ASV_767 5          5          5          5          22      21.1 Bacter~
## 5 ASV_11~ 5          5          5          5          20      19.1 Bacter~
## 6 ASV_28~ 5          5          5          5          20      19.1 Bacter~
## 7 ASV_996 5          5          5          5          19      18.3 Bacter~
## 8 ASV_322 5          5          5          5          18      17.2 Bacter~
## 9 ASV_15~ 5          5          5          5          18      17.1 Bacter~
## 10 ASV_23~ 5          5          5          5          18      17.3 Bacter~
## # i 128 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 308 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1    125    126 0.959      0.959 Positive
## 2    113    115 0.969      0.969 Positive
## 3    111    112 0.943      0.943 Positive
## # i 305 more rows
##
## $subgraph_list$OE2
## # A tbl_graph: 135 nodes and 230 edges

```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 165

```
## #
## # An undirected simple graph with 41 components
## #
## # Node Data: 135 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_649 5          5          5          5          27      26.5 Bacter~
## 2 ASV_705 5          5          5          5          27      26.5 Bacter~
## 3 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 4 ASV_913 5          5          5          5          26      25.2 Bacter~
## 5 ASV_819 5          5          5          5          26      25.5 Bacter~
## 6 ASV_17~ 5          5          5          5          24      23.5 Bacter~
## 7 ASV_767 5          5          5          5          22      21.1 Bacter~
## 8 ASV_26~ 5          5          5          5          21      20.4 Bacter~
## 9 ASV_11~ 5          5          5          5          20      19.1 Bacter~
## 10 ASV_28~ 5          5          5          5          20      19.1 Bacter~
## # i 125 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 230 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1  120   121 0.959      0.959 Positive
## 2  114   135 0.957      0.957 Positive
## 3  105   107 0.969      0.969 Positive
## # i 227 more rows
##
## $subgraph_list$OE3
## # A tbl_graph: 119 nodes and 160 edges
## #
## # An undirected simple graph with 33 components
## #
## # Node Data: 119 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_705 5          5          5          5          27      26.5 Bacter~
## 2 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 3 ASV_913 5          5          5          5          26      25.2 Bacter~
## 4 ASV_767 5          5          5          5          22      21.1 Bacter~
## 5 ASV_996 5          5          5          5          19      18.3 Bacter~
## 6 ASV_322 5          5          5          5          18      17.2 Bacter~
## 7 ASV_15~ 5          5          5          5          18      17.1 Bacter~
## 8 ASV_23~ 5          5          5          5          18      17.3 Bacter~
## 9 ASV_18~ 5          5          5          5          17      16.3 Bacter~
## 10 ASV_25~ 5          5          5          5          12      11.3 Bacter~
```

```

## # i 109 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 160 x 5
##   from      to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1  106    107  0.959        0.959 Positive
## 2  101    119  0.954        0.954 Positive
## 3   91     93  0.969        0.969 Positive
## # i 157 more rows
##
## $subgraph_list$OE4
## # A tbl_graph: 118 nodes and 171 edges
## #
## # An undirected simple graph with 34 components
## #
## # Node Data: 118 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_705  5          5          5          5          27      26.5 Bacter~
## 2 ASV_819  5          5          5          5          26      25.5 Bacter~
## 3 ASV_28~  5          5          5          5          25      24.5 Bacter~
## 4 ASV_24~  5          5          5          5          23      22.6 Bacter~
## 5 ASV_767  5          5          5          5          22      21.1 Bacter~
## 6 ASV_996  5          5          5          5          19      18.3 Bacter~
## 7 ASV_322  5          5          5          5          18      17.2 Bacter~
## 8 ASV_23~  5          5          5          5          18      17.3 Bacter~
## 9 ASV_18~  5          5          5          5          17      16.3 Bacter~
## 10 ASV_244 5          5          5          5          3       2.87 Bacter~
## # i 108 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 171 x 5
##   from      to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1  107    108  0.959        0.959 Positive
## 2  103    116  0.954        0.954 Positive
## 3   94     96  0.969        0.969 Positive
## # i 168 more rows
##
## $subgraph_list$OE5
## # A tbl_graph: 111 nodes and 165 edges
## #
## # An undirected simple graph with 29 components

```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 167

```
## #
## # Node Data: 111 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_705 5          5          5          5          27      26.5 Bacter~
## 2 ASV_913 5          5          5          5          26      25.2 Bacter~
## 3 ASV_819 5          5          5          5          26      25.5 Bacter~
## 4 ASV_17~ 5          5          5          5          24      23.5 Bacter~
## 5 ASV_767 5          5          5          5          22      21.1 Bacter~
## 6 ASV_26~ 5          5          5          5          21      20.4 Bacter~
## 7 ASV_322 5          5          5          5          18      17.2 Bacter~
## 8 ASV_18~ 5          5          5          5          17      16.3 Bacter~
## 9 ASV_25~ 5          5          5          5          12      11.3 Bacter~
## 10 ASV_244 5          5          5          5          3       2.87 Bacter~
## # i 101 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## # Genus <chr>, Species <chr>
## #
## # Edge Data: 165 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1 101    102 0.959      0.959 Positive
## 2  91     93 0.969      0.969 Positive
## 3  87     89 0.943      0.943 Positive
## # i 162 more rows
##
## $subgraph_list$OE6
## # A tbl_graph: 140 nodes and 234 edges
## #
## # An undirected simple graph with 36 components
## #
## # Node Data: 140 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_705 5          5          5          5          27      26.5 Bacter~
## 2 ASV_12~ 5          5          5          5          27      26.5 Bacter~
## 3 ASV_913 5          5          5          5          26      25.2 Bacter~
## 4 ASV_819 5          5          5          5          26      25.5 Bacter~
## 5 ASV_25~ 5          5          5          5          25      24.4 Bacter~
## 6 ASV_17~ 5          5          5          5          24      23.5 Bacter~
## 7 ASV_767 5          5          5          5          22      21.1 Bacter~
## 8 ASV_11~ 5          5          5          5          20      19.1 Bacter~
## 9 ASV_28~ 5          5          5          5          20      19.1 Bacter~
## 10 ASV_996 5          5          5          5          19      18.3 Bacter~
## # i 130 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
```

```

## # Genus <chr>, Species <chr>
## #
## # Edge Data: 234 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1  125  126  0.959        0.959 Positive
## 2  119  138  0.954        0.954 Positive
## 3  108  110  0.969        0.969 Positive
## # i 231 more rows
##
## $subgraph_list$WT1
## # A tbl_graph: 105 nodes and 130 edges
## #
## # An undirected simple graph with 37 components
## #
## # Node Data: 105 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_25~ 5          5          5          5          27      26.4 Bacter~
## 2 ASV_913 5          5          5          5          26      25.2 Bacter~
## 3 ASV_819 5          5          5          5          26      25.5 Bacter~
## 4 ASV_28~ 5          5          5          5          23      22.5 Bacter~
## 5 ASV_767 5          5          5          5          22      21.1 Bacter~
## 6 ASV_26~ 5          5          5          5          21      20.4 Bacter~
## 7 ASV_322 5          5          5          5          18      17.2 Bacter~
## 8 ASV_15~ 5          5          5          5          18      17.1 Bacter~
## 9 ASV_18~ 5          5          5          5          17      16.3 Bacter~
## 10 ASV_25~ 5          5          5          5          12      11.3 Bacter~
## # i 95 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## # Genus <chr>, Species <chr>
## #
## # Edge Data: 130 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1      1      2  0.976        0.976 Positive
## 2      1      3  0.990        0.990 Positive
## 3      2      3  0.974        0.974 Positive
## # i 127 more rows
##
## $subgraph_list$WT2
## # A tbl_graph: 83 nodes and 59 edges
## #
## # An undirected simple graph with 41 components
## #
## # Node Data: 83 x 14 (active)

```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 169

```
##      name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##      <chr>      <fct>          <ord>          <chr>          <ord>          <dbl>      <dbl> <chr>
##  1 ASV_913 5          5          5          5          26      25.2 Bacter~
##  2 ASV_819 5          5          5          5          26      25.5 Bacter~
##  3 ASV_767 5          5          5          5          22      21.1 Bacter~
##  4 ASV_11~ 5          5          5          5          20      19.1 Bacter~
##  5 ASV_996 5          5          5          5          19      18.3 Bacter~
##  6 ASV_322 5          5          5          5          18      17.2 Bacter~
##  7 ASV_23~ 5          5          5          5          18      17.3 Bacter~
##  8 ASV_244 5          5          5          5          3       2.87 Bacter~
##  9 ASV_367 5          5          5          5          2       1.89 Bacter~
## 10 ASV_277 5          5          5          5          1       0.950 Bacter~
## # i 73 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 59 x 5
##   from    to weight correlation corr_direction
##   <int> <int> <dbl>          <dbl> <chr>
##  1      1      2  0.974          0.974 Positive
##  2      1      3  0.967          0.967 Positive
##  3      2      3  0.969          0.969 Positive
## # i 56 more rows
##
## $subgraph_list$WT3
## # A tbl_graph: 117 nodes and 171 edges
## #
## # An undirected simple graph with 38 components
## #
## # Node Data: 117 x 14 (active)
##      name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##      <chr>      <fct>          <ord>          <chr>          <ord>          <dbl>      <dbl> <chr>
##  1 ASV_12~ 5          5          5          5          27      26.5 Bacter~
##  2 ASV_14~ 5          5          5          5          27      26.5 Bacter~
##  3 ASV_913 5          5          5          5          26      25.2 Bacter~
##  4 ASV_24~ 5          5          5          5          23      22.6 Bacter~
##  5 ASV_28~ 5          5          5          5          23      22.5 Bacter~
##  6 ASV_767 5          5          5          5          22      21.1 Bacter~
##  7 ASV_996 5          5          5          5          19      18.3 Bacter~
##  8 ASV_322 5          5          5          5          18      17.2 Bacter~
##  9 ASV_23~ 5          5          5          5          18      17.3 Bacter~
## 10 ASV_18~ 5          5          5          5          17      16.3 Bacter~
## # i 107 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
```

```

## # Edge Data: 171 x 5
##   from   to weight correlation corr_direction
##   <int> <int> <dbl>         <dbl> <chr>
## 1  106   107  0.959         0.959 Positive
## 2   95    97  0.969         0.969 Positive
## 3   92    93  0.943         0.943 Positive
## # i 168 more rows
##
## $subgraph_list$WT4
## # A tbl_graph: 111 nodes and 222 edges
## #
## # An undirected simple graph with 34 components
## #
## # Node Data: 111 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_705 5          5          5          5          27      26.5 Bacter~
## 2 ASV_913 5          5          5          5          26      25.2 Bacter~
## 3 ASV_819 5          5          5          5          26      25.5 Bacter~
## 4 ASV_767 5          5          5          5          22      21.1 Bacter~
## 5 ASV_996 5          5          5          5          19      18.3 Bacter~
## 6 ASV_322 5          5          5          5          18      17.2 Bacter~
## 7 ASV_23~ 5          5          5          5          18      17.3 Bacter~
## 8 ASV_18~ 5          5          5          5          17      16.3 Bacter~
## 9 ASV_25~ 5          5          5          5          12      11.3 Bacter~
## 10 ASV_244 5          5          5          5          3       2.87 Bacter~
## # i 101 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 222 x 5
##   from   to weight correlation corr_direction
##   <int> <int> <dbl>         <dbl> <chr>
## 1  102   103  0.959         0.959 Positive
## 2   92    94  0.969         0.969 Positive
## 3   88    90  0.943         0.943 Positive
## # i 219 more rows
##
## $subgraph_list$WT5
## # A tbl_graph: 90 nodes and 103 edges
## #
## # An undirected simple graph with 32 components
## #
## # Node Data: 90 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>    <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>

```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 171

```
## 1 ASV_819 5          5          5          5          26 25.5 Bacter~
## 2 ASV_767 5          5          5          5          22 21.1 Bacter~
## 3 ASV_28~ 5          5          5          5          20 19.1 Bacter~
## 4 ASV_996 5          5          5          5          19 18.3 Bacter~
## 5 ASV_322 5          5          5          5          18 17.2 Bacter~
## 6 ASV_23~ 5          5          5          5          18 17.3 Bacter~
## 7 ASV_18~ 5          5          5          5          17 16.3 Bacter~
## 8 ASV_25~ 5          5          5          5          12 11.3 Bacter~
## 9 ASV_367 5          5          5          5          2  1.89 Bacter~
## 10 ASV_277 5         5          5          5          1  0.950 Bacter~
## # i 80 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 103 x 5
##   from   to weight correlation corr_direction
##   <int> <int> <dbl>      <dbl> <chr>
## 1     1     2 0.969      0.969 Positive
## 2     1     3 0.955      0.955 Positive
## 3     2     3 0.953      0.953 Positive
## # i 100 more rows
##
## $subgraph_list$WT6
## # A tbl_graph: 111 nodes and 162 edges
## #
## # An undirected simple graph with 36 components
## #
## # Node Data: 111 x 14 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>   <dbl> <chr>
## 1 ASV_705 5          5          5          5          27    26.5 Bacter~
## 2 ASV_913 5          5          5          5          26    25.2 Bacter~
## 3 ASV_819 5          5          5          5          26    25.5 Bacter~
## 4 ASV_17~ 5          5          5          5          24    23.5 Bacter~
## 5 ASV_767 5          5          5          5          22    21.1 Bacter~
## 6 ASV_28~ 5          5          5          5          20    19.1 Bacter~
## 7 ASV_322 5          5          5          5          18    17.2 Bacter~
## 8 ASV_15~ 5          5          5          5          18    17.1 Bacter~
## 9 ASV_23~ 5          5          5          5          18    17.3 Bacter~
## 10 ASV_18~ 5         5          5          5          17    16.3 Bacter~
## # i 101 more rows
## # i 6 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>
## #
## # Edge Data: 162 x 5
##   from   to weight correlation corr_direction
```

```
##   <int> <int>  <dbl>      <dbl> <chr>
## 1   103   104  0.959      0.959 Positive
## 2    88    90  0.969      0.969 Positive
## 3    84    86  0.943      0.943 Positive
## # i 159 more rows
##
##
## $topology
## # A tibble: 432 x 4
##   Sample Topology      Target_network Random_network
##   * <chr>   <chr>      <dbl>          <dbl>
## 1 K01     Node         113             113
## 2 K01     Edge         161             161
## 3 K01     Degree        2.85            2.85
## 4 K01     Distance      1.43            4.37
## 5 K01     Diameter      2.88            9.87
## 6 K01     Density        0.0254          0.0254
## 7 K01     Transitivity_global 0.741           0.0245
## 8 K01     Transitivity_local 0.702           0.0243
## 9 K01     Betweenness     1.24            165.
## 10 K01    Betweenness_edge 2.61            150.
## # i 422 more rows
##
## $Robustness
##   Sample Proportion.removed remain.mean  remain.sd  remain.se
## K01.1      K01             0.05 0.948148148 0.006580269 0.0006580269
## K01.2      K01             0.10 0.887222222 0.009153909 0.0009153909
## K01.3      K01             0.15 0.831944444 0.010559230 0.0010559230
## K01.4      K01             0.20 0.772314815 0.012422251 0.0012422251
## K01.5      K01             0.25 0.719444444 0.012240008 0.0012240008
## K01.6      K01             0.30 0.669351852 0.013568445 0.0013568445
## K01.7      K01             0.35 0.608611111 0.014347703 0.0014347703
## K01.8      K01             0.40 0.556018519 0.015312386 0.0015312386
## K01.9      K01             0.45 0.493796296 0.016545419 0.0016545419
## K01.10     K01             0.50 0.449629630 0.017275932 0.0017275932
## K01.11     K01             0.55 0.392037037 0.017119832 0.0017119832
## K01.12     K01             0.60 0.333425926 0.018401002 0.0018401002
## K01.13     K01             0.65 0.282037037 0.018724074 0.0018724074
## K01.14     K01             0.70 0.225370370 0.018575483 0.0018575483
## K01.15     K01             0.75 0.171759259 0.020907245 0.0020907245
## K01.16     K01             0.80 0.133333333 0.020217602 0.0020217602
## K01.17     K01             0.85 0.076944444 0.021812177 0.0021812177
## K01.18     K01             0.90 0.041574074 0.017631929 0.0017631929
## K01.19     K01             0.95 0.007222222 0.009633298 0.0009633298
## K01.20     K01             1.00 0.000000000 0.000000000 0.0000000000
## K01.21     K01             0.05 0.948796296 0.006373696 0.0006373696
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 173

## K01.22	K01	0.10	0.885740741	0.009043502	0.0009043502
## K01.23	K01	0.15	0.835648148	0.011196130	0.0011196130
## K01.24	K01	0.20	0.771111111	0.012559870	0.0012559870
## K01.25	K01	0.25	0.721296296	0.013388867	0.0013388867
## K01.26	K01	0.30	0.665740741	0.013127595	0.0013127595
## K01.27	K01	0.35	0.608240741	0.015939742	0.0015939742
## K01.28	K01	0.40	0.555833333	0.016670563	0.0016670563
## K01.29	K01	0.45	0.492129630	0.017378639	0.0017378639
## K01.30	K01	0.50	0.443333333	0.018531608	0.0018531608
## K01.31	K01	0.55	0.395833333	0.019625315	0.0019625315
## K01.32	K01	0.60	0.332592593	0.020501736	0.0020501736
## K01.33	K01	0.65	0.279629630	0.019740807	0.0019740807
## K01.34	K01	0.70	0.225462963	0.021195209	0.0021195209
## K01.35	K01	0.75	0.179351852	0.022436884	0.0022436884
## K01.36	K01	0.80	0.135092593	0.020349741	0.0020349741
## K01.37	K01	0.85	0.079259259	0.019622887	0.0019622887
## K01.38	K01	0.90	0.043518519	0.017384867	0.0017384867
## K01.39	K01	0.95	0.010185185	0.011282811	0.0011282811
## K01.40	K01	1.00	0.000000000	0.000000000	0.0000000000
## K02.1	K02	0.05	0.943949580	0.006326384	0.0006326384
## K02.2	K02	0.10	0.886638655	0.009739686	0.0009739686
## K02.3	K02	0.15	0.830252101	0.009921446	0.0009921446
## K02.4	K02	0.20	0.773781513	0.010527010	0.0010527010
## K02.5	K02	0.25	0.716638655	0.011017021	0.0011017021
## K02.6	K02	0.30	0.663865546	0.013138419	0.0013138419
## K02.7	K02	0.35	0.605630252	0.013420687	0.0013420687
## K02.8	K02	0.40	0.550000000	0.013532894	0.0013532894
## K02.9	K02	0.45	0.498823529	0.016701137	0.0016701137
## K02.10	K02	0.50	0.442100840	0.017055079	0.0017055079
## K02.11	K02	0.55	0.395714286	0.018545756	0.0018545756
## K02.12	K02	0.60	0.339075630	0.018049897	0.0018049897
## K02.13	K02	0.65	0.285630252	0.018941533	0.0018941533
## K02.14	K02	0.70	0.231344538	0.020599709	0.0020599709
## K02.15	K02	0.75	0.183949580	0.020889913	0.0020889913
## K02.16	K02	0.80	0.132268908	0.019245071	0.0019245071
## K02.17	K02	0.85	0.085714286	0.021828489	0.0021828489
## K02.18	K02	0.90	0.042100840	0.017695497	0.0017695497
## K02.19	K02	0.95	0.009915966	0.010161545	0.0010161545
## K02.20	K02	1.00	0.000000000	0.000000000	0.0000000000
## K02.21	K02	0.05	0.943865546	0.005713187	0.0005713187
## K02.22	K02	0.10	0.886890756	0.009284121	0.0009284121
## K02.23	K02	0.15	0.831428571	0.009616232	0.0009616232
## K02.24	K02	0.20	0.771932773	0.012181727	0.0012181727
## K02.25	K02	0.25	0.720420168	0.011754698	0.0011754698
## K02.26	K02	0.30	0.661680672	0.013909525	0.0013909525
## K02.27	K02	0.35	0.607478992	0.013633718	0.0013633718

## K02.28	K02	0.40	0.552941176	0.015618830	0.0015618830
## K02.29	K02	0.45	0.495546218	0.015639596	0.0015639596
## K02.30	K02	0.50	0.439411765	0.018164923	0.0018164923
## K02.31	K02	0.55	0.394117647	0.017896109	0.0017896109
## K02.32	K02	0.60	0.339327731	0.016712238	0.0016712238
## K02.33	K02	0.65	0.286806723	0.020537286	0.0020537286
## K02.34	K02	0.70	0.235042017	0.018674540	0.0018674540
## K02.35	K02	0.75	0.181680672	0.020191396	0.0020191396
## K02.36	K02	0.80	0.126974790	0.020124695	0.0020124695
## K02.37	K02	0.85	0.085630252	0.017903879	0.0017903879
## K02.38	K02	0.90	0.040336134	0.017184449	0.0017184449
## K02.39	K02	0.95	0.010756303	0.012359606	0.0012359606
## K02.40	K02	1.00	0.000000000	0.000000000	0.0000000000
## K03.1	K03	0.05	0.941919192	0.008893780	0.0008893780
## K03.2	K03	0.10	0.882929293	0.010161537	0.0010161537
## K03.3	K03	0.15	0.827171717	0.014704139	0.0014704139
## K03.4	K03	0.20	0.770808081	0.014619789	0.0014619789
## K03.5	K03	0.25	0.712424242	0.017255538	0.0017255538
## K03.6	K03	0.30	0.655151515	0.018554380	0.0018554380
## K03.7	K03	0.35	0.597676768	0.019424641	0.0019424641
## K03.8	K03	0.40	0.542828283	0.019401548	0.0019401548
## K03.9	K03	0.45	0.486666667	0.019964939	0.0019964939
## K03.10	K03	0.50	0.428989899	0.019604204	0.0019604204
## K03.11	K03	0.55	0.383737374	0.020022414	0.0020022414
## K03.12	K03	0.60	0.331212121	0.019543127	0.0019543127
## K03.13	K03	0.65	0.280000000	0.023019525	0.0023019525
## K03.14	K03	0.70	0.221414141	0.025869616	0.0025869616
## K03.15	K03	0.75	0.178080808	0.020688710	0.0020688710
## K03.16	K03	0.80	0.132424242	0.022236826	0.0022236826
## K03.17	K03	0.85	0.088383838	0.018518519	0.0018518519
## K03.18	K03	0.90	0.045858586	0.019811551	0.0019811551
## K03.19	K03	0.95	0.016666667	0.013030465	0.0013030465
## K03.20	K03	1.00	0.000000000	0.000000000	0.0000000000
## K03.21	K03	0.05	0.941212121	0.007648361	0.0007648361
## K03.22	K03	0.10	0.883030303	0.011229669	0.0011229669
## K03.23	K03	0.15	0.824242424	0.012679715	0.0012679715
## K03.24	K03	0.20	0.770000000	0.013908840	0.0013908840
## K03.25	K03	0.25	0.713030303	0.016312364	0.0016312364
## K03.26	K03	0.30	0.653636364	0.017424335	0.0017424335
## K03.27	K03	0.35	0.598080808	0.017305635	0.0017305635
## K03.28	K03	0.40	0.542727273	0.019782138	0.0019782138
## K03.29	K03	0.45	0.484141414	0.021355451	0.0021355451
## K03.30	K03	0.50	0.431919192	0.021348935	0.0021348935
## K03.31	K03	0.55	0.384545455	0.022160687	0.0022160687
## K03.32	K03	0.60	0.326969697	0.021945716	0.0021945716
## K03.33	K03	0.65	0.274141414	0.021631880	0.0021631880

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 175

## K03.34	K03	0.70	0.229191919	0.021182950	0.0021182950
## K03.35	K03	0.75	0.180505051	0.022456354	0.0022456354
## K03.36	K03	0.80	0.132828283	0.022671939	0.0022671939
## K03.37	K03	0.85	0.089393939	0.020118950	0.0020118950
## K03.38	K03	0.90	0.050808081	0.018015256	0.0018015256
## K03.39	K03	0.95	0.015454545	0.013804714	0.0013804714
## K03.40	K03	1.00	0.000000000	0.000000000	0.0000000000
## K04.1	K04	0.05	0.947303371	0.008554813	0.0008554813
## K04.2	K04	0.10	0.880449438	0.012006280	0.0012006280
## K04.3	K04	0.15	0.828876404	0.016275004	0.0016275004
## K04.4	K04	0.20	0.765168539	0.016325464	0.0016325464
## K04.5	K04	0.25	0.713258427	0.016861875	0.0016861875
## K04.6	K04	0.30	0.647078652	0.018285231	0.0018285231
## K04.7	K04	0.35	0.592471910	0.017556571	0.0017556571
## K04.8	K04	0.40	0.527865169	0.020882383	0.0020882383
## K04.9	K04	0.45	0.482808989	0.021865324	0.0021865324
## K04.10	K04	0.50	0.431910112	0.023050103	0.0023050103
## K04.11	K04	0.55	0.366292135	0.023307581	0.0023307581
## K04.12	K04	0.60	0.322247191	0.022914716	0.0022914716
## K04.13	K04	0.65	0.256629213	0.028466211	0.0028466211
## K04.14	K04	0.70	0.213820225	0.028319569	0.0028319569
## K04.15	K04	0.75	0.159550562	0.022753877	0.0022753877
## K04.16	K04	0.80	0.116516854	0.025233502	0.0025233502
## K04.17	K04	0.85	0.065842697	0.024325967	0.0024325967
## K04.18	K04	0.90	0.032359551	0.019512288	0.0019512288
## K04.19	K04	0.95	0.005280899	0.010767087	0.0010767087
## K04.20	K04	1.00	0.000000000	0.000000000	0.0000000000
## K04.21	K04	0.05	0.946292135	0.008386959	0.0008386959
## K04.22	K04	0.10	0.882134831	0.011896914	0.0011896914
## K04.23	K04	0.15	0.827640449	0.015169065	0.0015169065
## K04.24	K04	0.20	0.763595506	0.015311312	0.0015311312
## K04.25	K04	0.25	0.710337079	0.016641111	0.0016641111
## K04.26	K04	0.30	0.646629213	0.017521672	0.0017521672
## K04.27	K04	0.35	0.596629213	0.019197352	0.0019197352
## K04.28	K04	0.40	0.533370787	0.019458296	0.0019458296
## K04.29	K04	0.45	0.482471910	0.020872610	0.0020872610
## K04.30	K04	0.50	0.432921348	0.022835276	0.0022835276
## K04.31	K04	0.55	0.372359551	0.026532469	0.0026532469
## K04.32	K04	0.60	0.323258427	0.027189285	0.0027189285
## K04.33	K04	0.65	0.266067416	0.028961838	0.0028961838
## K04.34	K04	0.70	0.213932584	0.025796517	0.0025796517
## K04.35	K04	0.75	0.158314607	0.024590570	0.0024590570
## K04.36	K04	0.80	0.119550562	0.025277179	0.0025277179
## K04.37	K04	0.85	0.065730337	0.022997763	0.0022997763
## K04.38	K04	0.90	0.035955056	0.019689251	0.0019689251
## K04.39	K04	0.95	0.007303371	0.011557647	0.0011557647

## K04.40	K04	1.00	0.000000000	0.000000000	0.000000000
## K05.1	K05	0.05	0.947910448	0.008615862	0.0008615862
## K05.2	K05	0.10	0.878955224	0.012709825	0.0012709825
## K05.3	K05	0.15	0.827313433	0.013459611	0.0013459611
## K05.4	K05	0.20	0.778208955	0.016973853	0.0016973853
## K05.5	K05	0.25	0.706417910	0.020131518	0.0020131518
## K05.6	K05	0.30	0.650895522	0.019080223	0.0019080223
## K05.7	K05	0.35	0.602686567	0.021190653	0.0021190653
## K05.8	K05	0.40	0.535223881	0.021843212	0.0021843212
## K05.9	K05	0.45	0.482089552	0.022198808	0.0022198808
## K05.10	K05	0.50	0.418955224	0.022275207	0.0022275207
## K05.11	K05	0.55	0.369253731	0.025066657	0.0025066657
## K05.12	K05	0.60	0.316865672	0.026821664	0.0026821664
## K05.13	K05	0.65	0.254626866	0.026138887	0.0026138887
## K05.14	K05	0.70	0.207611940	0.026757828	0.0026757828
## K05.15	K05	0.75	0.167910448	0.031205140	0.0031205140
## K05.16	K05	0.80	0.103283582	0.029293763	0.0029293763
## K05.17	K05	0.85	0.067761194	0.026531924	0.0026531924
## K05.18	K05	0.90	0.034029851	0.022358390	0.0022358390
## K05.19	K05	0.95	0.005820896	0.012167113	0.0012167113
## K05.20	K05	1.00	0.000000000	0.000000000	0.000000000
## K05.21	K05	0.05	0.948805970	0.008009515	0.0008009515
## K05.22	K05	0.10	0.878656716	0.012843672	0.0012843672
## K05.23	K05	0.15	0.827014925	0.016169328	0.0016169328
## K05.24	K05	0.20	0.774328358	0.016333404	0.0016333404
## K05.25	K05	0.25	0.706119403	0.019407623	0.0019407623
## K05.26	K05	0.30	0.655074627	0.019430798	0.0019430798
## K05.27	K05	0.35	0.603880597	0.021573711	0.0021573711
## K05.28	K05	0.40	0.532985075	0.020380352	0.0020380352
## K05.29	K05	0.45	0.485671642	0.024133666	0.0024133666
## K05.30	K05	0.50	0.424626866	0.024222544	0.0024222544
## K05.31	K05	0.55	0.366567164	0.025748590	0.0025748590
## K05.32	K05	0.60	0.322089552	0.023829661	0.0023829661
## K05.33	K05	0.65	0.252985075	0.027771308	0.0027771308
## K05.34	K05	0.70	0.209701493	0.025753396	0.0025753396
## K05.35	K05	0.75	0.166119403	0.032977710	0.0032977710
## K05.36	K05	0.80	0.110895522	0.028323274	0.0028323274
## K05.37	K05	0.85	0.068507463	0.028310560	0.0028310560
## K05.38	K05	0.90	0.036865672	0.021990541	0.0021990541
## K05.39	K05	0.95	0.006567164	0.012427857	0.0012427857
## K05.40	K05	1.00	0.000000000	0.000000000	0.000000000
## K06.1	K06	0.05	0.939354839	0.007567888	0.0007567888
## K06.2	K06	0.10	0.893010753	0.008423112	0.0008423112
## K06.3	K06	0.15	0.832688172	0.011469534	0.0011469534
## K06.4	K06	0.20	0.772580645	0.013786688	0.0013786688
## K06.5	K06	0.25	0.727096774	0.012674929	0.0012674929

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 177

## K06.6	K06	0.30	0.666451613	0.014972862	0.0014972862
## K06.7	K06	0.35	0.604408602	0.014919336	0.0014919336
## K06.8	K06	0.40	0.559462366	0.017787380	0.0017787380
## K06.9	K06	0.45	0.499892473	0.017324375	0.0017324375
## K06.10	K06	0.50	0.453225806	0.014925598	0.0014925598
## K06.11	K06	0.55	0.390967742	0.020760422	0.0020760422
## K06.12	K06	0.60	0.332150538	0.019565786	0.0019565786
## K06.13	K06	0.65	0.287634409	0.022272355	0.0022272355
## K06.14	K06	0.70	0.232365591	0.023323544	0.0023323544
## K06.15	K06	0.75	0.178172043	0.024128837	0.0024128837
## K06.16	K06	0.80	0.134731183	0.021421308	0.0021421308
## K06.17	K06	0.85	0.087634409	0.021197701	0.0021197701
## K06.18	K06	0.90	0.039247312	0.018107535	0.0018107535
## K06.19	K06	0.95	0.011935484	0.013488674	0.0013488674
## K06.20	K06	1.00	0.000000000	0.000000000	0.0000000000
## K06.21	K06	0.05	0.939784946	0.008087112	0.0008087112
## K06.22	K06	0.10	0.892365591	0.008439734	0.0008439734
## K06.23	K06	0.15	0.832903226	0.010852152	0.0010852152
## K06.24	K06	0.20	0.776881720	0.012070346	0.0012070346
## K06.25	K06	0.25	0.725483871	0.012736972	0.0012736972
## K06.26	K06	0.30	0.664731183	0.013616641	0.0013616641
## K06.27	K06	0.35	0.606236559	0.017208374	0.0017208374
## K06.28	K06	0.40	0.558172043	0.017229738	0.0017229738
## K06.29	K06	0.45	0.498279570	0.016117805	0.0016117805
## K06.30	K06	0.50	0.451827957	0.017289622	0.0017289622
## K06.31	K06	0.55	0.394946237	0.020496311	0.0020496311
## K06.32	K06	0.60	0.335591398	0.019649175	0.0019649175
## K06.33	K06	0.65	0.291935484	0.019474846	0.0019474846
## K06.34	K06	0.70	0.229892473	0.020377735	0.0020377735
## K06.35	K06	0.75	0.175591398	0.022044666	0.0022044666
## K06.36	K06	0.80	0.133978495	0.023991225	0.0023991225
## K06.37	K06	0.85	0.083225806	0.020195818	0.0020195818
## K06.38	K06	0.90	0.042688172	0.018177058	0.0018177058
## K06.39	K06	0.95	0.013333333	0.012979032	0.0012979032
## K06.40	K06	1.00	0.000000000	0.000000000	0.0000000000
## OE1.1	OE1	0.05	0.942000000	0.006579637	0.0006579637
## OE1.2	OE1	0.10	0.882592593	0.008512742	0.0008512742
## OE1.3	OE1	0.15	0.833037037	0.009500122	0.0009500122
## OE1.4	OE1	0.20	0.774814815	0.011723964	0.0011723964
## OE1.5	OE1	0.25	0.719111111	0.009398062	0.0009398062
## OE1.6	OE1	0.30	0.669777778	0.011779615	0.0011779615
## OE1.7	OE1	0.35	0.612888889	0.012078822	0.0012078822
## OE1.8	OE1	0.40	0.557777778	0.013503507	0.0013503507
## OE1.9	OE1	0.45	0.502962963	0.014066393	0.0014066393
## OE1.10	OE1	0.50	0.445925926	0.014007166	0.0014007166
## OE1.11	OE1	0.55	0.398518519	0.016845489	0.0016845489

##	OE1.12	OE1	0.60	0.342370370	0.017190061	0.0017190061
##	OE1.13	OE1	0.65	0.290814815	0.017510777	0.0017510777
##	OE1.14	OE1	0.70	0.236222222	0.017200214	0.0017200214
##	OE1.15	OE1	0.75	0.190296296	0.017857875	0.0017857875
##	OE1.16	OE1	0.80	0.142592593	0.017053945	0.0017053945
##	OE1.17	OE1	0.85	0.095629630	0.017651849	0.0017651849
##	OE1.18	OE1	0.90	0.052000000	0.016679506	0.0016679506
##	OE1.19	OE1	0.95	0.015851852	0.011865873	0.0011865873
##	OE1.20	OE1	1.00	0.000000000	0.000000000	0.0000000000
##	OE1.21	OE1	0.05	0.941481481	0.006002129	0.0006002129
##	OE1.22	OE1	0.10	0.881555556	0.009087139	0.0009087139
##	OE1.23	OE1	0.15	0.833481481	0.010069936	0.0010069936
##	OE1.24	OE1	0.20	0.776444444	0.011011148	0.0011011148
##	OE1.25	OE1	0.25	0.719111111	0.011756066	0.0011756066
##	OE1.26	OE1	0.30	0.668740741	0.012191177	0.0012191177
##	OE1.27	OE1	0.35	0.612740741	0.013649654	0.0013649654
##	OE1.28	OE1	0.40	0.558296296	0.013468780	0.0013468780
##	OE1.29	OE1	0.45	0.501185185	0.014748073	0.0014748073
##	OE1.30	OE1	0.50	0.444814815	0.014488551	0.0014488551
##	OE1.31	OE1	0.55	0.396296296	0.016729938	0.0016729938
##	OE1.32	OE1	0.60	0.343481481	0.015674814	0.0015674814
##	OE1.33	OE1	0.65	0.292074074	0.018121597	0.0018121597
##	OE1.34	OE1	0.70	0.238518519	0.017522801	0.0017522801
##	OE1.35	OE1	0.75	0.194518519	0.018102166	0.0018102166
##	OE1.36	OE1	0.80	0.143851852	0.016917062	0.0016917062
##	OE1.37	OE1	0.85	0.092592593	0.016829030	0.0016829030
##	OE1.38	OE1	0.90	0.048222222	0.013212866	0.0013212866
##	OE1.39	OE1	0.95	0.015185185	0.012518432	0.0012518432
##	OE1.40	OE1	1.00	0.000000000	0.000000000	0.0000000000
##	OE2.1	OE2	0.05	0.941503759	0.005909687	0.0005909687
##	OE2.2	OE2	0.10	0.890526316	0.007505114	0.0007505114
##	OE2.3	OE2	0.15	0.833082707	0.009677266	0.0009677266
##	OE2.4	OE2	0.20	0.773984962	0.011199187	0.0011199187
##	OE2.5	OE2	0.25	0.723007519	0.012632085	0.0012632085
##	OE2.6	OE2	0.30	0.666090226	0.013437078	0.0013437078
##	OE2.7	OE2	0.35	0.607067669	0.012536795	0.0012536795
##	OE2.8	OE2	0.40	0.558195489	0.013897710	0.0013897710
##	OE2.9	OE2	0.45	0.501278195	0.012419414	0.0012419414
##	OE2.10	OE2	0.50	0.454210526	0.015416173	0.0015416173
##	OE2.11	OE2	0.55	0.394060150	0.017334130	0.0017334130
##	OE2.12	OE2	0.60	0.339248120	0.018113337	0.0018113337
##	OE2.13	OE2	0.65	0.295263158	0.020496707	0.0020496707
##	OE2.14	OE2	0.70	0.235112782	0.020635536	0.0020635536
##	OE2.15	OE2	0.75	0.186466165	0.018755175	0.0018755175
##	OE2.16	OE2	0.80	0.140375940	0.017820140	0.0017820140
##	OE2.17	OE2	0.85	0.088270677	0.019402270	0.0019402270

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 179

## OE2.18	OE2	0.90	0.043383459	0.015444299	0.0015444299
## OE2.19	OE2	0.95	0.016015038	0.010774823	0.0010774823
## OE2.20	OE2	1.00	0.000000000	0.000000000	0.0000000000
## OE2.21	OE2	0.05	0.941654135	0.006056665	0.0006056665
## OE2.22	OE2	0.10	0.889849624	0.008165057	0.0008165057
## OE2.23	OE2	0.15	0.831278195	0.009754845	0.0009754845
## OE2.24	OE2	0.20	0.774661654	0.010974364	0.0010974364
## OE2.25	OE2	0.25	0.722631579	0.010681141	0.0010681141
## OE2.26	OE2	0.30	0.663233083	0.013986792	0.0013986792
## OE2.27	OE2	0.35	0.608872180	0.013933001	0.0013933001
## OE2.28	OE2	0.40	0.560601504	0.015163143	0.0015163143
## OE2.29	OE2	0.45	0.505563910	0.014182412	0.0014182412
## OE2.30	OE2	0.50	0.452706767	0.017763653	0.0017763653
## OE2.31	OE2	0.55	0.399699248	0.016302816	0.0016302816
## OE2.32	OE2	0.60	0.340225564	0.017734054	0.0017734054
## OE2.33	OE2	0.65	0.293984962	0.018921891	0.0018921891
## OE2.34	OE2	0.70	0.236992481	0.020035310	0.0020035310
## OE2.35	OE2	0.75	0.184887218	0.015890414	0.0015890414
## OE2.36	OE2	0.80	0.139849624	0.018663611	0.0018663611
## OE2.37	OE2	0.85	0.089548872	0.017949132	0.0017949132
## OE2.38	OE2	0.90	0.042180451	0.014570673	0.0014570673
## OE2.39	OE2	0.95	0.014436090	0.012263252	0.0012263252
## OE2.40	OE2	1.00	0.000000000	0.000000000	0.0000000000
## OE3.1	OE3	0.05	0.942222222	0.006667132	0.0006667132
## OE3.2	OE3	0.10	0.883760684	0.009642326	0.0009642326
## OE3.3	OE3	0.15	0.824700855	0.012238660	0.0012238660
## OE3.4	OE3	0.20	0.776068376	0.013027481	0.0013027481
## OE3.5	OE3	0.25	0.718803419	0.013168323	0.0013168323
## OE3.6	OE3	0.30	0.663504274	0.016008668	0.0016008668
## OE3.7	OE3	0.35	0.605299145	0.015480244	0.0015480244
## OE3.8	OE3	0.40	0.549487179	0.014843427	0.0014843427
## OE3.9	OE3	0.45	0.492478632	0.017041272	0.0017041272
## OE3.10	OE3	0.50	0.443760684	0.018649745	0.0018649745
## OE3.11	OE3	0.55	0.385982906	0.018965187	0.0018965187
## OE3.12	OE3	0.60	0.327948718	0.021651570	0.0021651570
## OE3.13	OE3	0.65	0.276153846	0.020280382	0.0020280382
## OE3.14	OE3	0.70	0.222307692	0.021128596	0.0021128596
## OE3.15	OE3	0.75	0.171794872	0.021578002	0.0021578002
## OE3.16	OE3	0.80	0.117948718	0.021076320	0.0021076320
## OE3.17	OE3	0.85	0.081623932	0.020232297	0.0020232297
## OE3.18	OE3	0.90	0.040683761	0.015324063	0.0015324063
## OE3.19	OE3	0.95	0.009829060	0.010334889	0.0010334889
## OE3.20	OE3	1.00	0.000000000	0.000000000	0.0000000000
## OE3.21	OE3	0.05	0.940854701	0.006944685	0.0006944685
## OE3.22	OE3	0.10	0.883333333	0.011025778	0.0011025778
## OE3.23	OE3	0.15	0.827777778	0.011420267	0.0011420267

##	OE3.24	OE3	0.20	0.775811966	0.011515496	0.0011515496
##	OE3.25	OE3	0.25	0.718974359	0.012641123	0.0012641123
##	OE3.26	OE3	0.30	0.661794872	0.013154587	0.0013154587
##	OE3.27	OE3	0.35	0.605042735	0.015040959	0.0015040959
##	OE3.28	OE3	0.40	0.550085470	0.014854608	0.0014854608
##	OE3.29	OE3	0.45	0.493076923	0.016102423	0.0016102423
##	OE3.30	OE3	0.50	0.444273504	0.019360265	0.0019360265
##	OE3.31	OE3	0.55	0.391709402	0.018425843	0.0018425843
##	OE3.32	OE3	0.60	0.330341880	0.019864239	0.0019864239
##	OE3.33	OE3	0.65	0.277435897	0.020233938	0.0020233938
##	OE3.34	OE3	0.70	0.221623932	0.018869035	0.0018869035
##	OE3.35	OE3	0.75	0.165299145	0.021840403	0.0021840403
##	OE3.36	OE3	0.80	0.115042735	0.024095712	0.0024095712
##	OE3.37	OE3	0.85	0.077435897	0.022328226	0.0022328226
##	OE3.38	OE3	0.90	0.040256410	0.018475435	0.0018475435
##	OE3.39	OE3	0.95	0.008717949	0.010932020	0.0010932020
##	OE3.40	OE3	1.00	0.000000000	0.000000000	0.000000000
##	OE4.1	OE4	0.05	0.940775862	0.007418328	0.0007418328
##	OE4.2	OE4	0.10	0.885258621	0.010081327	0.0010081327
##	OE4.3	OE4	0.15	0.835086207	0.012414507	0.0012414507
##	OE4.4	OE4	0.20	0.778620690	0.012060255	0.0012060255
##	OE4.5	OE4	0.25	0.720517241	0.013263745	0.0013263745
##	OE4.6	OE4	0.30	0.665431034	0.015337938	0.0015337938
##	OE4.7	OE4	0.35	0.610603448	0.015109141	0.0015109141
##	OE4.8	OE4	0.40	0.560603448	0.014828305	0.0014828305
##	OE4.9	OE4	0.45	0.505775862	0.017156487	0.0017156487
##	OE4.10	OE4	0.50	0.447586207	0.017096219	0.0017096219
##	OE4.11	OE4	0.55	0.389741379	0.018553964	0.0018553964
##	OE4.12	OE4	0.60	0.336551724	0.019409174	0.0019409174
##	OE4.13	OE4	0.65	0.289482759	0.018750563	0.0018750563
##	OE4.14	OE4	0.70	0.237844828	0.018946311	0.0018946311
##	OE4.15	OE4	0.75	0.183103448	0.022630440	0.0022630440
##	OE4.16	OE4	0.80	0.132241379	0.020437774	0.0020437774
##	OE4.17	OE4	0.85	0.084482759	0.021952901	0.0021952901
##	OE4.18	OE4	0.90	0.047586207	0.017818620	0.0017818620
##	OE4.19	OE4	0.95	0.011810345	0.011601835	0.0011601835
##	OE4.20	OE4	1.00	0.000000000	0.000000000	0.000000000
##	OE4.21	OE4	0.05	0.942844828	0.006084038	0.0006084038
##	OE4.22	OE4	0.10	0.882931034	0.010117376	0.0010117376
##	OE4.23	OE4	0.15	0.837844828	0.011342718	0.0011342718
##	OE4.24	OE4	0.20	0.777844828	0.013470128	0.0013470128
##	OE4.25	OE4	0.25	0.721206897	0.014248196	0.0014248196
##	OE4.26	OE4	0.30	0.666293103	0.015021444	0.0015021444
##	OE4.27	OE4	0.35	0.606982759	0.015980309	0.0015980309
##	OE4.28	OE4	0.40	0.563103448	0.014543338	0.0014543338
##	OE4.29	OE4	0.45	0.504655172	0.016458178	0.0016458178

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 181

##	OE4.30	OE4	0.50	0.449827586	0.017982170	0.0017982170
##	OE4.31	OE4	0.55	0.391465517	0.018946311	0.0018946311
##	OE4.32	OE4	0.60	0.337327586	0.017356590	0.0017356590
##	OE4.33	OE4	0.65	0.290948276	0.018996958	0.0018996958
##	OE4.34	OE4	0.70	0.238965517	0.018585697	0.0018585697
##	OE4.35	OE4	0.75	0.180517241	0.019826261	0.0019826261
##	OE4.36	OE4	0.80	0.128706897	0.022501870	0.0022501870
##	OE4.37	OE4	0.85	0.077931034	0.018859939	0.0018859939
##	OE4.38	OE4	0.90	0.046724138	0.019068150	0.0019068150
##	OE4.39	OE4	0.95	0.010689655	0.011103602	0.0011103602
##	OE4.40	OE4	1.00	0.000000000	0.000000000	0.0000000000
##	OE5.1	OE5	0.05	0.947570093	0.005774137	0.0005774137
##	OE5.2	OE5	0.10	0.884112150	0.008608714	0.0008608714
##	OE5.3	OE5	0.15	0.834672897	0.008679648	0.0008679648
##	OE5.4	OE5	0.20	0.782149533	0.011783850	0.0011783850
##	OE5.5	OE5	0.25	0.718971963	0.011446599	0.0011446599
##	OE5.6	OE5	0.30	0.669345794	0.012656357	0.0012656357
##	OE5.7	OE5	0.35	0.617570093	0.015977833	0.0015977833
##	OE5.8	OE5	0.40	0.555140187	0.013740594	0.0013740594
##	OE5.9	OE5	0.45	0.505233645	0.016949914	0.0016949914
##	OE5.10	OE5	0.50	0.446915888	0.017389764	0.0017389764
##	OE5.11	OE5	0.55	0.395420561	0.017657359	0.0017657359
##	OE5.12	OE5	0.60	0.347943925	0.017962511	0.0017962511
##	OE5.13	OE5	0.65	0.284112150	0.018644328	0.0018644328
##	OE5.14	OE5	0.70	0.239532710	0.019687210	0.0019687210
##	OE5.15	OE5	0.75	0.190280374	0.018526601	0.0018526601
##	OE5.16	OE5	0.80	0.132990654	0.016951736	0.0016951736
##	OE5.17	OE5	0.85	0.088411215	0.019541699	0.0019541699
##	OE5.18	OE5	0.90	0.044766355	0.019626168	0.0019626168
##	OE5.19	OE5	0.95	0.012336449	0.014299402	0.0014299402
##	OE5.20	OE5	1.00	0.000000000	0.000000000	0.0000000000
##	OE5.21	OE5	0.05	0.947196262	0.007199504	0.0007199504
##	OE5.22	OE5	0.10	0.885046729	0.009806451	0.0009806451
##	OE5.23	OE5	0.15	0.833271028	0.011011721	0.0011011721
##	OE5.24	OE5	0.20	0.782897196	0.010779733	0.0010779733
##	OE5.25	OE5	0.25	0.717570093	0.014646865	0.0014646865
##	OE5.26	OE5	0.30	0.670186916	0.013303113	0.0013303113
##	OE5.27	OE5	0.35	0.615607477	0.015414338	0.0015414338
##	OE5.28	OE5	0.40	0.559439252	0.014308037	0.0014308037
##	OE5.29	OE5	0.45	0.504766355	0.014641745	0.0014641745
##	OE5.30	OE5	0.50	0.447663551	0.016985272	0.0016985272
##	OE5.31	OE5	0.55	0.396261682	0.016105429	0.0016105429
##	OE5.32	OE5	0.60	0.346448598	0.015562441	0.0015562441
##	OE5.33	OE5	0.65	0.284579439	0.019221002	0.0019221002
##	OE5.34	OE5	0.70	0.235700935	0.017041278	0.0017041278
##	OE5.35	OE5	0.75	0.192056075	0.019358215	0.0019358215

## OE5.36	OE5	0.80	0.131962617	0.020201456	0.0020201456
## OE5.37	OE5	0.85	0.089532710	0.019073785	0.0019073785
## OE5.38	OE5	0.90	0.047943925	0.018341621	0.0018341621
## OE5.39	OE5	0.95	0.012056075	0.011216133	0.0011216133
## OE5.40	OE5	1.00	0.000000000	0.000000000	0.0000000000
## OE6.1	OE6	0.05	0.945072464	0.005563665	0.0005563665
## OE6.2	OE6	0.10	0.888695652	0.007247841	0.0007247841
## OE6.3	OE6	0.15	0.832971014	0.008822514	0.0008822514
## OE6.4	OE6	0.20	0.776014493	0.009275031	0.0009275031
## OE6.5	OE6	0.25	0.728043478	0.010824579	0.0010824579
## OE6.6	OE6	0.30	0.674057971	0.010899048	0.0010899048
## OE6.7	OE6	0.35	0.618115942	0.013837477	0.0013837477
## OE6.8	OE6	0.40	0.565144928	0.013760409	0.0013760409
## OE6.9	OE6	0.45	0.507463768	0.015430605	0.0015430605
## OE6.10	OE6	0.50	0.450507246	0.016090227	0.0016090227
## OE6.11	OE6	0.55	0.397028986	0.013829617	0.0013829617
## OE6.12	OE6	0.60	0.341811594	0.016956922	0.0016956922
## OE6.13	OE6	0.65	0.288985507	0.017457056	0.0017457056
## OE6.14	OE6	0.70	0.237753623	0.017039300	0.0017039300
## OE6.15	OE6	0.75	0.180652174	0.017425885	0.0017425885
## OE6.16	OE6	0.80	0.137898551	0.017553258	0.0017553258
## OE6.17	OE6	0.85	0.089710145	0.017743350	0.0017743350
## OE6.18	OE6	0.90	0.046956522	0.015397054	0.0015397054
## OE6.19	OE6	0.95	0.012608696	0.011074772	0.0011074772
## OE6.20	OE6	1.00	0.000000000	0.000000000	0.0000000000
## OE6.21	OE6	0.05	0.944275362	0.005612547	0.0005612547
## OE6.22	OE6	0.10	0.889057971	0.007333324	0.0007333324
## OE6.23	OE6	0.15	0.835072464	0.009161392	0.0009161392
## OE6.24	OE6	0.20	0.777608696	0.011030382	0.0011030382
## OE6.25	OE6	0.25	0.727101449	0.011662622	0.0011662622
## OE6.26	OE6	0.30	0.673405797	0.011294575	0.0011294575
## OE6.27	OE6	0.35	0.619710145	0.012759810	0.0012759810
## OE6.28	OE6	0.40	0.563188406	0.013031273	0.0013031273
## OE6.29	OE6	0.45	0.506304348	0.015540211	0.0015540211
## OE6.30	OE6	0.50	0.454565217	0.016825035	0.0016825035
## OE6.31	OE6	0.55	0.397028986	0.015765212	0.0015765212
## OE6.32	OE6	0.60	0.340942029	0.016587555	0.0016587555
## OE6.33	OE6	0.65	0.288695652	0.015913390	0.0015913390
## OE6.34	OE6	0.70	0.233333333	0.020624836	0.0020624836
## OE6.35	OE6	0.75	0.180000000	0.019315984	0.0019315984
## OE6.36	OE6	0.80	0.136666667	0.019731640	0.0019731640
## OE6.37	OE6	0.85	0.087318841	0.019001905	0.0019001905
## OE6.38	OE6	0.90	0.046594203	0.016162586	0.0016162586
## OE6.39	OE6	0.95	0.013985507	0.011434590	0.0011434590
## OE6.40	OE6	1.00	0.000000000	0.000000000	0.0000000000
## WT1.1	WT1	0.05	0.943030303	0.007109228	0.0007109228

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 183

##	WT1.2	WT1	0.10	0.885555556	0.009199098	0.0009199098
##	WT1.3	WT1	0.15	0.829090909	0.012406931	0.0012406931
##	WT1.4	WT1	0.20	0.770505051	0.014290756	0.0014290756
##	WT1.5	WT1	0.25	0.716363636	0.013671557	0.0013671557
##	WT1.6	WT1	0.30	0.660202020	0.015296723	0.0015296723
##	WT1.7	WT1	0.35	0.602626263	0.015343813	0.0015343813
##	WT1.8	WT1	0.40	0.551515152	0.017465984	0.0017465984
##	WT1.9	WT1	0.45	0.490303030	0.019915321	0.0019915321
##	WT1.10	WT1	0.50	0.437070707	0.018766188	0.0018766188
##	WT1.11	WT1	0.55	0.398787879	0.021172487	0.0021172487
##	WT1.12	WT1	0.60	0.339191919	0.020312929	0.0020312929
##	WT1.13	WT1	0.65	0.285252525	0.018392593	0.0018392593
##	WT1.14	WT1	0.70	0.231212121	0.019148911	0.0019148911
##	WT1.15	WT1	0.75	0.179292929	0.022853046	0.0022853046
##	WT1.16	WT1	0.80	0.125555556	0.022020726	0.0022020726
##	WT1.17	WT1	0.85	0.082525253	0.019318236	0.0019318236
##	WT1.18	WT1	0.90	0.042929293	0.020170110	0.0020170110
##	WT1.19	WT1	0.95	0.009595960	0.011958142	0.0011958142
##	WT1.20	WT1	1.00	0.000000000	0.000000000	0.0000000000
##	WT1.21	WT1	0.05	0.943838384	0.006629119	0.0006629119
##	WT1.22	WT1	0.10	0.883434343	0.011251673	0.0011251673
##	WT1.23	WT1	0.15	0.827676768	0.013300148	0.0013300148
##	WT1.24	WT1	0.20	0.770909091	0.014491278	0.0014491278
##	WT1.25	WT1	0.25	0.716060606	0.016108921	0.0016108921
##	WT1.26	WT1	0.30	0.660000000	0.016001400	0.0016001400
##	WT1.27	WT1	0.35	0.602828283	0.016611929	0.0016611929
##	WT1.28	WT1	0.40	0.548282828	0.016992226	0.0016992226
##	WT1.29	WT1	0.45	0.493535354	0.017408062	0.0017408062
##	WT1.30	WT1	0.50	0.437777778	0.017654963	0.0017654963
##	WT1.31	WT1	0.55	0.392626263	0.020638834	0.0020638834
##	WT1.32	WT1	0.60	0.340202020	0.020449459	0.0020449459
##	WT1.33	WT1	0.65	0.286363636	0.020624848	0.0020624848
##	WT1.34	WT1	0.70	0.234343434	0.019208291	0.0019208291
##	WT1.35	WT1	0.75	0.181414141	0.023631035	0.0023631035
##	WT1.36	WT1	0.80	0.131111111	0.026355076	0.0026355076
##	WT1.37	WT1	0.85	0.085151515	0.021690639	0.0021690639
##	WT1.38	WT1	0.90	0.043535354	0.019245544	0.0019245544
##	WT1.39	WT1	0.95	0.008888889	0.011688357	0.0011688357
##	WT1.40	WT1	1.00	0.000000000	0.000000000	0.0000000000
##	WT2.1	WT2	0.05	0.941012658	0.011267410	0.0011267410
##	WT2.2	WT2	0.10	0.883037975	0.012863695	0.0012863695
##	WT2.3	WT2	0.15	0.822278481	0.014827541	0.0014827541
##	WT2.4	WT2	0.20	0.764936709	0.017651514	0.0017651514
##	WT2.5	WT2	0.25	0.708734177	0.018997568	0.0018997568
##	WT2.6	WT2	0.30	0.649493671	0.020410383	0.0020410383
##	WT2.7	WT2	0.35	0.592025316	0.020263950	0.0020263950

## WT2.8	WT2	0.40	0.528734177	0.024328970	0.0024328970
## WT2.9	WT2	0.45	0.473037975	0.026359721	0.0026359721
## WT2.10	WT2	0.50	0.420379747	0.022920436	0.0022920436
## WT2.11	WT2	0.55	0.375822785	0.023089287	0.0023089287
## WT2.12	WT2	0.60	0.321645570	0.023395681	0.0023395681
## WT2.13	WT2	0.65	0.266455696	0.028067795	0.0028067795
## WT2.14	WT2	0.70	0.211392405	0.030717763	0.0030717763
## WT2.15	WT2	0.75	0.164303797	0.028785470	0.0028785470
## WT2.16	WT2	0.80	0.115696203	0.026988632	0.0026988632
## WT2.17	WT2	0.85	0.071139241	0.022599368	0.0022599368
## WT2.18	WT2	0.90	0.033670886	0.019971510	0.0019971510
## WT2.19	WT2	0.95	0.007974684	0.013055398	0.0013055398
## WT2.20	WT2	1.00	0.000000000	0.000000000	0.0000000000
## WT2.21	WT2	0.05	0.941898734	0.009190708	0.0009190708
## WT2.22	WT2	0.10	0.880379747	0.013266915	0.0013266915
## WT2.23	WT2	0.15	0.820886076	0.016817596	0.0016817596
## WT2.24	WT2	0.20	0.761012658	0.018460491	0.0018460491
## WT2.25	WT2	0.25	0.707974684	0.021683823	0.0021683823
## WT2.26	WT2	0.30	0.647468354	0.016721081	0.0016721081
## WT2.27	WT2	0.35	0.590126582	0.024056352	0.0024056352
## WT2.28	WT2	0.40	0.531518987	0.022721858	0.0022721858
## WT2.29	WT2	0.45	0.470253165	0.024029089	0.0024029089
## WT2.30	WT2	0.50	0.418860759	0.026812648	0.0026812648
## WT2.31	WT2	0.55	0.374556962	0.025703158	0.0025703158
## WT2.32	WT2	0.60	0.319367089	0.024790274	0.0024790274
## WT2.33	WT2	0.65	0.262151899	0.028063181	0.0028063181
## WT2.34	WT2	0.70	0.215063291	0.026594453	0.0026594453
## WT2.35	WT2	0.75	0.157088608	0.030057049	0.0030057049
## WT2.36	WT2	0.80	0.113544304	0.025976251	0.0025976251
## WT2.37	WT2	0.85	0.067215190	0.029436789	0.0029436789
## WT2.38	WT2	0.90	0.032911392	0.021961573	0.0021961573
## WT2.39	WT2	0.95	0.007594937	0.012721998	0.0012721998
## WT2.40	WT2	1.00	0.000000000	0.000000000	0.0000000000
## WT3.1	WT3	0.05	0.941810345	0.006863273	0.0006863273
## WT3.2	WT3	0.10	0.882413793	0.009292848	0.0009292848
## WT3.3	WT3	0.15	0.835689655	0.009864928	0.0009864928
## WT3.4	WT3	0.20	0.778275862	0.011301944	0.0011301944
## WT3.5	WT3	0.25	0.719913793	0.013505747	0.0013505747
## WT3.6	WT3	0.30	0.662241379	0.014517507	0.0014517507
## WT3.7	WT3	0.35	0.601724138	0.015106408	0.0015106408
## WT3.8	WT3	0.40	0.557931034	0.015979605	0.0015979605
## WT3.9	WT3	0.45	0.500000000	0.016530086	0.0016530086
## WT3.10	WT3	0.50	0.446637931	0.016008469	0.0016008469
## WT3.11	WT3	0.55	0.390086207	0.019193517	0.0019193517
## WT3.12	WT3	0.60	0.332155172	0.022441741	0.0022441741
## WT3.13	WT3	0.65	0.287672414	0.018649202	0.0018649202

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 185

## WT3.14	WT3	0.70	0.235862069	0.022841747	0.0022841747
## WT3.15	WT3	0.75	0.180086207	0.020204124	0.0020204124
## WT3.16	WT3	0.80	0.130000000	0.018119405	0.0018119405
## WT3.17	WT3	0.85	0.080862069	0.020421606	0.0020421606
## WT3.18	WT3	0.90	0.046810345	0.018266914	0.0018266914
## WT3.19	WT3	0.95	0.012844828	0.010786393	0.0010786393
## WT3.20	WT3	1.00	0.000000000	0.000000000	0.0000000000
## WT3.21	WT3	0.05	0.942758621	0.006987385	0.0006987385
## WT3.22	WT3	0.10	0.884396552	0.009577281	0.0009577281
## WT3.23	WT3	0.15	0.836206897	0.010890629	0.0010890629
## WT3.24	WT3	0.20	0.778879310	0.013609839	0.0013609839
## WT3.25	WT3	0.25	0.719655172	0.012995023	0.0012995023
## WT3.26	WT3	0.30	0.661982759	0.015021444	0.0015021444
## WT3.27	WT3	0.35	0.601551724	0.015135202	0.0015135202
## WT3.28	WT3	0.40	0.555258621	0.017928447	0.0017928447
## WT3.29	WT3	0.45	0.496379310	0.016171062	0.0016171062
## WT3.30	WT3	0.50	0.442931034	0.018694431	0.0018694431
## WT3.31	WT3	0.55	0.386034483	0.018646183	0.0018646183
## WT3.32	WT3	0.60	0.330000000	0.021012230	0.0021012230
## WT3.33	WT3	0.65	0.287241379	0.019407627	0.0019407627
## WT3.34	WT3	0.70	0.231206897	0.023195904	0.0023195904
## WT3.35	WT3	0.75	0.181465517	0.018394682	0.0018394682
## WT3.36	WT3	0.80	0.127758621	0.020807599	0.0020807599
## WT3.37	WT3	0.85	0.080775862	0.017656743	0.0017656743
## WT3.38	WT3	0.90	0.046034483	0.017638240	0.0017638240
## WT3.39	WT3	0.95	0.012327586	0.012748626	0.0012748626
## WT3.40	WT3	1.00	0.000000000	0.000000000	0.0000000000
## WT4.1	WT4	0.05	0.947247706	0.006696797	0.0006696797
## WT4.2	WT4	0.10	0.884678899	0.009952622	0.0009952622
## WT4.3	WT4	0.15	0.835137615	0.010305155	0.0010305155
## WT4.4	WT4	0.20	0.770000000	0.012524658	0.0012524658
## WT4.5	WT4	0.25	0.721376147	0.012346907	0.0012346907
## WT4.6	WT4	0.30	0.660183486	0.015258237	0.0015258237
## WT4.7	WT4	0.35	0.608807339	0.015998085	0.0015998085
## WT4.8	WT4	0.40	0.547706422	0.016260583	0.0016260583
## WT4.9	WT4	0.45	0.499357798	0.018498829	0.0018498829
## WT4.10	WT4	0.50	0.447798165	0.017721418	0.0017721418
## WT4.11	WT4	0.55	0.388807339	0.020525986	0.0020525986
## WT4.12	WT4	0.60	0.338348624	0.018864042	0.0018864042
## WT4.13	WT4	0.65	0.281284404	0.020419676	0.0020419676
## WT4.14	WT4	0.70	0.233119266	0.020744964	0.0020744964
## WT4.15	WT4	0.75	0.180825688	0.022140604	0.0022140604
## WT4.16	WT4	0.80	0.130275229	0.021347071	0.0021347071
## WT4.17	WT4	0.85	0.084495413	0.019133212	0.0019133212
## WT4.18	WT4	0.90	0.045321101	0.016014020	0.0016014020
## WT4.19	WT4	0.95	0.011467890	0.011762954	0.0011762954

##	WT4.20	WT4	1.00	0.000000000	0.000000000	0.000000000
##	WT4.21	WT4	0.05	0.946513761	0.007497034	0.0007497034
##	WT4.22	WT4	0.10	0.885137615	0.010307217	0.0010307217
##	WT4.23	WT4	0.15	0.834128440	0.010073614	0.0010073614
##	WT4.24	WT4	0.20	0.771009174	0.011730022	0.0011730022
##	WT4.25	WT4	0.25	0.718715596	0.014415909	0.0014415909
##	WT4.26	WT4	0.30	0.661651376	0.013817266	0.0013817266
##	WT4.27	WT4	0.35	0.607798165	0.014774443	0.0014774443
##	WT4.28	WT4	0.40	0.549541284	0.015456425	0.0015456425
##	WT4.29	WT4	0.45	0.501559633	0.017738681	0.0017738681
##	WT4.30	WT4	0.50	0.448899083	0.018127431	0.0018127431
##	WT4.31	WT4	0.55	0.386880734	0.017690688	0.0017690688
##	WT4.32	WT4	0.60	0.342568807	0.020750086	0.0020750086
##	WT4.33	WT4	0.65	0.276697248	0.018748311	0.0018748311
##	WT4.34	WT4	0.70	0.236880734	0.021202409	0.0021202409
##	WT4.35	WT4	0.75	0.181559633	0.019221876	0.0019221876
##	WT4.36	WT4	0.80	0.132201835	0.021029885	0.0021029885
##	WT4.37	WT4	0.85	0.082844037	0.020178220	0.0020178220
##	WT4.38	WT4	0.90	0.047706422	0.015917103	0.0015917103
##	WT4.39	WT4	0.95	0.010825688	0.011906986	0.0011906986
##	WT4.40	WT4	1.00	0.000000000	0.000000000	0.000000000
##	WT5.1	WT5	0.05	0.947325581	0.008173024	0.0008173024
##	WT5.2	WT5	0.10	0.882325581	0.011835128	0.0011835128
##	WT5.3	WT5	0.15	0.828139535	0.011869697	0.0011869697
##	WT5.4	WT5	0.20	0.773953488	0.014719407	0.0014719407
##	WT5.5	WT5	0.25	0.706511628	0.015948522	0.0015948522
##	WT5.6	WT5	0.30	0.654186047	0.017303894	0.0017303894
##	WT5.7	WT5	0.35	0.600348837	0.019600924	0.0019600924
##	WT5.8	WT5	0.40	0.547790698	0.019547897	0.0019547897
##	WT5.9	WT5	0.45	0.479069767	0.019555232	0.0019555232
##	WT5.10	WT5	0.50	0.423720930	0.019647204	0.0019647204
##	WT5.11	WT5	0.55	0.374883721	0.021301486	0.0021301486
##	WT5.12	WT5	0.60	0.309302326	0.025603808	0.0025603808
##	WT5.13	WT5	0.65	0.269069767	0.023020895	0.0023020895
##	WT5.14	WT5	0.70	0.217674419	0.023954731	0.0023954731
##	WT5.15	WT5	0.75	0.163139535	0.025998427	0.0025998427
##	WT5.16	WT5	0.80	0.116627907	0.023689577	0.0023689577
##	WT5.17	WT5	0.85	0.075348837	0.025469034	0.0025469034
##	WT5.18	WT5	0.90	0.036976744	0.020401539	0.0020401539
##	WT5.19	WT5	0.95	0.005813953	0.010517838	0.0010517838
##	WT5.20	WT5	1.00	0.000000000	0.000000000	0.000000000
##	WT5.21	WT5	0.05	0.947558140	0.007838662	0.0007838662
##	WT5.22	WT5	0.10	0.880465116	0.010198761	0.0010198761
##	WT5.23	WT5	0.15	0.826627907	0.012704990	0.0012704990
##	WT5.24	WT5	0.20	0.770581395	0.014676198	0.0014676198
##	WT5.25	WT5	0.25	0.709534884	0.015850598	0.0015850598

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 187

##	WT5.26	WT5	0.30	0.652674419	0.015384938	0.0015384938
##	WT5.27	WT5	0.35	0.601395349	0.018994046	0.0018994046
##	WT5.28	WT5	0.40	0.549767442	0.019560818	0.0019560818
##	WT5.29	WT5	0.45	0.477558140	0.020987250	0.0020987250
##	WT5.30	WT5	0.50	0.428255814	0.024848834	0.0024848834
##	WT5.31	WT5	0.55	0.375348837	0.023142784	0.0023142784
##	WT5.32	WT5	0.60	0.314186047	0.021420408	0.0021420408
##	WT5.33	WT5	0.65	0.265116279	0.022540078	0.0022540078
##	WT5.34	WT5	0.70	0.211046512	0.024645744	0.0024645744
##	WT5.35	WT5	0.75	0.155116279	0.025508684	0.0025508684
##	WT5.36	WT5	0.80	0.111046512	0.027024605	0.0027024605
##	WT5.37	WT5	0.85	0.071976744	0.023875637	0.0023875637
##	WT5.38	WT5	0.90	0.041279070	0.023742557	0.0023742557
##	WT5.39	WT5	0.95	0.009069767	0.012649508	0.0012649508
##	WT5.40	WT5	1.00	0.000000000	0.000000000	0.0000000000
##	WT6.1	WT6	0.05	0.948095238	0.005307814	0.0005307814
##	WT6.2	WT6	0.10	0.898285714	0.007879158	0.0007879158
##	WT6.3	WT6	0.15	0.837809524	0.008609823	0.0008609823
##	WT6.4	WT6	0.20	0.784761905	0.010309132	0.0010309132
##	WT6.5	WT6	0.25	0.734285714	0.011365869	0.0011365869
##	WT6.6	WT6	0.30	0.669428571	0.014660469	0.0014660469
##	WT6.7	WT6	0.35	0.616000000	0.013595984	0.0013595984
##	WT6.8	WT6	0.40	0.568761905	0.014394646	0.0014394646
##	WT6.9	WT6	0.45	0.516761905	0.015822141	0.0015822141
##	WT6.10	WT6	0.50	0.460761905	0.014565482	0.0014565482
##	WT6.11	WT6	0.55	0.398571429	0.020231374	0.0020231374
##	WT6.12	WT6	0.60	0.347523810	0.022468028	0.0022468028
##	WT6.13	WT6	0.65	0.293047619	0.020478036	0.0020478036
##	WT6.14	WT6	0.70	0.231904762	0.019304426	0.0019304426
##	WT6.15	WT6	0.75	0.182095238	0.018375840	0.0018375840
##	WT6.16	WT6	0.80	0.131904762	0.022422311	0.0022422311
##	WT6.17	WT6	0.85	0.087333333	0.020576229	0.0020576229
##	WT6.18	WT6	0.90	0.038095238	0.017855940	0.0017855940
##	WT6.19	WT6	0.95	0.007904762	0.011001729	0.0011001729
##	WT6.20	WT6	1.00	0.000000000	0.000000000	0.0000000000
##	WT6.21	WT6	0.05	0.948571429	0.006768277	0.0006768277
##	WT6.22	WT6	0.10	0.898761905	0.006988717	0.0006988717
##	WT6.23	WT6	0.15	0.835142857	0.010192494	0.0010192494
##	WT6.24	WT6	0.20	0.783619048	0.010579317	0.0010579317
##	WT6.25	WT6	0.25	0.734190476	0.009956041	0.0009956041
##	WT6.26	WT6	0.30	0.670095238	0.012221941	0.0012221941
##	WT6.27	WT6	0.35	0.617428571	0.014005918	0.0014005918
##	WT6.28	WT6	0.40	0.569047619	0.014602861	0.0014602861
##	WT6.29	WT6	0.45	0.516000000	0.014790195	0.0014790195
##	WT6.30	WT6	0.50	0.462190476	0.015101298	0.0015101298
##	WT6.31	WT6	0.55	0.397809524	0.015835743	0.0015835743

##	WT6.32	WT6	0.60	0.346380952	0.020780042	0.0020780042
##	WT6.33	WT6	0.65	0.291047619	0.021115288	0.0021115288
##	WT6.34	WT6	0.70	0.235142857	0.019366972	0.0019366972
##	WT6.35	WT6	0.75	0.185714286	0.017519250	0.0017519250
##	WT6.36	WT6	0.80	0.128380952	0.022262590	0.0022262590
##	WT6.37	WT6	0.85	0.088285714	0.019562779	0.0019562779
##	WT6.38	WT6	0.90	0.039142857	0.016181420	0.0016181420
##	WT6.39	WT6	0.95	0.010000000	0.011355789	0.0011355789
##	WT6.40	WT6	1.00	0.000000000	0.000000000	0.0000000000
##		weighted				
##	K01.1	weighted				
##	K01.2	weighted				
##	K01.3	weighted				
##	K01.4	weighted				
##	K01.5	weighted				
##	K01.6	weighted				
##	K01.7	weighted				
##	K01.8	weighted				
##	K01.9	weighted				
##	K01.10	weighted				
##	K01.11	weighted				
##	K01.12	weighted				
##	K01.13	weighted				
##	K01.14	weighted				
##	K01.15	weighted				
##	K01.16	weighted				
##	K01.17	weighted				
##	K01.18	weighted				
##	K01.19	weighted				
##	K01.20	weighted				
##	K01.21	unweighted				
##	K01.22	unweighted				
##	K01.23	unweighted				
##	K01.24	unweighted				
##	K01.25	unweighted				
##	K01.26	unweighted				
##	K01.27	unweighted				
##	K01.28	unweighted				
##	K01.29	unweighted				
##	K01.30	unweighted				
##	K01.31	unweighted				
##	K01.32	unweighted				
##	K01.33	unweighted				
##	K01.34	unweighted				
##	K01.35	unweighted				
##	K01.36	unweighted				

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 189

```
## K01.37 unweighted
## K01.38 unweighted
## K01.39 unweighted
## K01.40 unweighted
## K02.1   weighted
## K02.2   weighted
## K02.3   weighted
## K02.4   weighted
## K02.5   weighted
## K02.6   weighted
## K02.7   weighted
## K02.8   weighted
## K02.9   weighted
## K02.10  weighted
## K02.11  weighted
## K02.12  weighted
## K02.13  weighted
## K02.14  weighted
## K02.15  weighted
## K02.16  weighted
## K02.17  weighted
## K02.18  weighted
## K02.19  weighted
## K02.20  weighted
## K02.21 unweighted
## K02.22 unweighted
## K02.23 unweighted
## K02.24 unweighted
## K02.25 unweighted
## K02.26 unweighted
## K02.27 unweighted
## K02.28 unweighted
## K02.29 unweighted
## K02.30 unweighted
## K02.31 unweighted
## K02.32 unweighted
## K02.33 unweighted
## K02.34 unweighted
## K02.35 unweighted
## K02.36 unweighted
## K02.37 unweighted
## K02.38 unweighted
## K02.39 unweighted
## K02.40 unweighted
## K03.1   weighted
## K03.2   weighted
```

```
## K03.3    weighted
## K03.4    weighted
## K03.5    weighted
## K03.6    weighted
## K03.7    weighted
## K03.8    weighted
## K03.9    weighted
## K03.10   weighted
## K03.11   weighted
## K03.12   weighted
## K03.13   weighted
## K03.14   weighted
## K03.15   weighted
## K03.16   weighted
## K03.17   weighted
## K03.18   weighted
## K03.19   weighted
## K03.20   weighted
## K03.21   unweighted
## K03.22   unweighted
## K03.23   unweighted
## K03.24   unweighted
## K03.25   unweighted
## K03.26   unweighted
## K03.27   unweighted
## K03.28   unweighted
## K03.29   unweighted
## K03.30   unweighted
## K03.31   unweighted
## K03.32   unweighted
## K03.33   unweighted
## K03.34   unweighted
## K03.35   unweighted
## K03.36   unweighted
## K03.37   unweighted
## K03.38   unweighted
## K03.39   unweighted
## K03.40   unweighted
## K04.1    weighted
## K04.2    weighted
## K04.3    weighted
## K04.4    weighted
## K04.5    weighted
## K04.6    weighted
## K04.7    weighted
## K04.8    weighted
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 191

```
## K04.9      weighted
## K04.10     weighted
## K04.11     weighted
## K04.12     weighted
## K04.13     weighted
## K04.14     weighted
## K04.15     weighted
## K04.16     weighted
## K04.17     weighted
## K04.18     weighted
## K04.19     weighted
## K04.20     weighted
## K04.21     unweighted
## K04.22     unweighted
## K04.23     unweighted
## K04.24     unweighted
## K04.25     unweighted
## K04.26     unweighted
## K04.27     unweighted
## K04.28     unweighted
## K04.29     unweighted
## K04.30     unweighted
## K04.31     unweighted
## K04.32     unweighted
## K04.33     unweighted
## K04.34     unweighted
## K04.35     unweighted
## K04.36     unweighted
## K04.37     unweighted
## K04.38     unweighted
## K04.39     unweighted
## K04.40     unweighted
## K05.1      weighted
## K05.2      weighted
## K05.3      weighted
## K05.4      weighted
## K05.5      weighted
## K05.6      weighted
## K05.7      weighted
## K05.8      weighted
## K05.9      weighted
## K05.10     weighted
## K05.11     weighted
## K05.12     weighted
## K05.13     weighted
## K05.14     weighted
```

```
## K05.15    weighted
## K05.16    weighted
## K05.17    weighted
## K05.18    weighted
## K05.19    weighted
## K05.20    weighted
## K05.21    unweighted
## K05.22    unweighted
## K05.23    unweighted
## K05.24    unweighted
## K05.25    unweighted
## K05.26    unweighted
## K05.27    unweighted
## K05.28    unweighted
## K05.29    unweighted
## K05.30    unweighted
## K05.31    unweighted
## K05.32    unweighted
## K05.33    unweighted
## K05.34    unweighted
## K05.35    unweighted
## K05.36    unweighted
## K05.37    unweighted
## K05.38    unweighted
## K05.39    unweighted
## K05.40    unweighted
## K06.1     weighted
## K06.2     weighted
## K06.3     weighted
## K06.4     weighted
## K06.5     weighted
## K06.6     weighted
## K06.7     weighted
## K06.8     weighted
## K06.9     weighted
## K06.10    weighted
## K06.11    weighted
## K06.12    weighted
## K06.13    weighted
## K06.14    weighted
## K06.15    weighted
## K06.16    weighted
## K06.17    weighted
## K06.18    weighted
## K06.19    weighted
## K06.20    weighted
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 193

```
## K06.21 unweighted
## K06.22 unweighted
## K06.23 unweighted
## K06.24 unweighted
## K06.25 unweighted
## K06.26 unweighted
## K06.27 unweighted
## K06.28 unweighted
## K06.29 unweighted
## K06.30 unweighted
## K06.31 unweighted
## K06.32 unweighted
## K06.33 unweighted
## K06.34 unweighted
## K06.35 unweighted
## K06.36 unweighted
## K06.37 unweighted
## K06.38 unweighted
## K06.39 unweighted
## K06.40 unweighted
## OE1.1  weighted
## OE1.2  weighted
## OE1.3  weighted
## OE1.4  weighted
## OE1.5  weighted
## OE1.6  weighted
## OE1.7  weighted
## OE1.8  weighted
## OE1.9  weighted
## OE1.10 weighted
## OE1.11 weighted
## OE1.12 weighted
## OE1.13 weighted
## OE1.14 weighted
## OE1.15 weighted
## OE1.16 weighted
## OE1.17 weighted
## OE1.18 weighted
## OE1.19 weighted
## OE1.20 weighted
## OE1.21 unweighted
## OE1.22 unweighted
## OE1.23 unweighted
## OE1.24 unweighted
## OE1.25 unweighted
## OE1.26 unweighted
```

```
## OE1.27 unweighted
## OE1.28 unweighted
## OE1.29 unweighted
## OE1.30 unweighted
## OE1.31 unweighted
## OE1.32 unweighted
## OE1.33 unweighted
## OE1.34 unweighted
## OE1.35 unweighted
## OE1.36 unweighted
## OE1.37 unweighted
## OE1.38 unweighted
## OE1.39 unweighted
## OE1.40 unweighted
## OE2.1   weighted
## OE2.2   weighted
## OE2.3   weighted
## OE2.4   weighted
## OE2.5   weighted
## OE2.6   weighted
## OE2.7   weighted
## OE2.8   weighted
## OE2.9   weighted
## OE2.10  weighted
## OE2.11  weighted
## OE2.12  weighted
## OE2.13  weighted
## OE2.14  weighted
## OE2.15  weighted
## OE2.16  weighted
## OE2.17  weighted
## OE2.18  weighted
## OE2.19  weighted
## OE2.20  weighted
## OE2.21  unweighted
## OE2.22  unweighted
## OE2.23  unweighted
## OE2.24  unweighted
## OE2.25  unweighted
## OE2.26  unweighted
## OE2.27  unweighted
## OE2.28  unweighted
## OE2.29  unweighted
## OE2.30  unweighted
## OE2.31  unweighted
## OE2.32  unweighted
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 195

```
## OE2.33 unweighted
## OE2.34 unweighted
## OE2.35 unweighted
## OE2.36 unweighted
## OE2.37 unweighted
## OE2.38 unweighted
## OE2.39 unweighted
## OE2.40 unweighted
## OE3.1   weighted
## OE3.2   weighted
## OE3.3   weighted
## OE3.4   weighted
## OE3.5   weighted
## OE3.6   weighted
## OE3.7   weighted
## OE3.8   weighted
## OE3.9   weighted
## OE3.10  weighted
## OE3.11  weighted
## OE3.12  weighted
## OE3.13  weighted
## OE3.14  weighted
## OE3.15  weighted
## OE3.16  weighted
## OE3.17  weighted
## OE3.18  weighted
## OE3.19  weighted
## OE3.20  weighted
## OE3.21 unweighted
## OE3.22 unweighted
## OE3.23 unweighted
## OE3.24 unweighted
## OE3.25 unweighted
## OE3.26 unweighted
## OE3.27 unweighted
## OE3.28 unweighted
## OE3.29 unweighted
## OE3.30 unweighted
## OE3.31 unweighted
## OE3.32 unweighted
## OE3.33 unweighted
## OE3.34 unweighted
## OE3.35 unweighted
## OE3.36 unweighted
## OE3.37 unweighted
## OE3.38 unweighted
```

```
## OE3.39 unweighted
## OE3.40 unweighted
## OE4.1   weighted
## OE4.2   weighted
## OE4.3   weighted
## OE4.4   weighted
## OE4.5   weighted
## OE4.6   weighted
## OE4.7   weighted
## OE4.8   weighted
## OE4.9   weighted
## OE4.10  weighted
## OE4.11  weighted
## OE4.12  weighted
## OE4.13  weighted
## OE4.14  weighted
## OE4.15  weighted
## OE4.16  weighted
## OE4.17  weighted
## OE4.18  weighted
## OE4.19  weighted
## OE4.20  weighted
## OE4.21  unweighted
## OE4.22  unweighted
## OE4.23  unweighted
## OE4.24  unweighted
## OE4.25  unweighted
## OE4.26  unweighted
## OE4.27  unweighted
## OE4.28  unweighted
## OE4.29  unweighted
## OE4.30  unweighted
## OE4.31  unweighted
## OE4.32  unweighted
## OE4.33  unweighted
## OE4.34  unweighted
## OE4.35  unweighted
## OE4.36  unweighted
## OE4.37  unweighted
## OE4.38  unweighted
## OE4.39  unweighted
## OE4.40  unweighted
## OE5.1   weighted
## OE5.2   weighted
## OE5.3   weighted
## OE5.4   weighted
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 197

```
## OE5.5      weighted
## OE5.6      weighted
## OE5.7      weighted
## OE5.8      weighted
## OE5.9      weighted
## OE5.10     weighted
## OE5.11     weighted
## OE5.12     weighted
## OE5.13     weighted
## OE5.14     weighted
## OE5.15     weighted
## OE5.16     weighted
## OE5.17     weighted
## OE5.18     weighted
## OE5.19     weighted
## OE5.20     weighted
## OE5.21     unweighted
## OE5.22     unweighted
## OE5.23     unweighted
## OE5.24     unweighted
## OE5.25     unweighted
## OE5.26     unweighted
## OE5.27     unweighted
## OE5.28     unweighted
## OE5.29     unweighted
## OE5.30     unweighted
## OE5.31     unweighted
## OE5.32     unweighted
## OE5.33     unweighted
## OE5.34     unweighted
## OE5.35     unweighted
## OE5.36     unweighted
## OE5.37     unweighted
## OE5.38     unweighted
## OE5.39     unweighted
## OE5.40     unweighted
## OE6.1      weighted
## OE6.2      weighted
## OE6.3      weighted
## OE6.4      weighted
## OE6.5      weighted
## OE6.6      weighted
## OE6.7      weighted
## OE6.8      weighted
## OE6.9      weighted
## OE6.10     weighted
```

```
## OE6.11    weighted
## OE6.12    weighted
## OE6.13    weighted
## OE6.14    weighted
## OE6.15    weighted
## OE6.16    weighted
## OE6.17    weighted
## OE6.18    weighted
## OE6.19    weighted
## OE6.20    weighted
## OE6.21    unweighted
## OE6.22    unweighted
## OE6.23    unweighted
## OE6.24    unweighted
## OE6.25    unweighted
## OE6.26    unweighted
## OE6.27    unweighted
## OE6.28    unweighted
## OE6.29    unweighted
## OE6.30    unweighted
## OE6.31    unweighted
## OE6.32    unweighted
## OE6.33    unweighted
## OE6.34    unweighted
## OE6.35    unweighted
## OE6.36    unweighted
## OE6.37    unweighted
## OE6.38    unweighted
## OE6.39    unweighted
## OE6.40    unweighted
## WT1.1     weighted
## WT1.2     weighted
## WT1.3     weighted
## WT1.4     weighted
## WT1.5     weighted
## WT1.6     weighted
## WT1.7     weighted
## WT1.8     weighted
## WT1.9     weighted
## WT1.10    weighted
## WT1.11    weighted
## WT1.12    weighted
## WT1.13    weighted
## WT1.14    weighted
## WT1.15    weighted
## WT1.16    weighted
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 199

```
## WT1.17    weighted
## WT1.18    weighted
## WT1.19    weighted
## WT1.20    weighted
## WT1.21    unweighted
## WT1.22    unweighted
## WT1.23    unweighted
## WT1.24    unweighted
## WT1.25    unweighted
## WT1.26    unweighted
## WT1.27    unweighted
## WT1.28    unweighted
## WT1.29    unweighted
## WT1.30    unweighted
## WT1.31    unweighted
## WT1.32    unweighted
## WT1.33    unweighted
## WT1.34    unweighted
## WT1.35    unweighted
## WT1.36    unweighted
## WT1.37    unweighted
## WT1.38    unweighted
## WT1.39    unweighted
## WT1.40    unweighted
## WT2.1     weighted
## WT2.2     weighted
## WT2.3     weighted
## WT2.4     weighted
## WT2.5     weighted
## WT2.6     weighted
## WT2.7     weighted
## WT2.8     weighted
## WT2.9     weighted
## WT2.10    weighted
## WT2.11    weighted
## WT2.12    weighted
## WT2.13    weighted
## WT2.14    weighted
## WT2.15    weighted
## WT2.16    weighted
## WT2.17    weighted
## WT2.18    weighted
## WT2.19    weighted
## WT2.20    weighted
## WT2.21    unweighted
## WT2.22    unweighted
```

```
## WT2.23 unweighted
## WT2.24 unweighted
## WT2.25 unweighted
## WT2.26 unweighted
## WT2.27 unweighted
## WT2.28 unweighted
## WT2.29 unweighted
## WT2.30 unweighted
## WT2.31 unweighted
## WT2.32 unweighted
## WT2.33 unweighted
## WT2.34 unweighted
## WT2.35 unweighted
## WT2.36 unweighted
## WT2.37 unweighted
## WT2.38 unweighted
## WT2.39 unweighted
## WT2.40 unweighted
## WT3.1   weighted
## WT3.2   weighted
## WT3.3   weighted
## WT3.4   weighted
## WT3.5   weighted
## WT3.6   weighted
## WT3.7   weighted
## WT3.8   weighted
## WT3.9   weighted
## WT3.10  weighted
## WT3.11  weighted
## WT3.12  weighted
## WT3.13  weighted
## WT3.14  weighted
## WT3.15  weighted
## WT3.16  weighted
## WT3.17  weighted
## WT3.18  weighted
## WT3.19  weighted
## WT3.20  weighted
## WT3.21 unweighted
## WT3.22 unweighted
## WT3.23 unweighted
## WT3.24 unweighted
## WT3.25 unweighted
## WT3.26 unweighted
## WT3.27 unweighted
## WT3.28 unweighted
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 201

```
## WT3.29 unweighted
## WT3.30 unweighted
## WT3.31 unweighted
## WT3.32 unweighted
## WT3.33 unweighted
## WT3.34 unweighted
## WT3.35 unweighted
## WT3.36 unweighted
## WT3.37 unweighted
## WT3.38 unweighted
## WT3.39 unweighted
## WT3.40 unweighted
## WT4.1   weighted
## WT4.2   weighted
## WT4.3   weighted
## WT4.4   weighted
## WT4.5   weighted
## WT4.6   weighted
## WT4.7   weighted
## WT4.8   weighted
## WT4.9   weighted
## WT4.10  weighted
## WT4.11  weighted
## WT4.12  weighted
## WT4.13  weighted
## WT4.14  weighted
## WT4.15  weighted
## WT4.16  weighted
## WT4.17  weighted
## WT4.18  weighted
## WT4.19  weighted
## WT4.20  weighted
## WT4.21 unweighted
## WT4.22 unweighted
## WT4.23 unweighted
## WT4.24 unweighted
## WT4.25 unweighted
## WT4.26 unweighted
## WT4.27 unweighted
## WT4.28 unweighted
## WT4.29 unweighted
## WT4.30 unweighted
## WT4.31 unweighted
## WT4.32 unweighted
## WT4.33 unweighted
## WT4.34 unweighted
```

```
## WT4.35 unweighted
## WT4.36 unweighted
## WT4.37 unweighted
## WT4.38 unweighted
## WT4.39 unweighted
## WT4.40 unweighted
## WT5.1   weighted
## WT5.2   weighted
## WT5.3   weighted
## WT5.4   weighted
## WT5.5   weighted
## WT5.6   weighted
## WT5.7   weighted
## WT5.8   weighted
## WT5.9   weighted
## WT5.10  weighted
## WT5.11  weighted
## WT5.12  weighted
## WT5.13  weighted
## WT5.14  weighted
## WT5.15  weighted
## WT5.16  weighted
## WT5.17  weighted
## WT5.18  weighted
## WT5.19  weighted
## WT5.20  weighted
## WT5.21 unweighted
## WT5.22 unweighted
## WT5.23 unweighted
## WT5.24 unweighted
## WT5.25 unweighted
## WT5.26 unweighted
## WT5.27 unweighted
## WT5.28 unweighted
## WT5.29 unweighted
## WT5.30 unweighted
## WT5.31 unweighted
## WT5.32 unweighted
## WT5.33 unweighted
## WT5.34 unweighted
## WT5.35 unweighted
## WT5.36 unweighted
## WT5.37 unweighted
## WT5.38 unweighted
## WT5.39 unweighted
## WT5.40 unweighted
```

7.3. GET NETWORK TOPOLOGY INFORMATION FROM SAMPLE 203

```
## WT6.1      weighted
## WT6.2      weighted
## WT6.3      weighted
## WT6.4      weighted
## WT6.5      weighted
## WT6.6      weighted
## WT6.7      weighted
## WT6.8      weighted
## WT6.9      weighted
## WT6.10     weighted
## WT6.11     weighted
## WT6.12     weighted
## WT6.13     weighted
## WT6.14     weighted
## WT6.15     weighted
## WT6.16     weighted
## WT6.17     weighted
## WT6.18     weighted
## WT6.19     weighted
## WT6.20     weighted
## WT6.21     unweighted
## WT6.22     unweighted
## WT6.23     unweighted
## WT6.24     unweighted
## WT6.25     unweighted
## WT6.26     unweighted
## WT6.27     unweighted
## WT6.28     unweighted
## WT6.29     unweighted
## WT6.30     unweighted
## WT6.31     unweighted
## WT6.32     unweighted
## WT6.33     unweighted
## WT6.34     unweighted
## WT6.35     unweighted
## WT6.36     unweighted
## WT6.37     unweighted
## WT6.38     unweighted
## WT6.39     unweighted
## WT6.40     unweighted
##
## $sample_stat
##      Sample Node Edge Status
## K01      K01  113  161      OK
## K02      K02  122  164      OK
## K03      K03  102  379      OK
```

```
## K04      K04      94  107      OK
## K05      K05      71   63      OK
## K06      K06      97  119      OK
## OE1      OE1     138  308      OK
## OE2      OE2     135  230      OK
## OE3      OE3     119  160      OK
## OE4      OE4     118  171      OK
## OE5      OE5     111  165      OK
## OE6      OE6     140  234      OK
## WT1      WT1     105  130      OK
## WT2      WT2      83   59      OK
## WT3      WT3     117  171      OK
## WT4      WT4     111  222      OK
## WT5      WT5      90  103      OK
## WT6      WT6     111  162      OK
```

7.4 Get network topology information by parallel

```
# ---- Same as get_network_topology(), but with parallel execution -----
# Useful when `bootstrap` is large (the random-ER-network reference loop is
# the main bottleneck) or when running on `graph_obj_list` of many networks.
# With `mat = NULL`, cohesion / robustness / stability fields are NA - see
# get_network_topology() docs for the full list of skipped metrics.
t4 <- system.time(
  topology_parallel <- get_network_topology_parallel(
    graph_obj      = graph_obj,
    mat            = NULL,                # no abundance matrix -> cohesion + robustness
    transform.method = "none",           # ignored when mat = NULL
    method         = "WGCNA",           # ignored when mat = NULL (no correlation rebuild)
    cor.method     = "pearson",         # ignored when mat = NULL
    proc           = "bonferroni",      # ignored when mat = NULL
    bootstrap      = 100,               # # of random ER networks for the Random_network
    parallel       = TRUE,              # enable parallel backend
    n_workers      = 2,                 # # of parallel workers (keep <= cores; 2 is safe)
  )
)
```

```
## Number of parallel workers: 2
```

```
## Warning: package 'future' was built under R version 4.5.2
## Warning: package 'future' was built under R version 4.5.2
```

```
topology_parallel
```

```
## $topology
## # A tibble: 24 x 3
##   Topology      Target_network Random_nerwork
##   <chr>          <dbl>          <dbl>
## 1 Node           213            213
## 2 Edge           844            844
## 3 Degree          7.92            7.92
## 4 Distance        1.45            2.81
## 5 Diameter        3.83            5.08
## 6 Density         0.0374          0.0374
## 7 Transitivity_global 0.851          0.0369
## 8 Transitivity_local 0.783          0.0369
## 9 Betweenness     3.47           191.
## 10 Betweenness_edge 2.58           75.1
## # i 14 more rows
##
## $Robustness
##   Proportion.removed remain.mean remain.sd remain.se weighted
## 1          0.05          NA          NA          NA weighted
## 2          0.10          NA          NA          NA weighted
## 3          0.15          NA          NA          NA weighted
## 4          0.20          NA          NA          NA weighted
## 5          0.25          NA          NA          NA weighted
## 6          0.30          NA          NA          NA weighted
## 7          0.35          NA          NA          NA weighted
## 8          0.40          NA          NA          NA weighted
## 9          0.45          NA          NA          NA weighted
## 10         0.50          NA          NA          NA weighted
## 11         0.55          NA          NA          NA weighted
## 12         0.60          NA          NA          NA weighted
## 13         0.65          NA          NA          NA weighted
## 14         0.70          NA          NA          NA weighted
## 15         0.75          NA          NA          NA weighted
## 16         0.80          NA          NA          NA weighted
## 17         0.85          NA          NA          NA weighted
## 18         0.90          NA          NA          NA weighted
## 19         0.95          NA          NA          NA weighted
## 20         1.00          NA          NA          NA weighted
## 21         0.05          NA          NA          NA unweighted
## 22         0.10          NA          NA          NA unweighted
## 23         0.15          NA          NA          NA unweighted
## 24         0.20          NA          NA          NA unweighted
## 25         0.25          NA          NA          NA unweighted
```

## 26	0.30	NA	NA	NA unweighted
## 27	0.35	NA	NA	NA unweighted
## 28	0.40	NA	NA	NA unweighted
## 29	0.45	NA	NA	NA unweighted
## 30	0.50	NA	NA	NA unweighted
## 31	0.55	NA	NA	NA unweighted
## 32	0.60	NA	NA	NA unweighted
## 33	0.65	NA	NA	NA unweighted
## 34	0.70	NA	NA	NA unweighted
## 35	0.75	NA	NA	NA unweighted
## 36	0.80	NA	NA	NA unweighted
## 37	0.85	NA	NA	NA unweighted
## 38	0.90	NA	NA	NA unweighted
## 39	0.95	NA	NA	NA unweighted
## 40	1.00	NA	NA	NA unweighted

7.5 Get network topology information with matrix by parallel

```
# ---- Parallel + abundance matrix -> FULL topology panel -----
# Same as above, but `mat` is now provided, so every topology field gets a
# real value (cohesion, robustness, stability included).
t5 <- system.time(
  topology_parallel_with_mat <- get_network_topology_parallel(
    graph_obj      = graph_obj,
    mat            = otu_rare_relative, # abundance matrix -> enables cohesion + rob
    transform.method = "none",          # pre-correlation transform on `mat`
    method         = "WGCNA",          # correlation backend used to rebuild network
    cor.method     = "pearson",         # Pearson correlation
    proc          = "bonferroni",      # multiple-testing correction
    bootstrap     = 100,                # bootstrap iterations for robustness + # of
    parallel      = TRUE,              # enable parallel backend
    n_workers     = 2                  # # of parallel workers
  )
)
```

```
## Number of parallel workers: 2
```

```
## Warning: package 'future' was built under R version 4.5.2
```

```
## Warning: package 'future' was built under R version 4.5.2
```

7.5. GET NETWORK TOPOLOGY INFORMATION WITH MATRIX BY PARALLEL207

```
topology_parallel_with_mat
```

```
## $topology
## # A tibble: 24 x 3
##   Topology Target_network Random_network
##   <chr>      <dbl>      <dbl>
## 1 Node      213        213
## 2 Edge      844        844
## 3 Degree    7.92       7.92
## 4 Distance  1.45       2.81
## 5 Diameter  3.83       5.07
## 6 Density   0.0374     0.0374
## 7 Transitivity_global 0.851     0.0369
## 8 Transitivity_local  0.783     0.0372
## 9 Betweenness 3.47      191.
## 10 Betweenness_edge 2.58      75.0
## # i 14 more rows
##
## $Robustness
##   Proportion.removed remain.mean remain.sd remain.se weighted
## 1 0.05 0.93765258 0.006506724 0.0006506724 weighted
## 2 0.10 0.88042254 0.009907812 0.0009907812 weighted
## 3 0.15 0.81859155 0.012057928 0.0012057928 weighted
## 4 0.20 0.75816901 0.013453813 0.0013453813 weighted
## 5 0.25 0.70455399 0.012630308 0.0012630308 weighted
## 6 0.30 0.64333333 0.016021401 0.0016021401 weighted
## 7 0.35 0.58652582 0.016937398 0.0016937398 weighted
## 8 0.40 0.53028169 0.016859929 0.0016859929 weighted
## 9 0.45 0.47309859 0.018089169 0.0018089169 weighted
## 10 0.50 0.42140845 0.015919144 0.0015919144 weighted
## 11 0.55 0.37046948 0.018756744 0.0018756744 weighted
## 12 0.60 0.31450704 0.015513637 0.0015513637 weighted
## 13 0.65 0.26737089 0.019720511 0.0019720511 weighted
## 14 0.70 0.21652582 0.017859085 0.0017859085 weighted
## 15 0.75 0.17023474 0.016271873 0.0016271873 weighted
## 16 0.80 0.12455399 0.016887373 0.0016887373 weighted
## 17 0.85 0.08291080 0.015878534 0.0015878534 weighted
## 18 0.90 0.04262911 0.013423744 0.0013423744 weighted
## 19 0.95 0.01413146 0.009998184 0.0009998184 weighted
## 20 1.00 0.00000000 0.000000000 0.0000000000 weighted
## 21 0.05 0.93666667 0.006347900 0.0006347900 unweighted
## 22 0.10 0.88004695 0.009586197 0.0009586197 unweighted
## 23 0.15 0.81882629 0.011087843 0.0011087843 unweighted
## 24 0.20 0.75755869 0.014084823 0.0014084823 unweighted
## 25 0.25 0.70258216 0.013923536 0.0013923536 unweighted
```

```
## 26      0.30  0.64220657 0.014112853 0.0014112853 unweighted
## 27      0.35  0.58239437 0.013335467 0.0013335467 unweighted
## 28      0.40  0.53103286 0.016600696 0.0016600696 unweighted
## 29      0.45  0.47755869 0.015290812 0.0015290812 unweighted
## 30      0.50  0.42812207 0.015401290 0.0015401290 unweighted
## 31      0.55  0.36694836 0.016218423 0.0016218423 unweighted
## 32      0.60  0.31892019 0.018094092 0.0018094092 unweighted
## 33      0.65  0.26699531 0.017332399 0.0017332399 unweighted
## 34      0.70  0.21441315 0.017260315 0.0017260315 unweighted
## 35      0.75  0.16938967 0.018751817 0.0018751817 unweighted
## 36      0.80  0.12624413 0.017219505 0.0017219505 unweighted
## 37      0.85  0.07910798 0.015222150 0.0015222150 unweighted
## 38      0.90  0.04286385 0.012864348 0.0012864348 unweighted
## 39      0.95  0.01413146 0.009519073 0.0009519073 unweighted
## 40      1.00  0.00000000 0.000000000 0.0000000000 unweighted
```

7.6 Integrated Value of Influence (IVI)

```
# ---- Compute IVI (Salavaty et al. 2020) per node -----
# IVI integrates 6 centrality measures across 3 scales (local / semi-local /
# global) into a single robust influence score. Thin wrapper over
# influential::ivi() - that package must be installed.
obj_aug <- get_node_ivi(
  graph_obj = graph_obj, # input tbl_graph
  weights   = NULL,      # NULL = unweighted IVI; pass "weight" to use the graph's `weight`
  mode      = "all",     # edge mode; "all" is the right choice for undirected graphs
  directed  = FALSE,     # treat the graph as undirected
  d         = 3,         # search radius for the collective-influence component (paper)
  scale     = "z-scale", # "range" [1,100] for single-network view, "z-scale" for cluster-rank
  ncores    = 1          # parallel cores for the cluster-rank step; 1 is the safe default
)

obj_aug # tbl_graph with one extra node column: IVI

## # A tbl_graph: 213 nodes and 844 edges
## #
## # An undirected simple graph with 29 components
## #
## # Node Data: 213 x 15 (active)
##   name      modularity modularity2 modularity3 Modularity Degree Strength Kingdom
##   <chr>     <fct>      <ord>      <chr>      <ord>      <dbl>    <dbl> <chr>
## 1 ASV_649 5          5          5          5          27      26.5 Bacter~
```

```
## 2 ASV_705 5 5 5 5 27 26.5 Bacter~
## 3 ASV_12~ 5 5 5 5 27 26.5 Bacter~
## 4 ASV_13~ 5 5 5 5 27 26.5 Bacter~
## 5 ASV_14~ 5 5 5 5 27 26.5 Bacter~
## 6 ASV_14~ 5 5 5 5 27 26.5 Bacter~
## 7 ASV_24~ 5 5 5 5 27 26.5 Bacter~
## 8 ASV_25~ 5 5 5 5 27 26.4 Bacter~
## 9 ASV_28~ 5 5 5 5 27 26.5 Bacter~
## 10 ASV_28~ 5 5 5 5 27 26.5 Bacter~
## # i 203 more rows
## # i 7 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## # Genus <chr>, Species <chr>, IVI <dbl>
## #
## # Edge Data: 844 x 5
## from to weight correlation corr_direction
## <int> <int> <dbl> <dbl> <chr>
## 1 194 195 0.959 0.959 Positive
## 2 185 208 0.954 0.954 Positive
## 3 185 213 0.957 0.957 Positive
## # i 841 more rows
```

```
class(obj_aug) # still a tbl_graph - all other vertex/edge columns are preserved
```

```
## [1] "tbl_graph" "igraph"
```

```
# ---- Top-10 most influential nodes -----
# Activate the node table, convert to tibble, sort by IVI descending, take 10.
obj_aug %>%
  tidygraph::activate(nodes) %>%
  tidygraph::as_tibble() %>%
  dplyr::arrange(desc(IVI)) %>%
  dplyr::slice_head(n = 10)
```

```
## # A tibble: 10 x 15
## name modularity modularity2 modularity3 Modularity Degree Strength Kingdom
## <chr> <fct> <ord> <chr> <ord> <dbl> <dbl> <chr>
## 1 ASV_913 5 5 5 5 26 25.2 Bacter~
## 2 ASV_25~ 5 5 5 5 27 26.4 Bacter~
## 3 ASV_11~ 5 5 5 5 20 19.1 Bacter~
## 4 ASV_28~ 5 5 5 5 23 22.5 Bacter~
## 5 ASV_26~ 5 5 5 5 21 20.4 Bacter~
## 6 ASV_24~ 5 5 5 5 23 22.6 Bacter~
## 7 ASV_996 5 5 5 5 19 18.3 Bacter~
## 8 ASV_28~ 5 5 5 5 20 19.1 Bacter~
```

```

## 9 ASV_767 5          5          5          5          22      21.1 Bacter~
## 10 ASV_802 8         8          8          8          18      17.5 Bacter~
## # i 7 more variables: Phylum <chr>, Class <chr>, Order <chr>, Family <chr>,
## #   Genus <chr>, Species <chr>, IVI <dbl>

```

Chapter 8

Multi-Network and comparison

Load R Package

```
library(tidyverse) # data manipulation / piping utilities
library(ggNetView) # graph builders + info-extraction helpers
```

Example data

```
# Access built-in example datasets in ggNetView

# ---- Dataset 1: Relative abundance table of rarefied ASVs/OTUs ----
# Rows = ASVs/OTUs (features / variables), Columns = samples
data("otu_rare_relative")
dim(otu_rare_relative) # Check matrix dimensions: n_features x n_samples

## [1] 2859    18

otu_rare_relative[1:5, 1:5] # Preview first 5 rows x 5 cols to confirm data layout

##           K01      K02      K03      K04      K05
## ASV_1 0.03306667 0.05453333 0.02013333 0.03613333 0.02686667
## ASV_2 0.05750000 0.03393333 0.06046667 0.05810000 0.07320000
## ASV_3 0.01733333 0.01296667 0.02290000 0.02336667 0.03106667
## ASV_4 0.04266667 0.01093333 0.01416667 0.01933333 0.03346667
## ASV_6 0.02646667 0.01856667 0.02110000 0.02353333 0.03806667
```

```
# ---- Dataset 2: Taxonomic annotation table for ASVs/OTUs ----
# Rows = ASVs/OTUs (row names should match the abundance table above)
# Columns = taxonomic ranks (Kingdom ~ Species, etc.)
data("tax_tab")
dim(tax_tab)
```

```
## [1] 2859    8
```

```
tax_tab[1:5, 1:5]
```

```
## # A tibble: 5 x 5
##   OTUID Kingdom Phylum      Class      Order
##   <chr>  <chr>    <chr>      <chr>      <chr>
## 1 ASV_2  Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 2 ASV_3  Bacteria Verrucomicrobia Spartobacteria Unassigned
## 3 ASV_31 Bacteria Actinobacteria Actinobacteria Actinomycetales
## 4 ASV_27 Archaea  Thaumarchaeota  Unassigned  Nitrososphaerales
## 5 ASV_9  Bacteria Unassigned      Unassigned  Unassigned
```

```
# ---- Dataset 3: Sample metadata ----
# Rows = samples (sample IDs should match colnames of `otu_rare_relative`)
# Columns = experimental grouping variables (e.g. treatment, time point, site)
data("otu_sample")
otu_sample # print the full sample-info table (small enough to view)
```

```
##   Sample Group
## 1      K01    KO
## 2      K02    KO
## 3      K03    KO
## 4      K04    KO
## 5      K05    KO
## 6      K06    KO
## 7      OE1    OE
## 8      OE2    OE
## 9      OE3    OE
## 10     OE4    OE
## 11     OE5    OE
## 12     OE6    OE
## 13     WT1    WT
## 14     WT2    WT
## 15     WT3    WT
## 16     WT4    WT
## 17     WT5    WT
## 18     WT6    WT
```

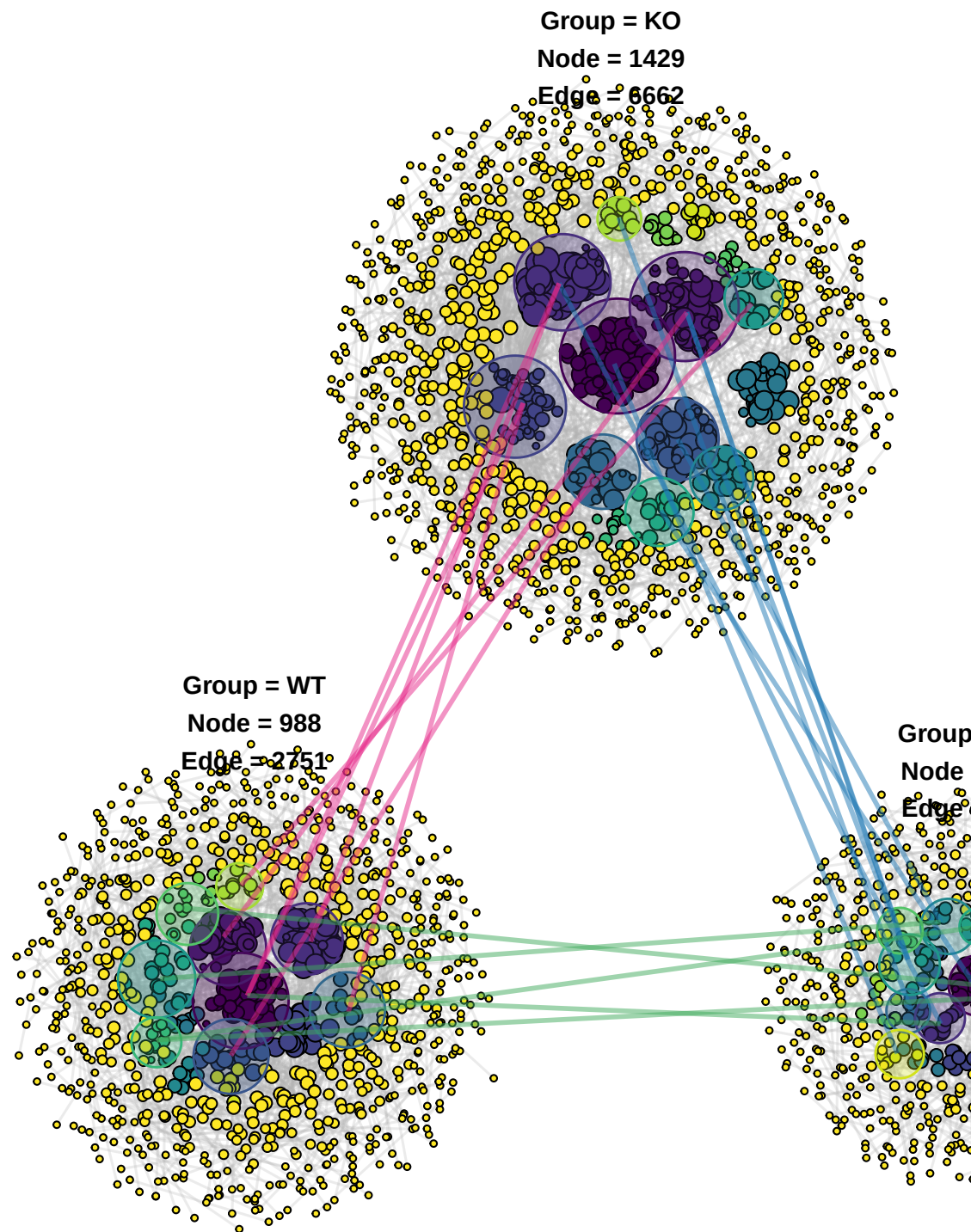
8.1 Comparison of multi-sample networks

8.1.1 layout = “gephi”, scale = FALSE

```
# ---- Multi-network "linked" view (no per-group scale normalisation) ----
# `ggNetView_multi_link()` builds ONE network per group (defined by `group_info`),
# lays each one out independently, and places all sub-networks on a circular
# anchor frame so that cross-group edges/relationships can be visually compared.
out1 <- ggNetView_multi_link(
  mat           = otu_rare_relative, # variables x samples abundance matrix
  group_info    = otu_sample,        # sample metadata; defines how samples are split into groups
  transform.method = "none",         # pre-correlation transform on `mat`; "none" = keep as-is
  r.threshold   = 0.7,               # |r| cutoff for keeping an edge
  p.threshold   = 0.05,              # adjusted p-value cutoff
  method        = "WGCNA",          # correlation backend: WGCNA::corAndPvalue
  cor.method     = "pearson",        # Pearson correlation
  proc          = "BH",              # multiple-testing correction (Benjamini-Hochberg)
  module.method  = "Fast_greedy",    # community detection per sub-network
  layout.module  = "adjacent",       # neighbouring modules placed close together
  center        = T,                # pull a node toward the centre of each sub-layout
  top_modules    = 15,              # keep top-15 modules per sub-network; rest -> "Others"
  layout         = "gephi",         # global node-positioning algorithm per sub-network
  shrink        = 0.5,              # shrink each sub-network inward (smaller = more compact)
  scale         = F,                # do NOT normalise each group to a common scale -> sub-networks
  jitter        = T,                # jitter node positions slightly to reduce overlap
  jitter_sd     = 0.3,              # SD of the jitter (bigger -> more spread)
  anchor_dist    = 30,              # distance between groups on the outer anchor circle
  seed          = 1115,             # fix RNG for reproducibility
  orientation    = "up",            # frame orientation; one of "up"/"down"/"left"/"right"
  angle         = 0,                # additional rotation angle (degrees)
)
```

```
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
```

```
p1 <- out1[["p"]] # the assembled ggplot object
p1
```



```
out1$info           # auxiliary table: per-group node/edge counts, module info, etc.
```

```
## # A tibble: 22 x 11
##   modA  modB  overlap sizeA sizeB overlap_coef pvalue  FDR Group  GroupA
##   <chr> <chr>   <int> <int> <int>      <dbl>  <dbl> <dbl> <chr>  <chr>
## 1 15     1       2     6    14      0.333 0.0140 0.776 KO_to_OE KO
## 2 16     3       2    19     8      0.25  0.0450 1     KO_to_OE KO
## 3 52     6       1     4     3      0.333 0.0280 0.895 KO_to_OE KO
## 4 4      8       2    16     6      0.333 0.0182 0.776 KO_to_OE KO
## 5 3      9       2    22     3      0.667 0.00741 0.633 KO_to_OE KO
## 6 7      9       1     4     3      0.333 0.0280 0.895 KO_to_OE KO
## 7 3     12       2    22     3      0.667 0.00741 0.633 KO_to_OE KO
## 8 1     15       2    32     3      0.667 0.0157 0.776 KO_to_OE KO
## 9 Others Others  224  263  342      0.852 0.00109 0.279 KO_to_OE KO
## 10 16     1       4     29    28      0.143 0.0415 1     KO_to_WT KO
## # i 12 more rows
## # i 1 more variable: GroupB <chr>
```

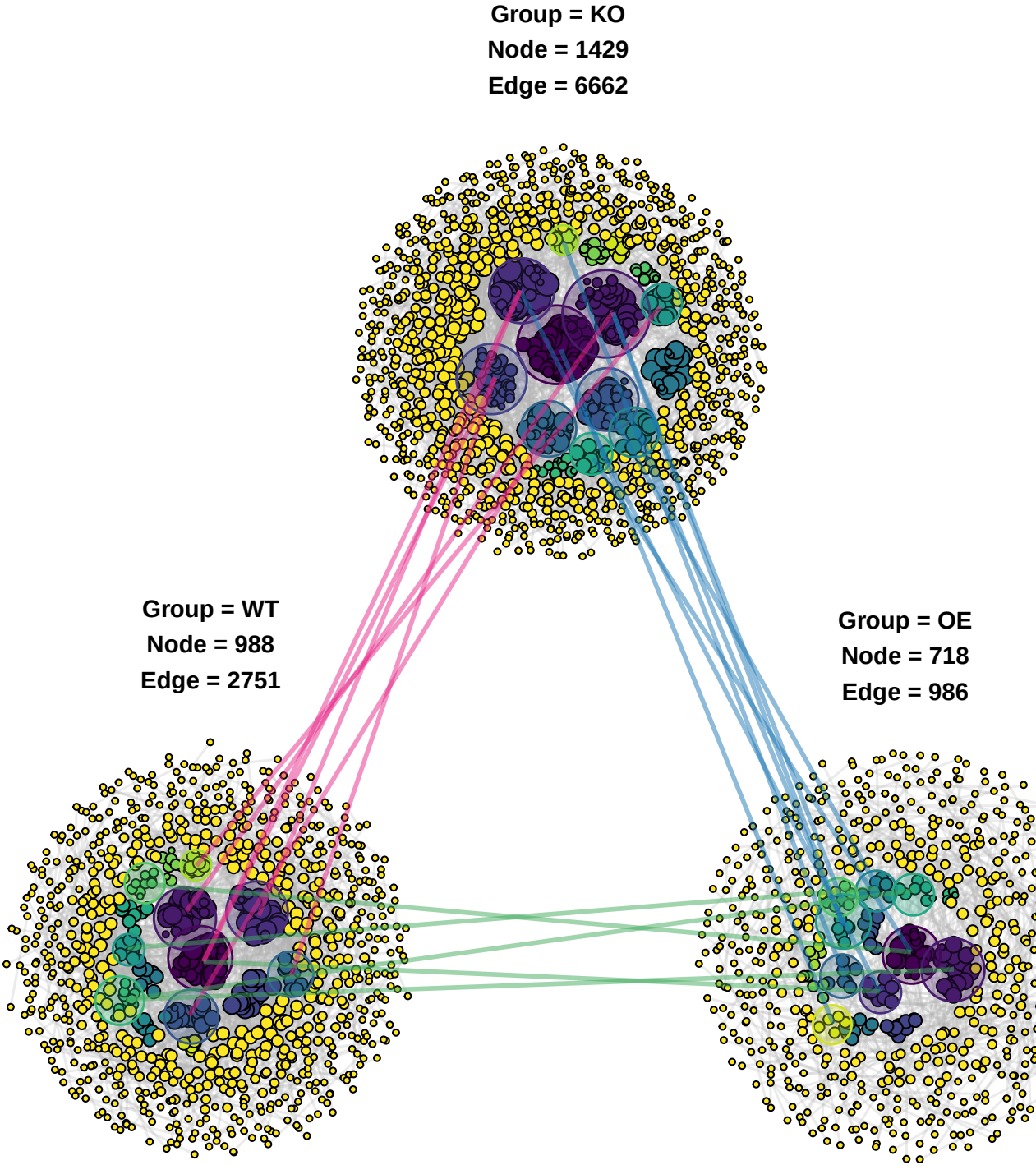
8.1.2 layout = “gephi”, scale = TRUE

```
####----Plot2----####
# ---- Same setup but with `scale = TRUE` -> each group normalised to a common size ----
# Useful when groups have very different network sizes and you want them
# visually comparable side-by-side. With `scale = TRUE` the native size
# differences are removed, so jitter_sd is also reduced (0.01) accordingly.
out2 <- ggNetView_multi_link(
  mat           = otu_rare_relative,
  group_info    = otu_sample,
  transfrom.method = "none",
  r.threshold   = 0.7,
  p.threshold   = 0.05,
  method        = "WGCNA",
  cor.method     = "pearson",
  proc          = "BH",
  module.method  = "Fast_greedy",
  layout.module  = "adjacent",
  center        = T,
  top_modules    = 15,
  layout         = "gephi",
  shrink         = 0.5,
  scale          = T,                               # normalise each group to a comparable coordinate scale
  jitter         = T,
  jitter_sd      = 0.01,                             # smaller jitter - sub-networks are already on a unified
```

```
    anchor_dist = 1,  
    seed        = 1115,  
    orientation  = "up",  
    angle       = 0  
  )
```

```
## Scale for size is already present.  
## Adding another scale for size, which will replace the existing scale.  
## Scale for size is already present.  
## Adding another scale for size, which will replace the existing scale.
```

```
p2 <- out2[["p"]]  
p2
```

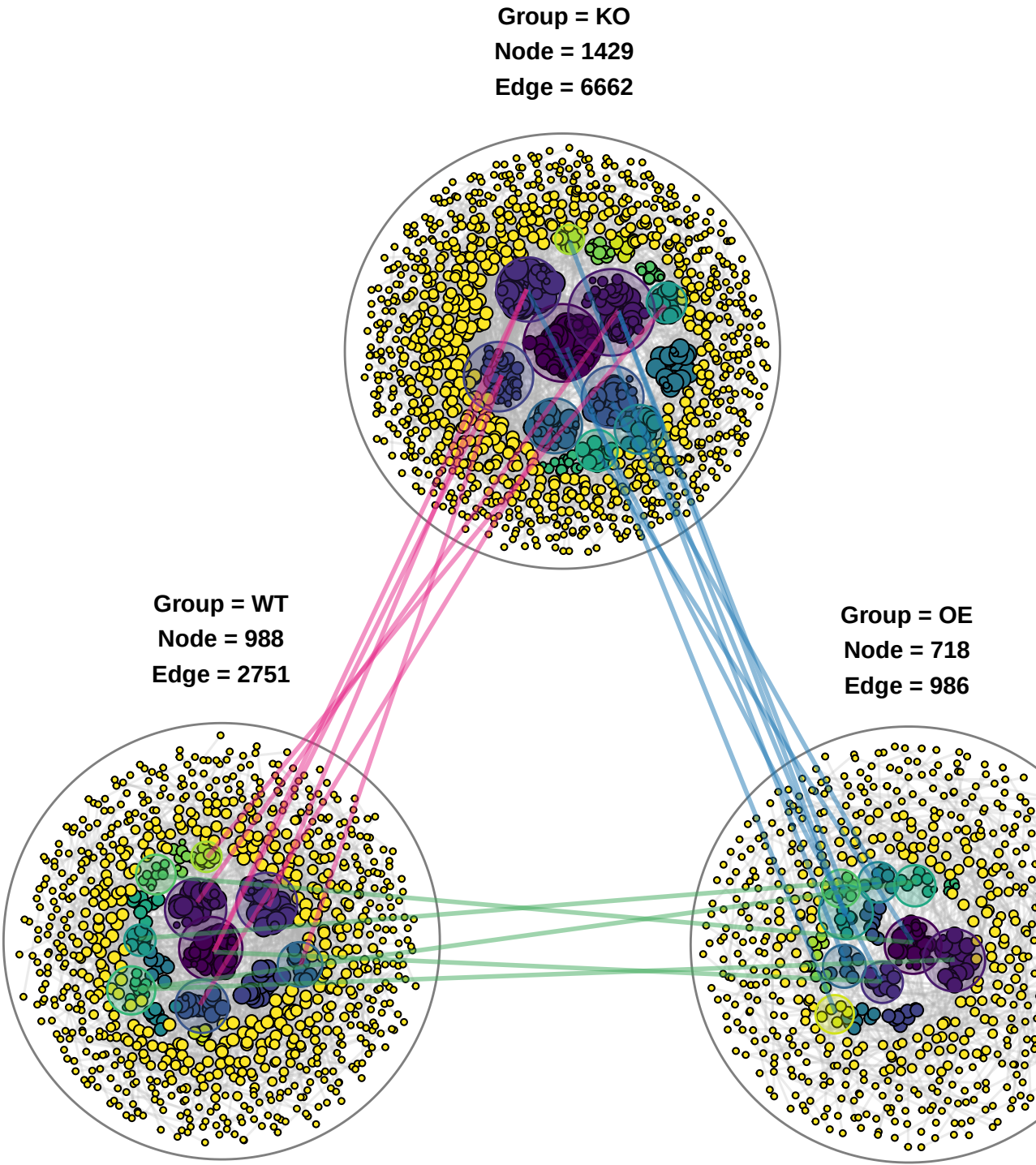


```
add_group_outer = TRUE
```

```
####----Plot2----####
# ---- Same setup but with `scale = TRUE` -> each group normalised to a common size --
# Useful when groups have very different network sizes and you want them
# visually comparable side-by-side. With `scale = TRUE` the native size
# differences are removed, so jitter_sd is also reduced (0.01) accordingly.
out3 <- ggNetView_multi_link(
  mat           = otu_rare_relative,
  group_info    = otu_sample,
  transfrom.method = "none",
  r.threshold   = 0.7,
  p.threshold   = 0.05,
  method        = "WGCNA",
  cor.method    = "pearson",
  proc          = "BH",
  module.method = "Fast_greedy",
  layout.module = "adjacent",
  center        = T,
  top_modules   = 15,
  layout        = "gephi",
  shrink        = 0.5,
  scale         = T,                # normalise each group to a comparable coordin
  jitter        = T,
  jitter_sd     = 0.01,            # smaller jitter - sub-networks are already on
  anchor_dist   = 1,
  seed          = 1115,
  orientation   = "up",
  angle         = 0,
  add_group_outer = T
)
```

```
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
```

```
p3 <- out3[["p"]]
p3
```



8.1.3 layout = “circular_modules_equal_gephi_layout”

```

# ---- Circular-modules layout: nodes within each module on equal arcs ----
# Better than "gephi" when you want module structure to dominate the picture.
# `link_level = "Node"`: cross-group links are drawn between individual NODES.
out_multi_link <- ggNetView_multi_link(
  mat           = otu_rare_relative,
  group_info    = otu_sample,
  transform.method = "none",
  r.threshold   = 0.7,
  p.threshold   = 0.05,
  method        = "WGCNA",
  cor.method     = "pearson",
  proc          = "BH",
  module.method  = "Fast_greedy",
  layout        = "circular_modules_equal_gephi_layout", # per-sub-network layout
  layout.module  = "order",          # modules placed in a fixed order (not random)
  top_modules    = 15,
  scale_groups   = T,                # normalise each group to a comparable scale
  add_outer      = "circle",         # outer frame style; "circle" / "manual" / etc
  r              = 5,                # radius of each per-group sub-network
  jitter         = F,                # don't jitter nodes
  anchor_dist    = 1,                # distance between groups on the anchor frame
  layout_anchor_dist = 150,          # anchor frame size in the per-sub-network layout
  seed           = 1115,
  dropOthers     = T,                # drop the "Others" module before plotting
  link_level     = "Node"            # connect cross-group items at NODE level
)

```

```

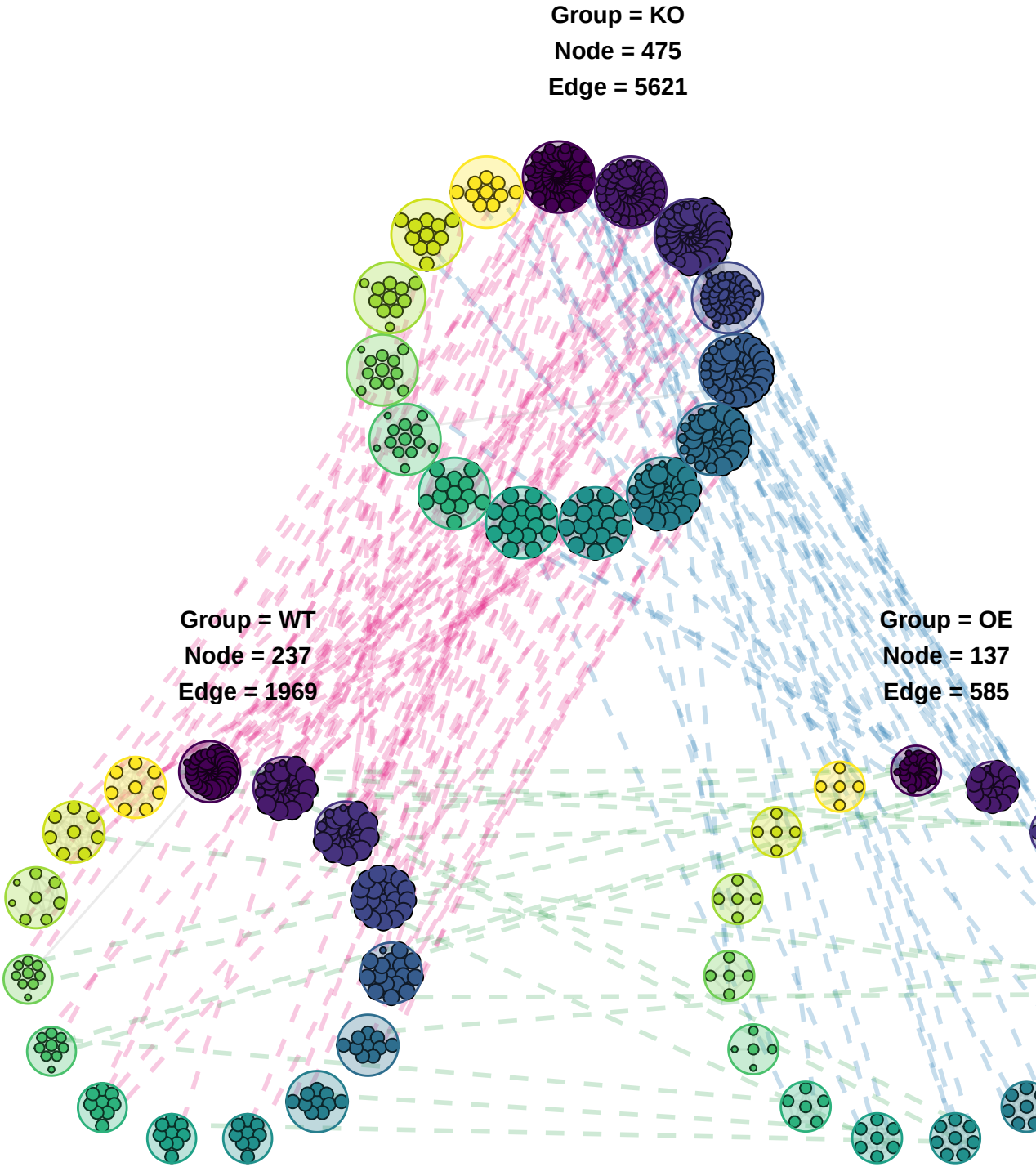
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.

```

```

p_multi_link <- out_multi_link[["p"]]
p_multi_link

```



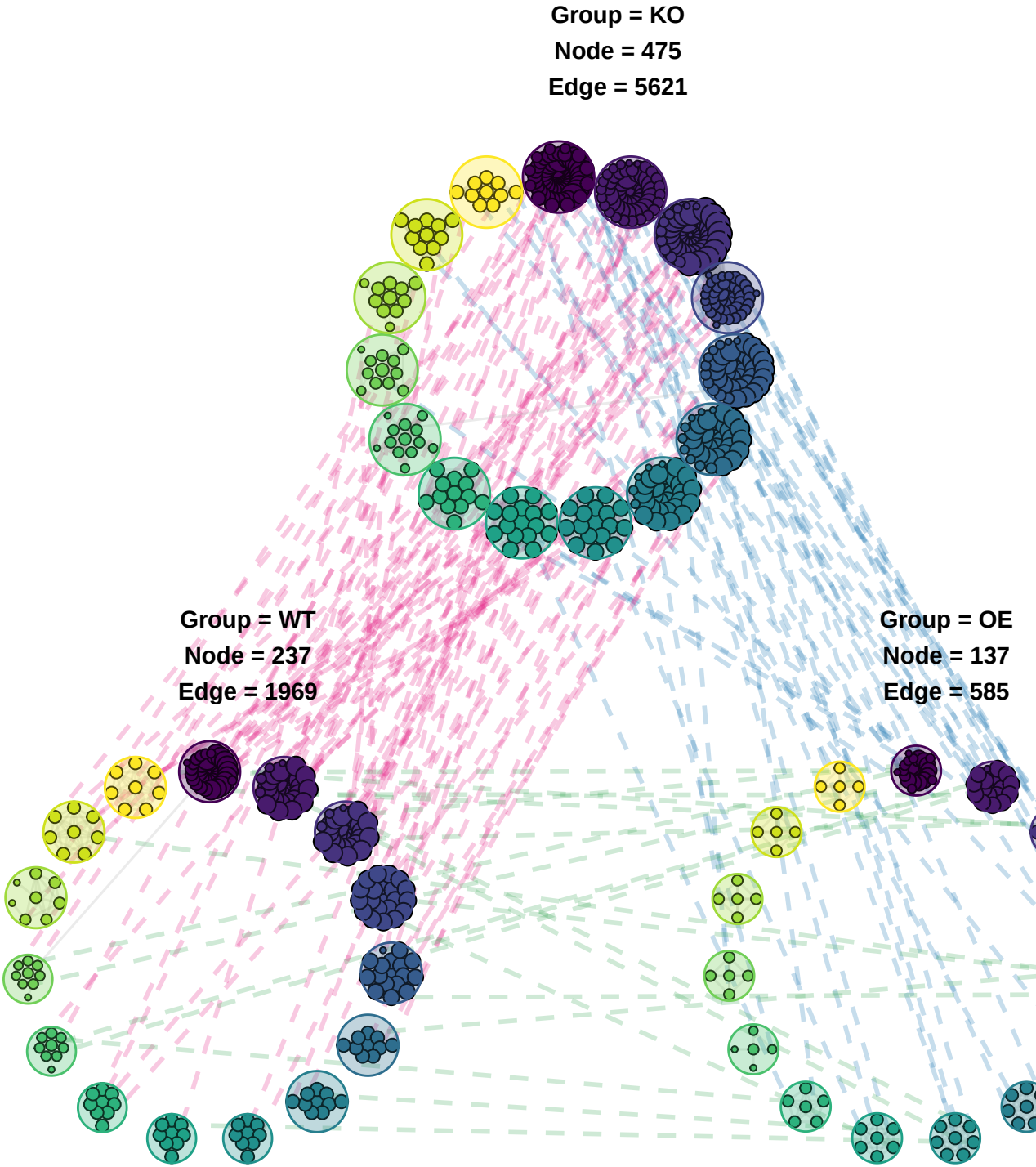
8.1.4 layout = “circular_modules_equal_gephi_layout”

```
link_level = “Node”
```

```
# ---- Duplicate of the previous chunk (kept here for reference) ----
out_multi_link <- ggNetView_multi_link(
  mat                = otu_rare_relative,
  group_info         = otu_sample,
  transfrom.method   = "none",
  r.threshold        = 0.7,
  p.threshold        = 0.05,
  method             = "WGCNA",
  cor.method         = "pearson",
  proc               = "BH",
  module.method      = "Fast_greedy",
  layout             = "circular_modules_equal_gephi_layout",
  layout.module      = "order",
  top_modules        = 15,
  scale_groups       = T,
  add_outer          = "circle",
  r                  = 5,
  jitter             = F,
  anchor_dist        = 1,
  layout_anchor_dist = 150,
  seed               = 1115,
  dropOthers         = T,
  link_level         = "Node"
)
```

```
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
```

```
p_multi_link <- out_multi_link[["p"]]
p_multi_link
```



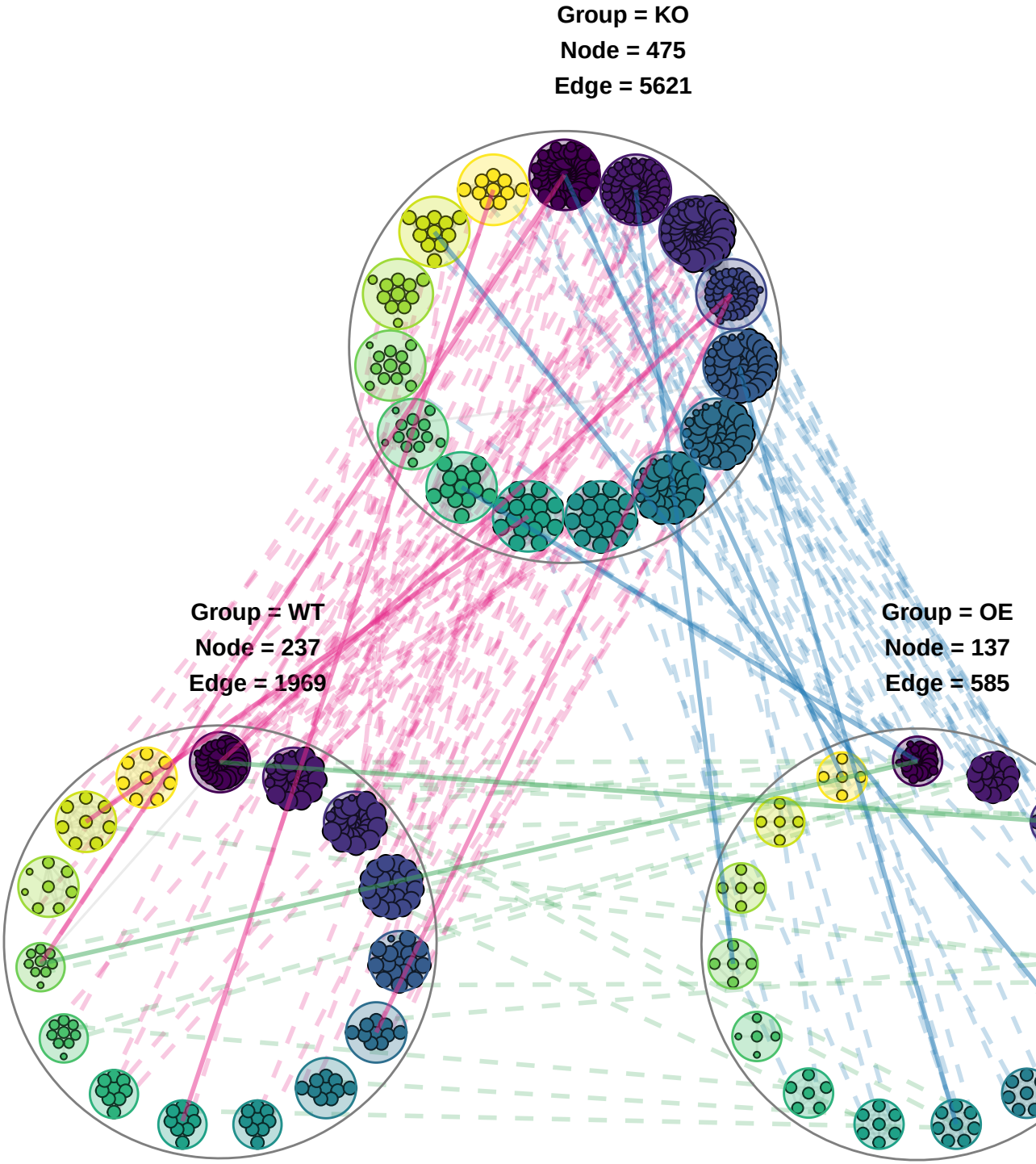
8.1.5 layout = “circular_modules_equal_gephi_layout”

link_level = “Module&Node2”

```
# ---- Cross-group links at "Module & Node2" level ----
# Instead of drawing one line per shared node, draw bundled links between
# modules AND between the corresponding nodes - useful when there are many
# shared items and per-node links become visually cluttered.
out_multi_link2 <- ggNetView_multi_link(
  mat          = otu_rare_relative,
  group_info   = otu_sample,
  transfrom.method = "none",
  r.threshold  = 0.7,
  p.threshold  = 0.05,
  method       = "WGCNA",
  cor.method   = "pearson",
  proc         = "BH",
  module.method = "Fast_greedy",
  layout       = "circular_modules_equal_gephi_layout",
  layout.module = "order",
  top_modules  = 15,
  scale_groups = T,
  add_outer    = "circle",
  r            = 5,
  jitter       = F,
  anchor_dist  = 1,
  layout_anchor_dist = 150,
  seed         = 1115,
  dropOthers   = T,
  link_level   = "Module&Node2", # bundled module-level + node-level cross-group
  add_group_outer = T           # draw a translucent boundary around each group
)
```

```
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
```

```
p_multi_link2 <- out_multi_link2[["p"]]
p_multi_link2
```



8.1.6 add group compare

```

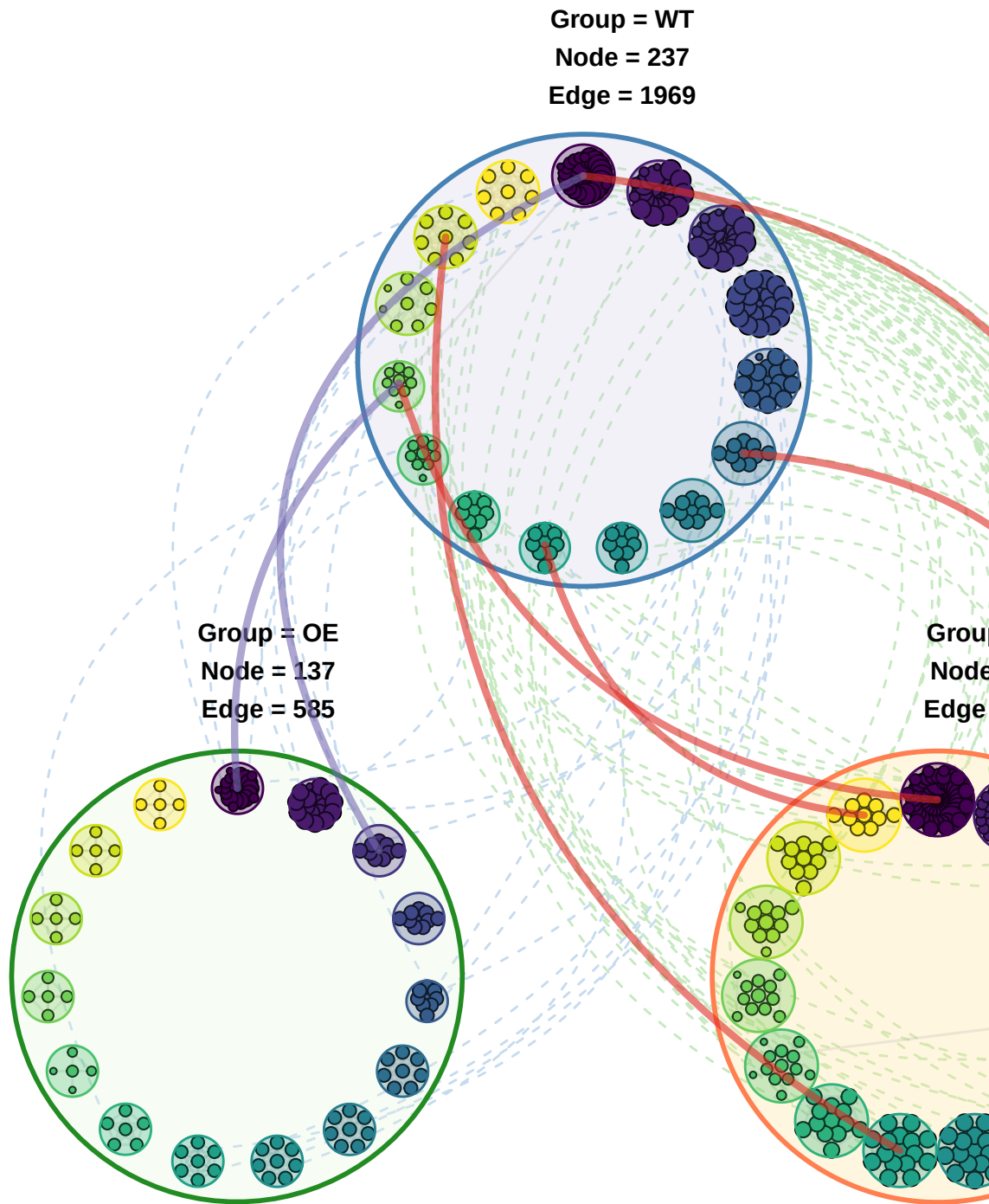
# ---- Add cross-group comparison brackets + heavy styling -----
# This example shows almost every styling knob the function exposes:
# curved cross-group links, group-comparison annotations, per-group ring
# colours, custom module/node link palettes, etc.
out_multi_link3 <- ggNetView_multi_link(
  mat                      = otu_rare_relative,
  group_info               = otu_sample,
  transform.method         = "none",
  r.threshold              = 0.7,
  p.threshold              = 0.05,
  method                   = "WGCNA",
  cor.method               = "pearson",
  proc                     = "BH",
  module.method            = "Fast_greedy",
  layout                   = "circular_modules_equal_gephi_layout",
  layout.module            = "order",
  top_modules              = 15,
  scale_groups             = T,
  add_outer                = "circle",
  r                         = 5,
  jitter                   = F,
  anchor_dist              = 1,
  layout_anchor_dist       = 150,
  seed                     = 1115,
  dropOthers               = T,
  link_level               = "Module&Node2",
  link_curve                = T,           # draw cross-group links as curves (no
  link_curve_mode           = "cross",     # curve geometry: "cross" bends links
  link_curve_adaptive_range = c(0.7, 1.5), # min/max curvature; adapts to link le
  link_curve_adaptive_bins  = 7,          # # of bins used to map link length ->
  comparisons               = T,           # draw group-comparison brackets (like
  comparisons_groups        = list(        # which group pairs to highlight
    c("WT", "KO"),
    c("WT", "OE")
  ),
  order                     = c("WT", "KO", "OE"), # MUST contain ALL unique group
  add_group_outer           = T,           # draw a coloured ring around each gro
  add_group_outer_color     = c("steelblue", "coral", "forestgreen"), # ring
  add_group_outer_fill      = c("#d0d1e6", "#fee391", "#e5f5e0"),      # ring
  add_group_outer_fill_alpha = 0.3,        # ring fill transparency
  add_group_outer_linewidth = 1,           # ring border line width
  link_color_module         = c("#de2d26", "#756bb1"), # palette for MODULE-level
  link_color_node           = c("#c7e9c0", "#c6dbef"), # palette for NODE-level

```

```
link_linewidth_module = c(1.5, 1.5), # line widths for module-level links
link_linealpha_module = 0.6,         # alpha for module-level links
link_linealpha_node   = 1,           # alpha for node-level links
link_linewidth_node   = 0.5,         # line width for node-level links
add_group_outer_expand = 2.5         # how much the outer ring is expanded outward beyond
)
```

```
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
```

```
p_multi_link3 <- out_multi_link3[["p"]]
p_multi_link3
```



8.2 multi network (4 groups)

```
# ---- Build a 4-group dataset by duplicating the first 6 samples as a fake "ko" group ----
# This is purely for demoing the multi-group layouts below; the new "ko" group
# carries the same abundance values as samples 1:6, just with renamed columns.
otu_rare_relative2 <- otu_rare_relative %>%
  dplyr::bind_cols(otu_rare_relative %>%
    dplyr::select(1:6) %>%
    purrr::set_names(paste0("ko", 1:6))) # 6 new fake samples named ko1..ko6

otu_sample2 <- otu_sample %>%
  dplyr::bind_rows(otu_sample %>%
    dplyr::slice(1:6) %>%
    dplyr::mutate(Sample = paste0("ko", 1:6),
                  Group = rep("ko", 6))) # 6 new metadata rows, Group = "ko"
```

8.2.1 group_layout = "row"

```
# ---- Place the 4 sub-networks on a single ROW ----
# `group_layout = "row"` puts all groups along one horizontal line (nrow = 1).
out_group4_1 <- ggNetView_multi_link(
  mat = otu_rare_relative2,
  group_info = otu_sample2,
  transform.method = "none",
  r.threshold = 0.7,
  p.threshold = 0.05,
  method = "WGCNA",
  cor.method = "pearson",
  proc = "BH",
  module.method = "Fast_greedy",
  layout.module = "random",
  center = T,
  top_modules = 10,
  layout = "gephi",
  group_layout = "row", # group-frame layout: "row" / "square" / "triangular"
  nrow = 1, # # of rows used when `group_layout = "row"` / group
  shrink = 0.9,
  scale_groups = T,
  add_outer = "manual", # outer frame drawn manually (uses `r`)
  r = 12.5, # radius of each per-group sub-network
  jitter = T,
  jitter_sd = 0.01,
```

```

anchor_dist           = 1.25,
layout_anchor_dist    = 175,
seed                  = 1115,
orientation           = "up",
dropOthers            = T,
angle                 = 0,
node_add              = 10,           # extra padding (in coord units) added
pointsize             = c(1, 3),     # node-size range (min, max) mapped fr
link_level            = "Module&Node2",
comparisons           = T,
comparisons_groups    = list(       # group pairs to bracket; all names MU
  c("WT", "KO"),
  c("KO", "ko"),
  c("ko", "OE")
),
order                 = c("WT", "KO", "ko", "OE"), # MUST list ALL 4 unique
add_group_outer       = T,
add_group_outer_fill_alpha = 0.3,
add_group_outer_linewidth = 1,
link_linealpha_module = 0.6,
link_linewidth_node   = 0.5,
add_group_outer_expand = 2.5
)

```

```

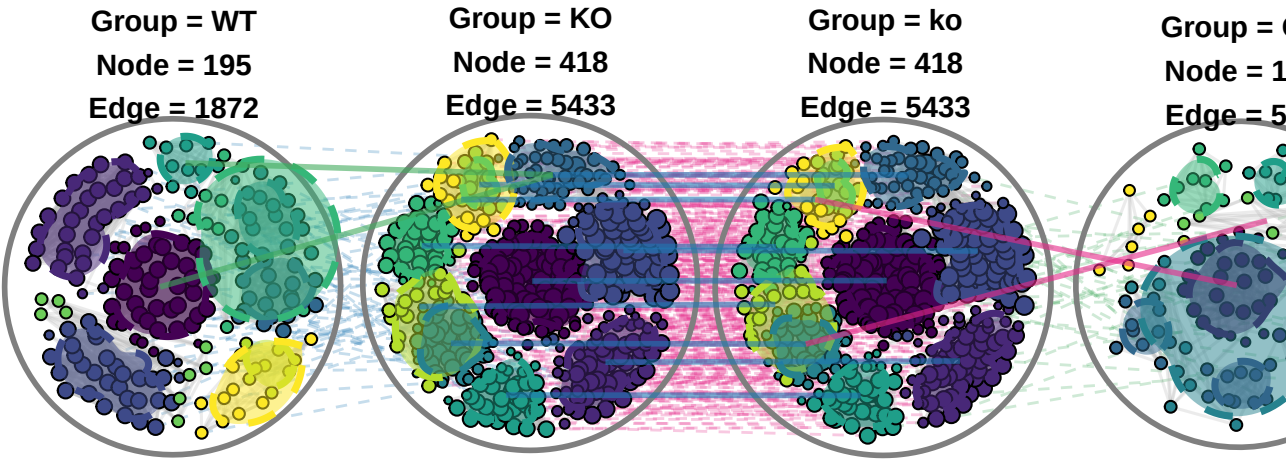
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.

```

```

p_group_4_1 <- out_group4_1[["p"]]
p_group_4_1

```



8.2.2 group_layout = “square”

```

# ---- Place the 4 sub-networks on a 2x2 SQUARE grid ----
# `group_layout = "square"` arranges groups in a square frame (4 corners for 4 groups)
out_group4_2 <- ggNetView_multi_link(
  mat                      = otu_rare_relative2,
  group_info              = otu_sample2,
  transform.method        = "none",
  r.threshold             = 0.7,
  p.threshold             = 0.05,
  method                  = "WGCNA",
  cor.method              = "pearson",
  proc                    = "BH",
  module.method           = "Fast_greedy",
  layout.module           = "order",
  center                  = T,
  top_modules             = 10,
  layout                  = "circular_modules_equal_gephi_layout",
  group_layout            = "square",      # 2x2 grid for 4 groups
  scale_groups            = T,
  add_outer               = "circle",
  r                       = 12.5,
  jitter                  = T,
  jitter_sd               = 0.01,
  anchor_dist             = 1.25,
  layout_anchor_dist      = 175,
  seed                    = 1115,
  orientation             = "up",
  dropOthers              = T,
  angle                   = 0,
  node_add                = 10,
  pointsize               = c(1, 3),
  link_level              = "Module&Node2",
  comparisons             = T,
  comparisons_groups      = list(
    c("WT", "KO"),
    c("WT", "ko"),
    c("WT", "OE")
  ),
  order                   = c("WT", "KO", "ko", "OE"),      # all 4 groups
  add_group_outer         = T,
  add_group_outer_color   = c("steelblue", "coral", "forestgreen", "#756bb1"), #
  add_group_outer_fill    = c("#deebf7", "#fee391", "#e5f5e0", "#efedf5"),      #
  add_group_outer_fill_alpha = 0.4,
  add_group_outer_linewidth = 1,

```

```

link_color_module      = c("#de2d26", "#756bb1", "#2c7fb8"),      # one colour per compo
link_color_node        = c("#c7e9c0", "#c6dbef", "#fde0dd"),      # one colour per compo
link_linewidth_module  = c(1.5, 1.5, 1.5),
link_linealpha_module  = 0.6,
link_linealpha_node    = 1,
link_linewidth_node    = 0.5,
add_group_outer_expand = 2.5
)

```

```

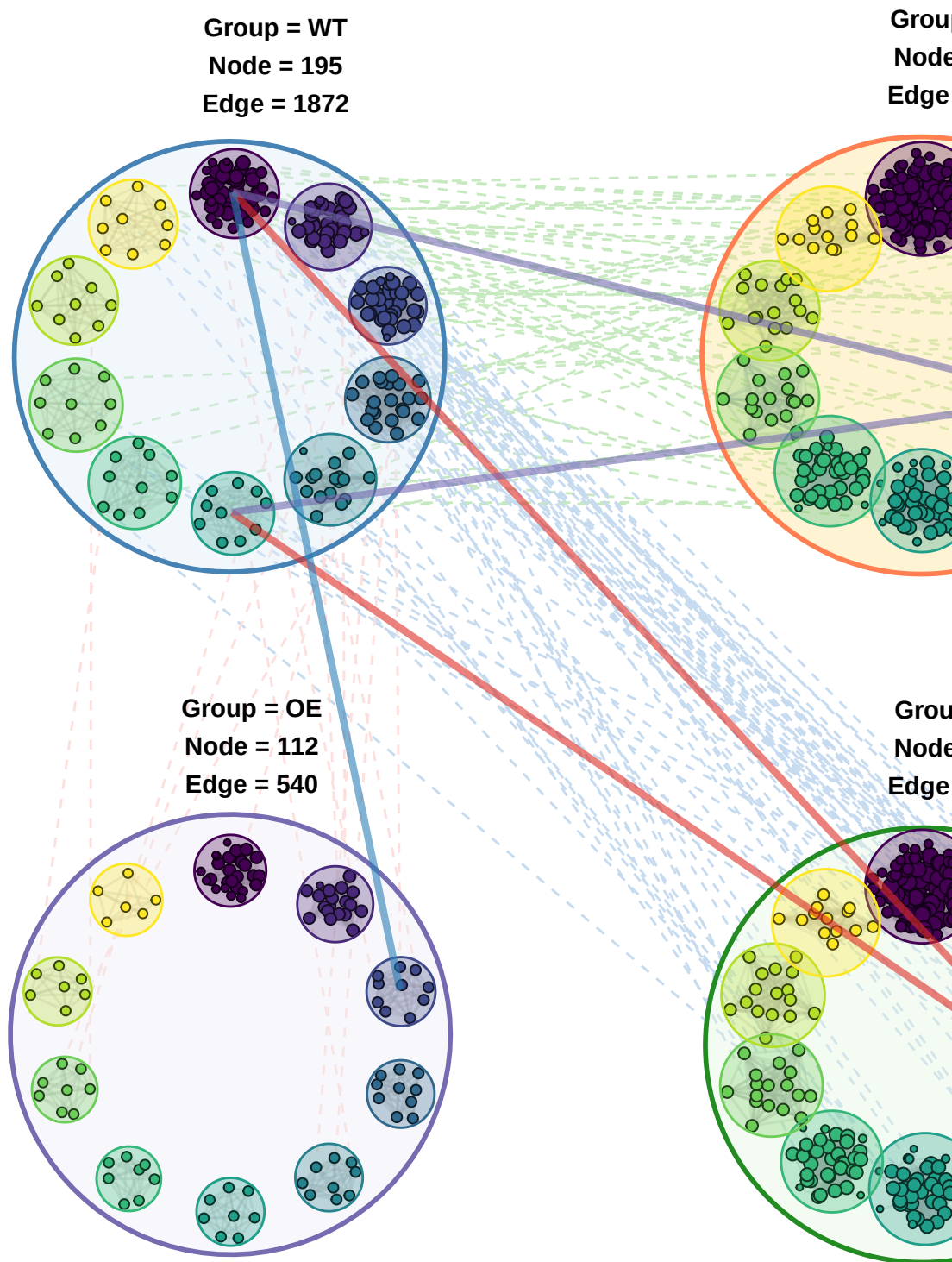
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.

```

```

p_group_4_2 <- out_group4_2[["p"]]
p_group_4_2

```



8.3 multi network (8 groups)

```
# ---- Build an 8-group dataset by repeatedly recycling otu_rare_relative columns ----
# This is purely for demoing the 8-group layouts below; abundance values are
# borrowed from the original matrix and renamed so each fake group has 6 samples.
otu_rare_relative3 <- otu_rare_relative %>%
  dplyr::bind_cols(otu_rare_relative %>% dplyr::select(1:6) %>% purrr::set_names(paste0("ko",
  dplyr::bind_cols(otu_rare_relative %>% dplyr::select(7:12) %>% purrr::set_names(paste0("oe",
  dplyr::bind_cols(otu_rare_relative %>% dplyr::select(13:18) %>% purrr::set_names(paste0("wt",
  dplyr::bind_cols(otu_rare_relative %>% dplyr::select(1:6) %>% purrr::set_names(paste0("aaa",
  dplyr::bind_cols(otu_rare_relative %>% dplyr::select(7:12) %>% purrr::set_names(paste0("bbb",

otu_sample3 <- otu_sample %>%
  dplyr::bind_rows(otu_sample %>% dplyr::slice(1:6) %>%
    dplyr::mutate(Sample = paste0("ko", 1:6), Group = rep("ko", 6))) %>%
  dplyr::bind_rows(otu_sample %>% dplyr::slice(1:6) %>%
    dplyr::mutate(Sample = paste0("oe", 1:6), Group = rep("oe", 6))) %>%
  dplyr::bind_rows(otu_sample %>% dplyr::slice(1:6) %>%
    dplyr::mutate(Sample = paste0("wt", 1:6), Group = rep("wt", 6))) %>%
  dplyr::bind_rows(otu_sample %>% dplyr::slice(1:6) %>%
    dplyr::mutate(Sample = paste0("aaa", 1:6), Group = rep("aaa", 6))) %>%
  dplyr::bind_rows(otu_sample %>% dplyr::slice(1:6) %>%
    dplyr::mutate(Sample = paste0("bbb", 1:6), Group = rep("bbb", 6)))
# Final groups in otu_sample3 (8 total, CASE-SENSITIVE):
# "WT", "KO", "OE" (from the original 18 samples)
# "ko", "oe", "wt", "aaa", "bbb" (from the bind_rows additions)
```

8.3.1 group_layout = "triangle_down"

```
# ---- Place the 8 sub-networks on a downward-pointing TRIANGLE frame ----
# `group_layout = "triangle_down"` arranges groups along a triangular outline
# with the apex pointing down. Useful for emphasising hierarchy or a "funnel"
# narrative across groups.
out_group4_1 <- ggNetView_multi_link(
  mat = otu_rare_relative3,
  group_info = otu_sample3,
  transfrom.method = "none",
  r.threshold = 0.7,
  p.threshold = 0.05,
  method = "WGCNA",
  cor.method = "pearson",
  proc = "BH",
```

```

module.method                = "Fast_greedy",
layout.module                 = "random",
center                        = T,
top_modules                   = 10,
layout                        = "gephi",
group_layout                  = "triangle_down",
nrow                           = 1,
shrink                        = 0.9,
scale_groups                  = T,
add_outer                     = "manual",
r                              = 5,
jitter                       = T,
jitter_sd                     = 0.01,
anchor_dist                   = 2,
layout_anchor_dist            = 250,
seed                          = 1115,
orientation                   = "up",
dropOthers                    = T,
angle                         = 0,
node_add                      = 10,
pointsize                     = c(1, 3),
link_level                    = "Module&Node2",
comparisons                   = T,
comparisons_groups            = list(c("WT", "wt"),
                                     c("wt", "KO"),
                                     c("KO", "ko"),
                                     c("ko", "OE"),
                                     c("OE", "oe"),
                                     c("oe", "aaa"),
                                     c("aaa", "bbb")),
order                         = c("WT", "wt", "KO", "ko", "OE", "oe", "aaa", "bbb"),
add_group_outer               = T,
add_group_outer_fill_alpha    = 0.3,
add_group_outer_linewidth     = 1,
link_linealpha_module         = 0.6,
link_linewidth_node           = 0.5,
add_group_outer_expand        = 2.5
)

```

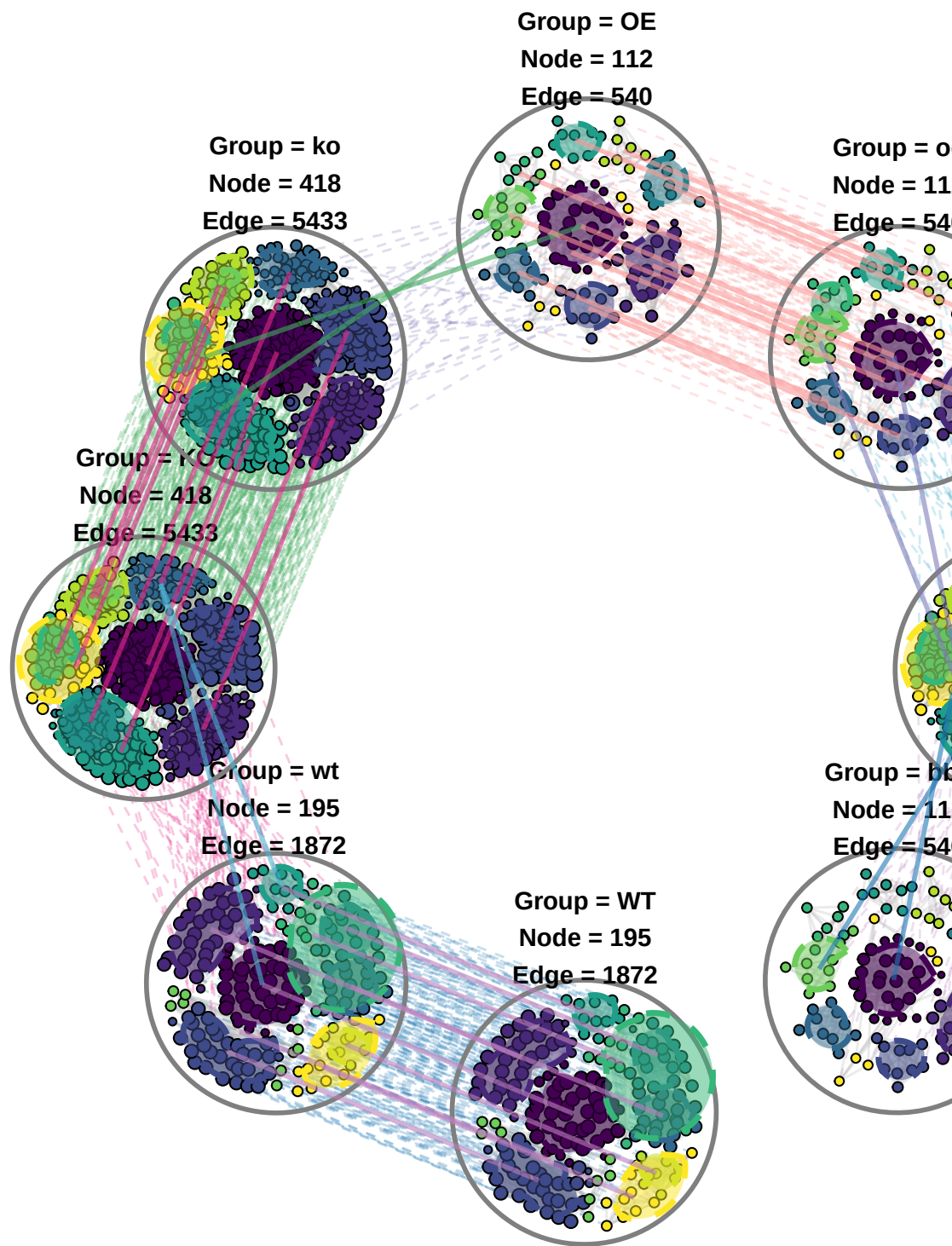
```

## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.

```

```
## Scale for size is already present.  
## Adding another scale for size, which will replace the existing scale.  
## Scale for size is already present.  
## Adding another scale for size, which will replace the existing scale.  
## Scale for size is already present.  
## Adding another scale for size, which will replace the existing scale.  
## Scale for size is already present.  
## Adding another scale for size, which will replace the existing scale.
```

```
p_group_4_1 <- out_group4_1[["p"]]  
p_group_4_1
```



8.3.2 group_layout = "snake_vertical"

```

# ---- Place sub-networks in a vertical SNAKE (zig-zag) frame ----
# `group_layout = "snake_vertical"` runs groups down-up-down... along a vertical
# meander. Useful when you have many groups and want them to fit a tall canvas.
out_group4_1 <- ggNetView_multi_link(
  mat                      = otu_rare_relative3,    # NOTE: this is the 4-group dataset
  group_info               = otu_sample3,          # NOTE: this is the 4-group metadata
  transform.method         = "none",
  r.threshold              = 0.7,
  p.threshold              = 0.05,
  method                   = "WGCNA",
  cor.method               = "pearson",
  proc                     = "BH",
  module.method            = "Fast_greedy",
  layout.module            = "random",
  center                   = T,
  top_modules              = 10,
  layout                   = "gephi",
  group_layout             = "snake_vertical",
  nrow                     = 1,
  shrink                   = 0.9,
  scale_groups             = T,
  add_outer                = "manual",
  r                         = 10,
  jitter                   = T,
  jitter_sd                = 0.01,
  anchor_dist              = 1.25,
  layout_anchor_dist       = 175,
  seed                     = 1115,
  orientation              = "up",
  dropOthers               = T,
  angle                    = 0,
  node_add                 = 10,
  pointsize                = c(1, 3),
  link_level               = "Module&Node2",
  comparisons              = T,
  comparisons_groups       = list(c("WT", "wt"),
                                   c("wt", "KO"),
                                   c("KO", "ko"),
                                   c("ko", "OE"),
                                   c("OE", "oe"),
                                   c("oe", "aaa"),
                                   c("aaa", "bbb")),
  order                    = c("WT", "wt", "KO", "ko", "OE", "oe", "aaa", "bbb"), # 8 names,

```

```

add_group_outer          = T,
add_group_outer_fill_alpha = 0.3,
add_group_outer_linewidth = 1,
link_linealpha_module     = 0.6,
link_linewidth_node       = 0.5,
add_group_outer_expand    = 2.5
)

```

```

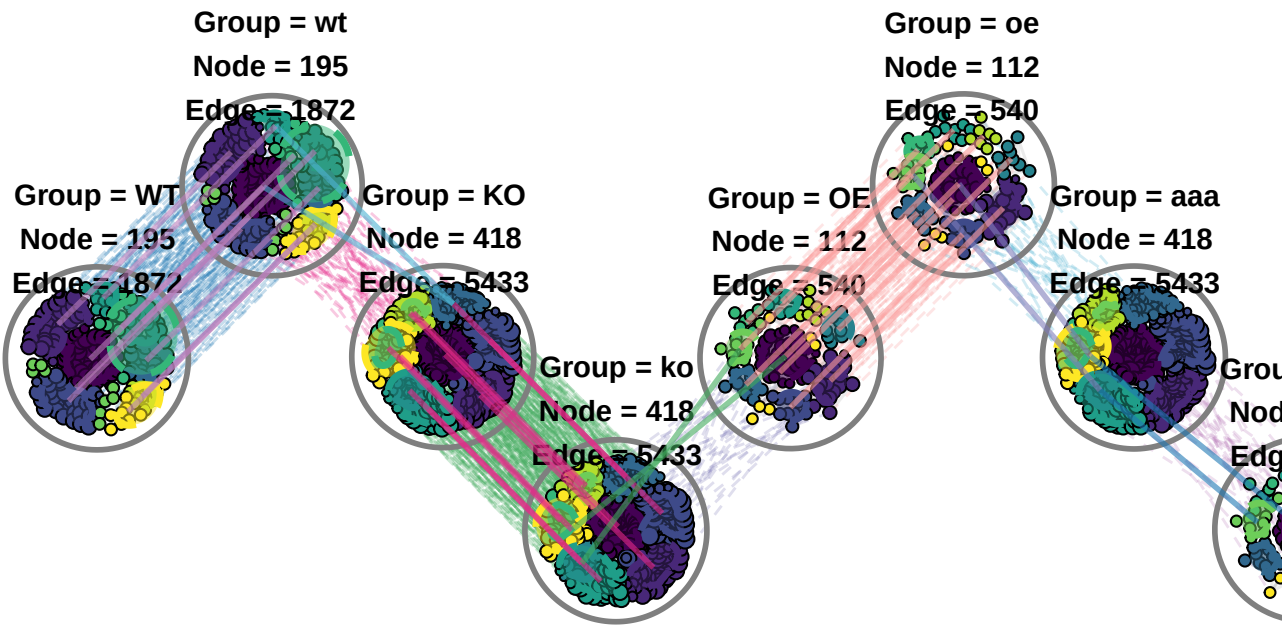
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.
## Scale for size is already present.
## Adding another scale for size, which will replace the existing scale.

```

```

p_group_4_1 <- out_group4_1[["p"]]
p_group_4_1

```



Chapter 9

Network & Environment

9.1 Network Environment

Chapter 10

Multi-omics network analysis

10.1 Multi-omics

Chapter 11

Gallery of Reproducible Examples

11.1 Water

11.2 WGCNA